

ARIN 5201 & MAIE 5421

Machine Learning for Computer Vision

Wenhan LUO

10/2/2026

Content

Topic	Week
Machine Learning and Computer Vision Basics	
Introduction to Computer Vision	1
Machine Learning Elements	2
Deep Learning	3-4
 Advanced Topics in Computer Vision	
Image Classification	5-6
Detection & Segmentation	7
Self Attention & Transformer	8
Generative Models	9-10
Multi-modal Learning	11
Final project representation	12-13

Image Classification

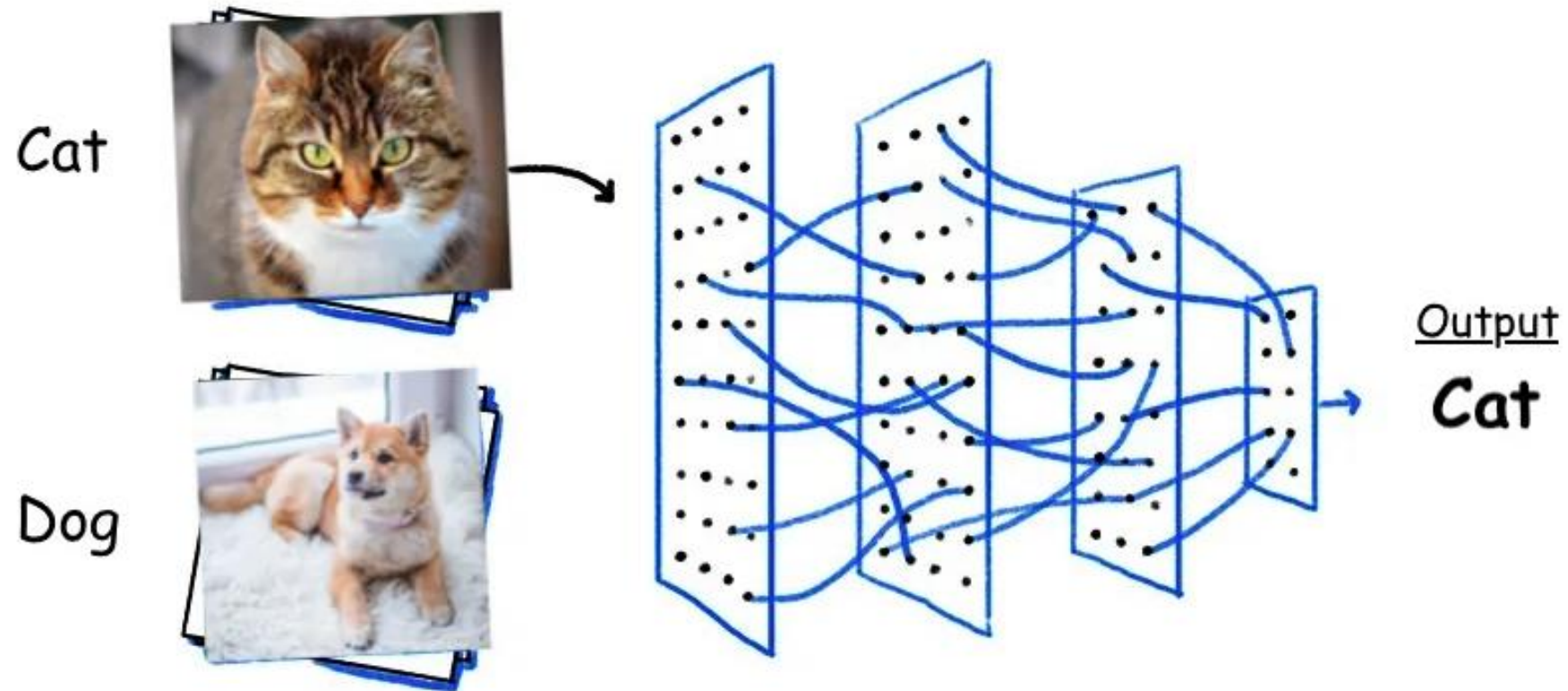


Image Classification

- Traditional Methods (Pre-Deep Learning)
 - Key Handcrafted Feature Extraction Methods
 - Local Feature Descriptors
 - Global Feature Descriptors
 - Bag-of-Visual-Words (BoVW) Representation
 - Limitations on Handcrafted Features
- Image Classification in Deep Learning Era
 - From Perceptrons to CNNs
 - Key CNN Architectures
 - Lightweight Models

Image Classification: A core task in Computer Vision



This image by Nikita is
licensed under [CC-BY 2.0](#)

(assume given a set of labels)
{dog, cat, truck, plane, ...}



cat
dog
bird
deer
truck

Image Classification: Paradigm Comparison

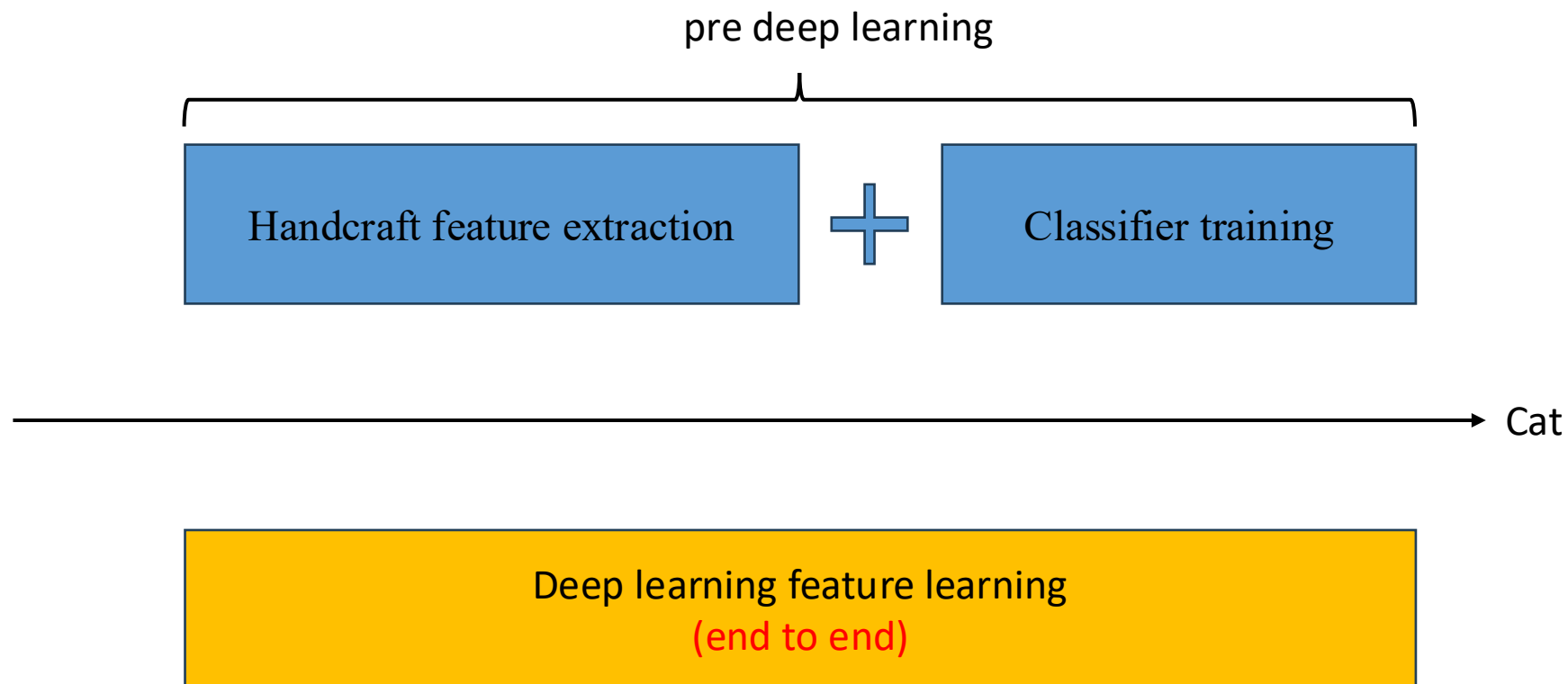
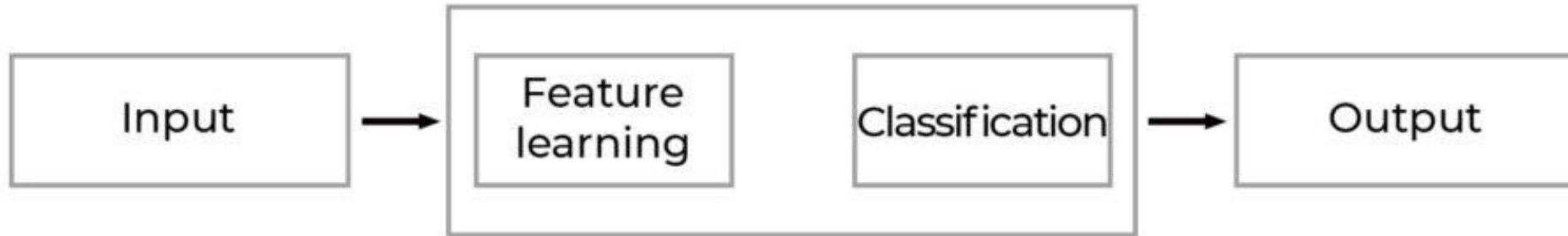


Image Classification: Paradigm Comparison

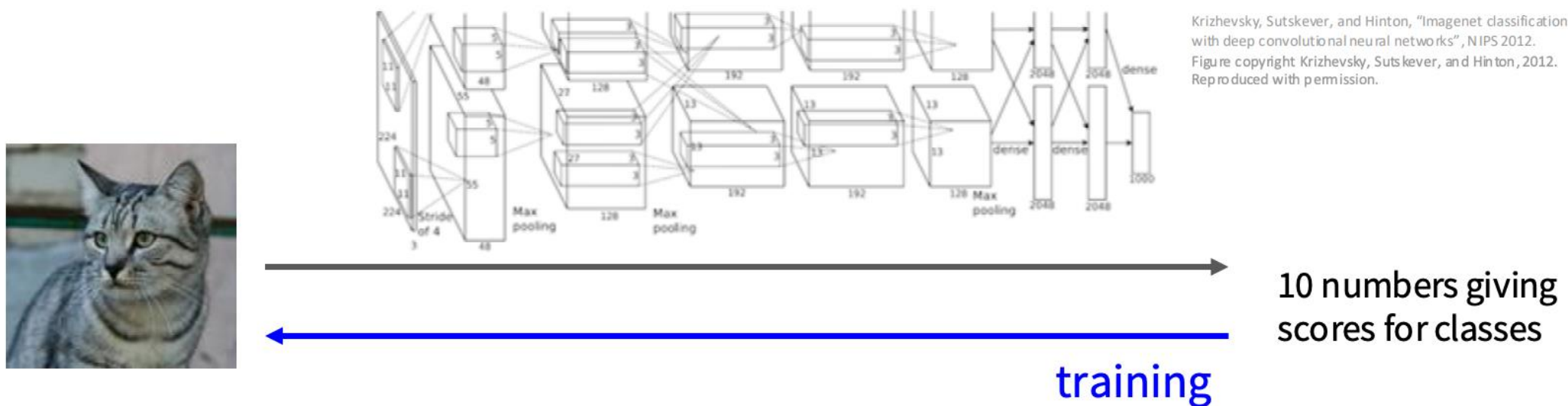
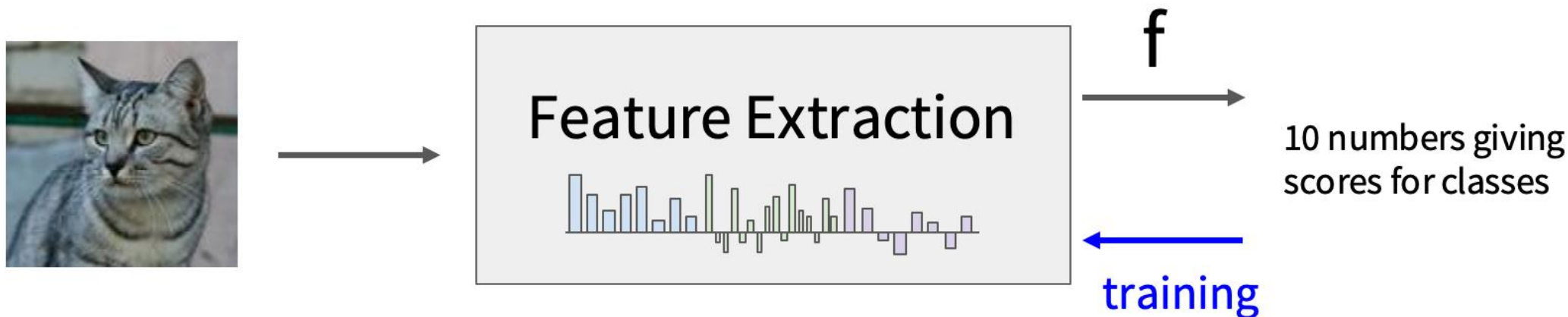


Feature Engineering
Traditional Approach



Feature Engineering
Deep Learning Approach

Image Classification: Paradigm Comparison



Handcrafted Feature Extraction: Local Features

Feature is important:
the development of CV is the advancement of feature learning

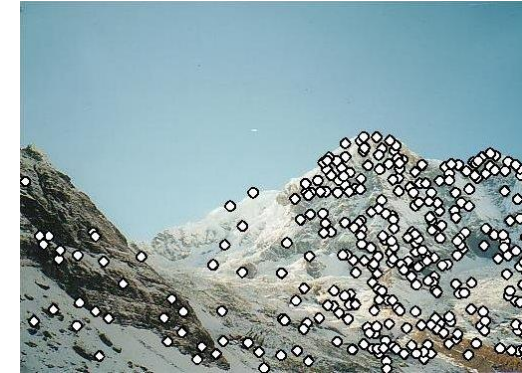
In [computer vision](#) and [image processing](#), a **feature** is a piece of information about the content of an image; typically about whether a certain region of the image has certain properties. Features may be specific structures in the image such as points, edges or objects. Features may also be the result of a general [neighborhood operation](#) or **feature detection** applied to the image.

Image Classification

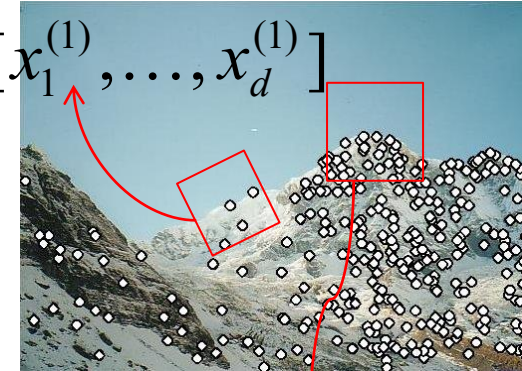
- Traditional Methods (Pre-Deep Learning)
 - Key Handcrafted Feature Extraction Methods
 - Local Feature Descriptors
 - Global Feature Descriptors
 - Bag-of-Visual-Words (BoVW) Representation
 - Limitations on Handcrafted Features
- Image Classification in Deep Learning Era
 - From Perceptrons to CNNs
 - Key CNN Architectures
 - Lightweight Models

Local Features: Main Components

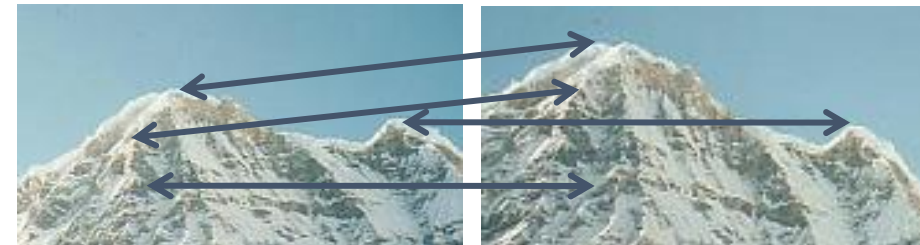
- 1) **Detection:** Identify the interest points
- 2) **Description:** Extract vector feature descriptor surrounding each interest point.
- 3) **Matching:** Determine correspondence between descriptors in two views



$$\mathbf{x}_1 = [x_1^{(1)}, \dots, x_d^{(1)}]$$

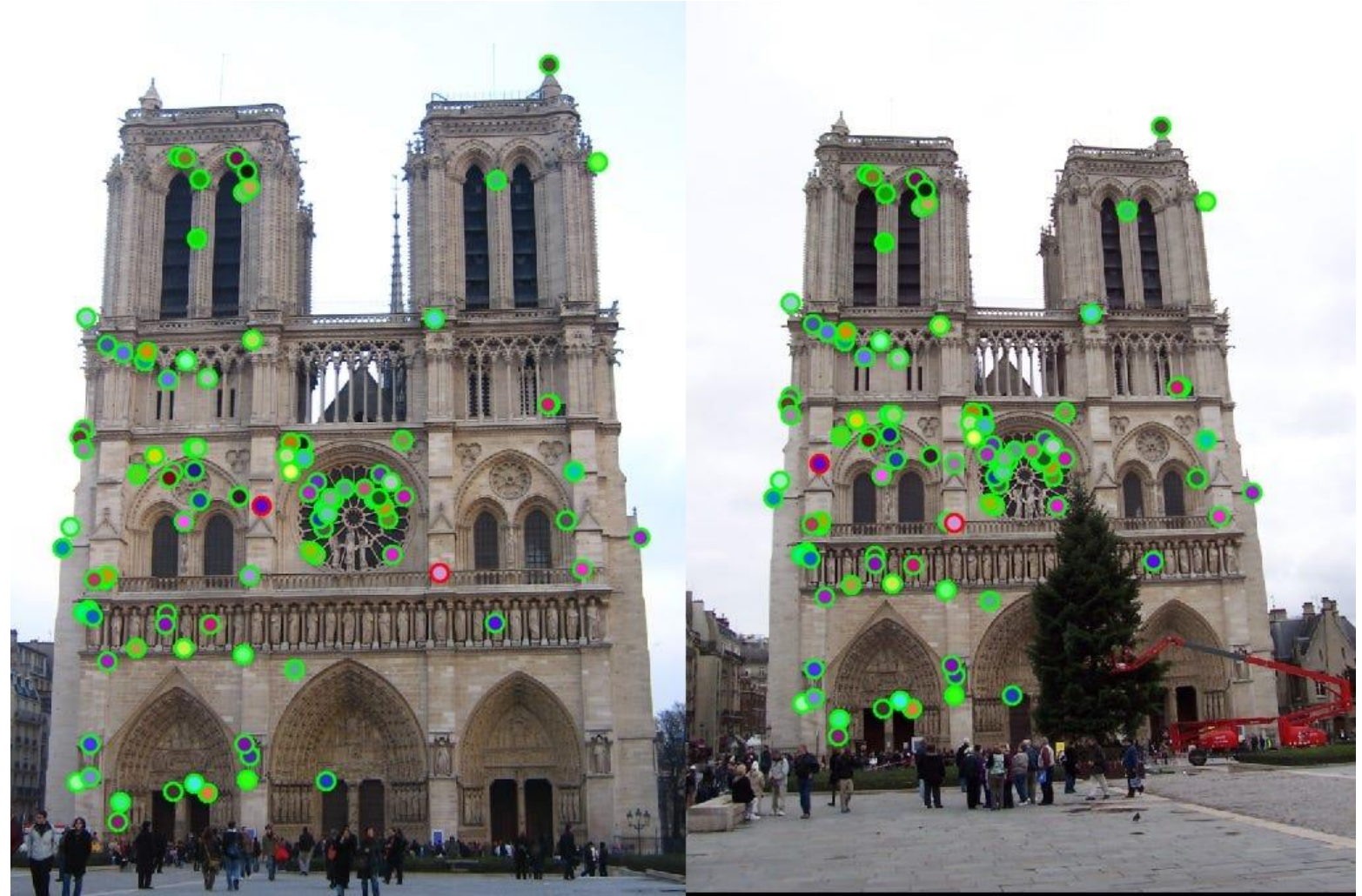


$$\mathbf{x}_2 = [x_1^{(2)}, \dots, x_d^{(2)}]$$



Local Features: Main Components

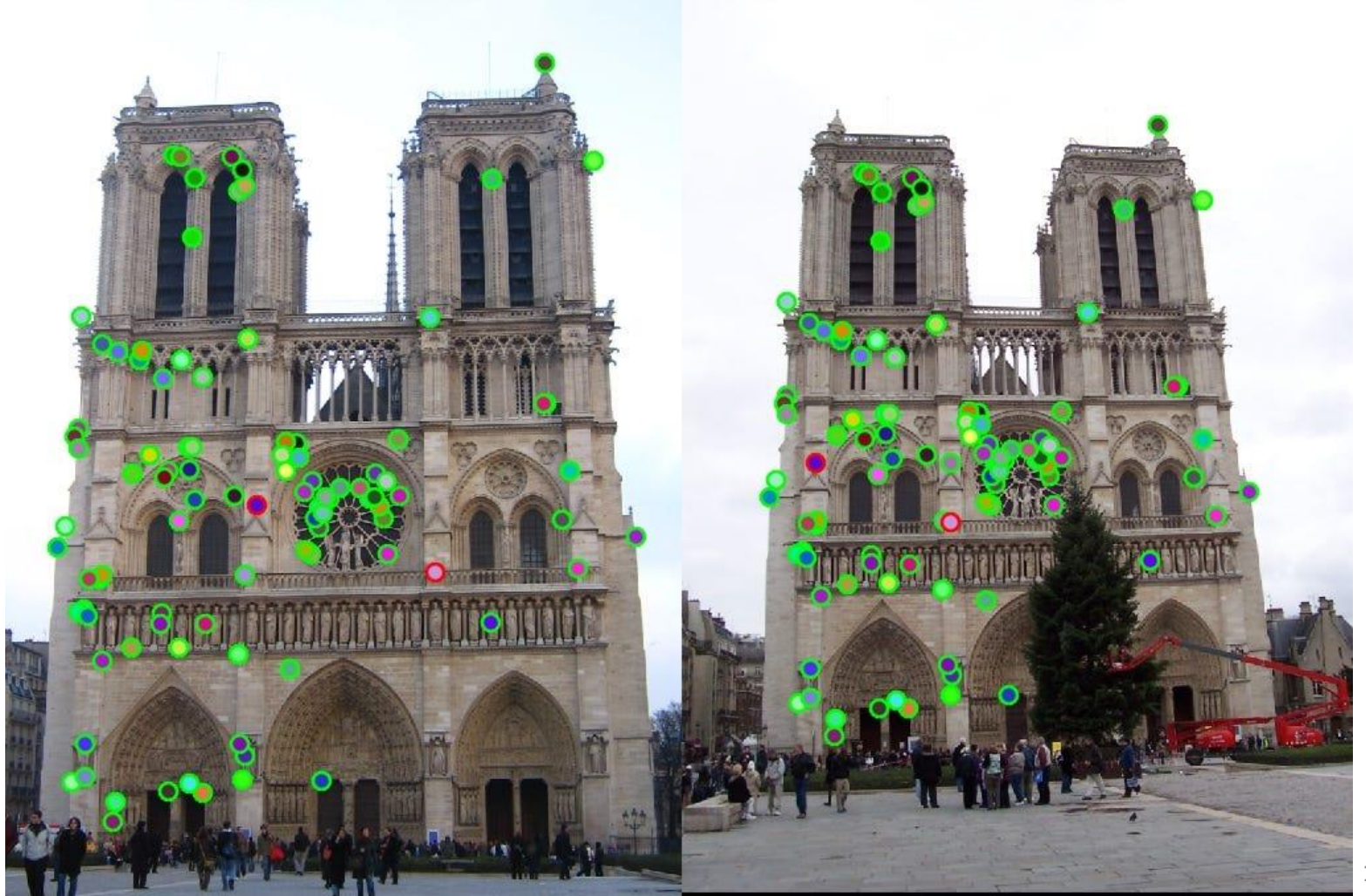
Interest point or **Feature Point** is the point which is expressive in texture. Interest point is the point at which the direction of the boundary of the object changes abruptly or intersection point between two or more edge segments.



Local Features: Main Components

Properties Of Interest Point

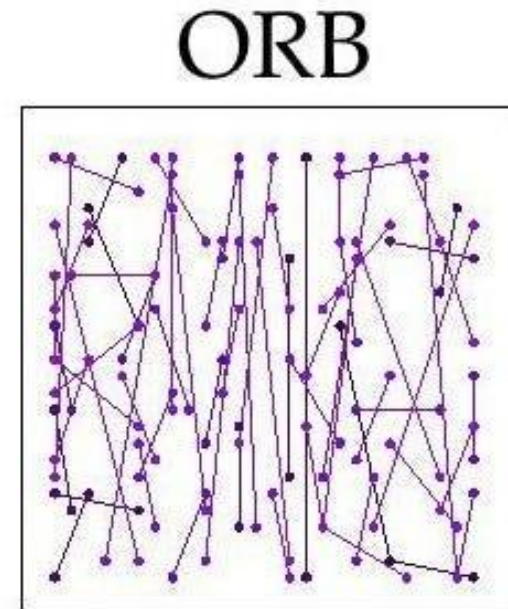
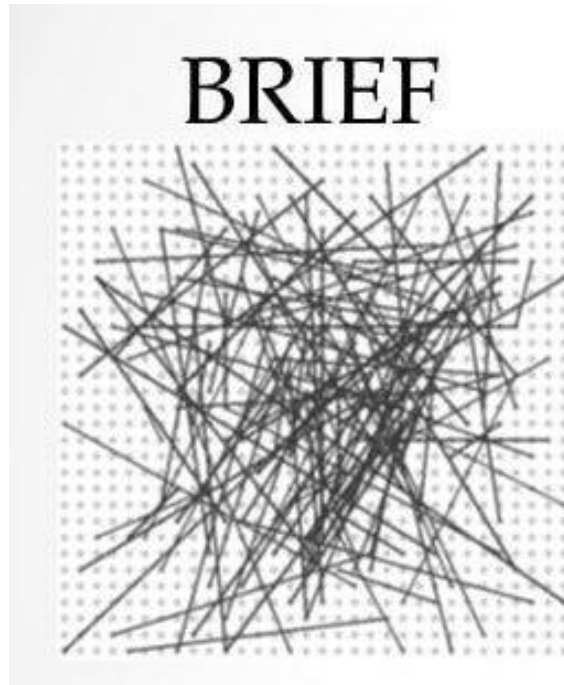
- 1) It has a well-defined *position* in image space or well localized.
- 2) It is *stable* under local and global perturbations in the image domain as illumination/brightness variations, such that the interest points can be reliably computed with a high degree of *repeatability*.
- 3) Should provide efficient detection.



Local Features: Main Components

Feature Descriptor

A feature descriptor is an algorithm which takes an image and outputs feature descriptors/feature vectors. Feature descriptors encode interesting information into a series of numbers and act as a sort of numerical “fingerprint” that can be used to differentiate one feature from another.

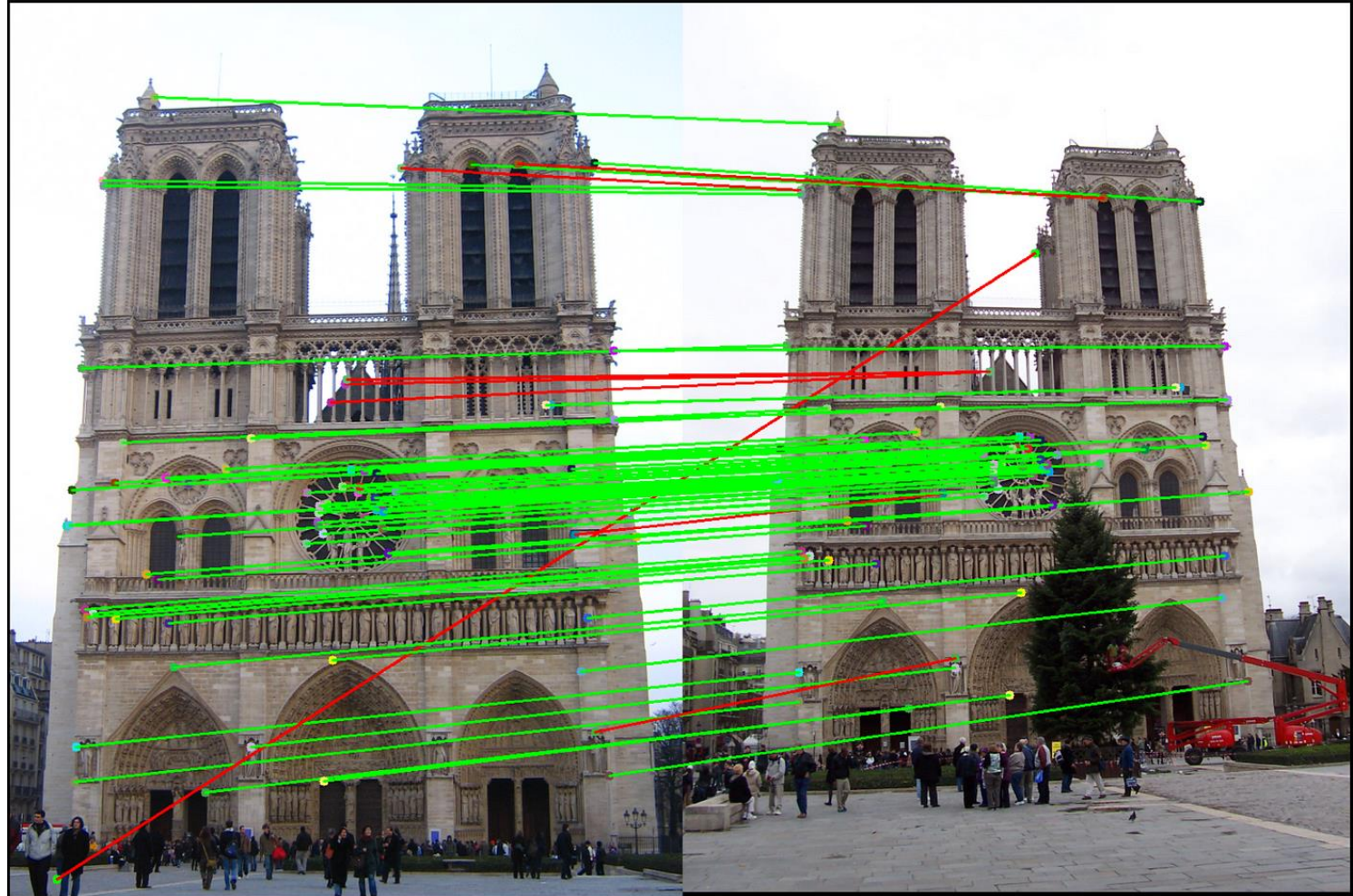


Ideally, this information would be invariant under image transformation, so we can find the feature again even if the image is transformed in some way. After detecting interest point we go on to compute a descriptor for every one of them

Local Features: Main Components

Features Matching

Features matching or generally image matching, a part of many computer vision applications such as image registration, camera calibration and object recognition, is the task of establishing correspondences between two images of the same scene/object. A common approach to image matching consists of detecting a set of interest points each associated with image descriptors from image data. Once the features and their descriptors have been extracted from two or more images, the next step is to establish some preliminary feature matches between these images.



Scale Invariant Feature Transform (SIFT)

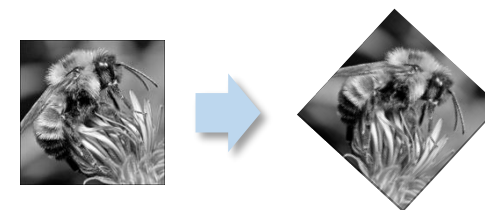
Invariance vs. discriminability

- Invariance:
 - Descriptor shouldn't change even if image is transformed
- Discriminability:
 - Descriptor should be highly unique for each point

Scale Invariant Feature Transform (SIFT)

- Geometric

Rotation



Scale

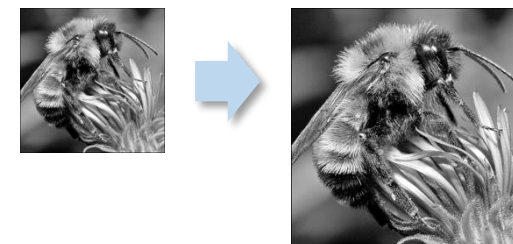


Image transformations revisited

- Photometric
Intensity change



Scale Invariant Feature Transform (SIFT)

Key Properties of SIFT

- ✓ **Scale Invariance** – Detects features at different image scales.
- ✓ **Rotation Invariance** – Aligns features based on dominant gradient orientation.
- ✓ **Illumination & Affine Robustness** – Uses gradient-based descriptors, making it less sensitive to lighting changes.
- ✓ **Distinctiveness** – Each feature is represented by a **128-dimensional vector**, allowing high matching accuracy.

Scale Invariant Feature Transform (SIFT)

How SIFT Works (Step-by-Step)

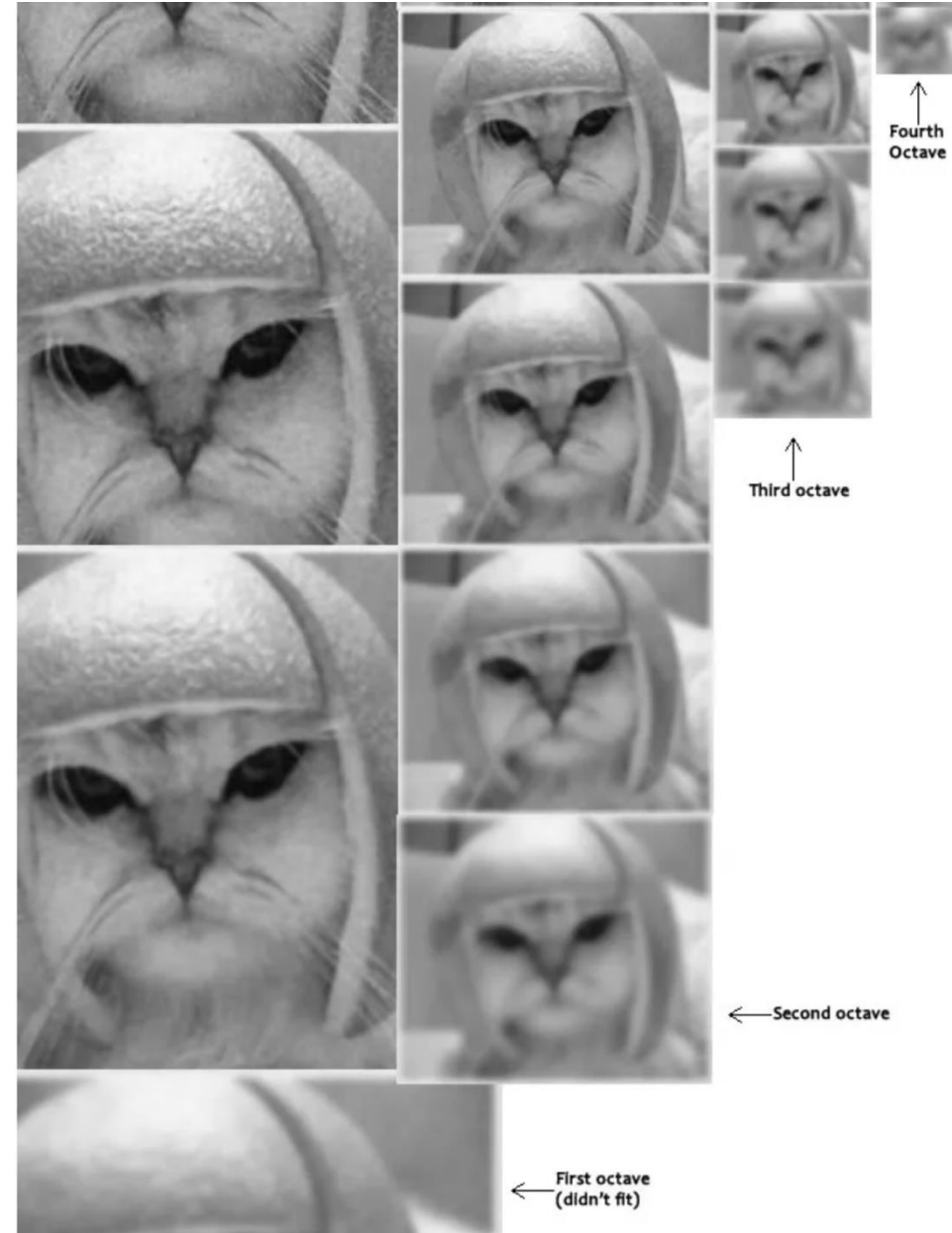
Step 1: Scale-Space Extrema Detection

•Gaussian Pyramid Construction

- The image is blurred at different scales (using Gaussian filters) to create multiple **octaves** (groups of scaled images). $L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$
- Each octave contains progressively downsampled images.

•Difference of Gaussians (DoG)

- Compute the difference between consecutive blurred images to detect **potential keypoints** (local maxima/minima in scale-space).

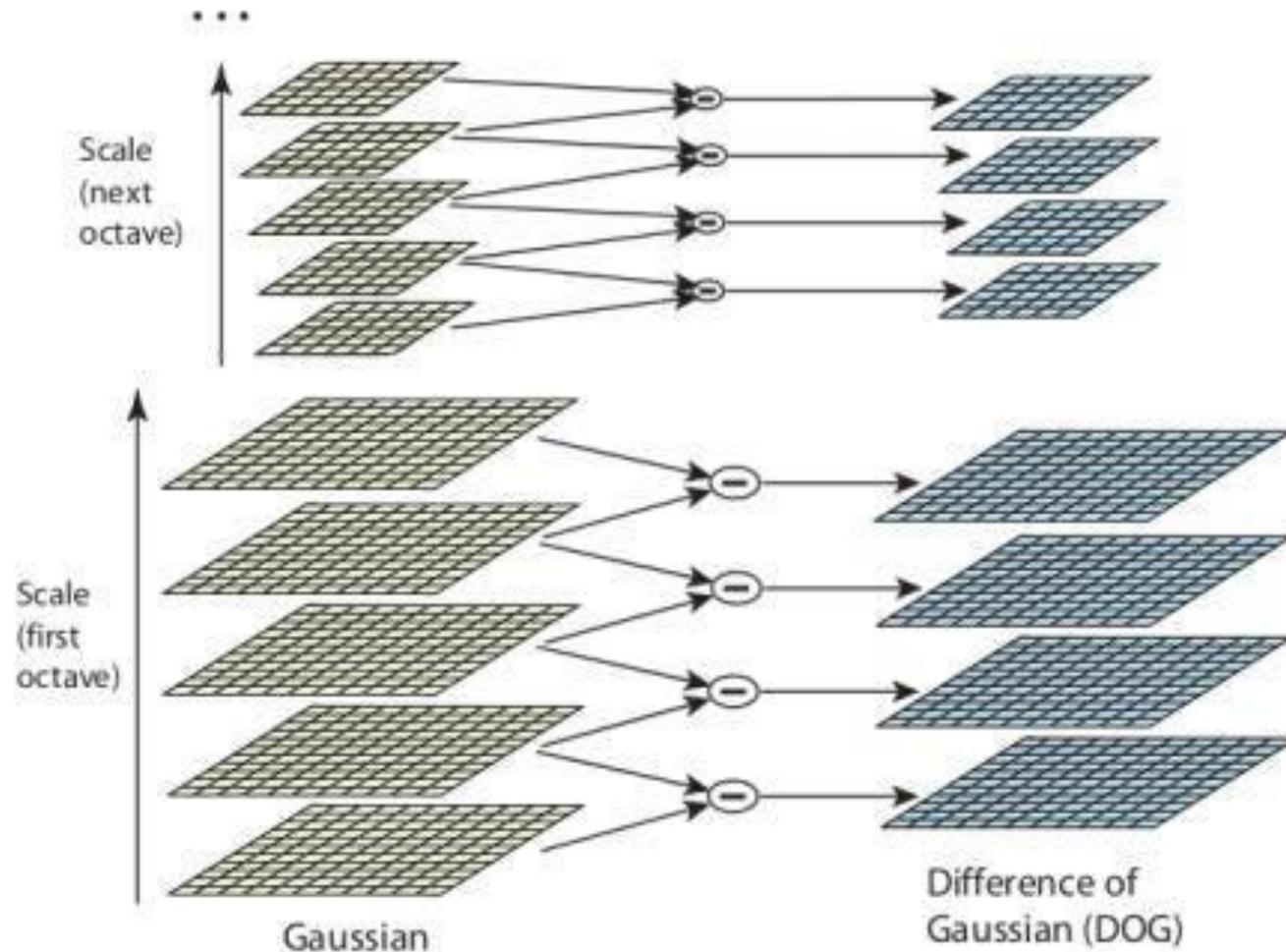


Scale Invariant Feature Transform (SIFT)

DoG

The difference of Gaussian is obtained as the difference of Gaussian blurring of an image with two different σ , let it be σ and $k\sigma$. This process is done for different octaves of the image in the Gaussian Pyramid.

$$D(x,y,\sigma)=L(x,y,k\sigma)-L(x,y,\sigma)$$



Scale Invariant Feature Transform (SIFT)

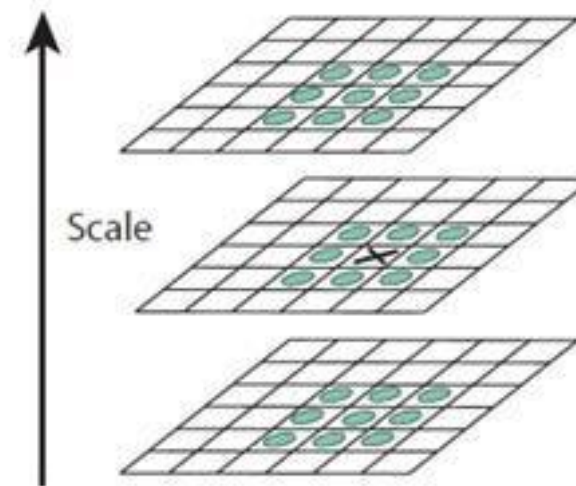
Step 2: Keypoint Localization

•Remove Low-Contrast Points

- Only keep keypoints with strong DoG responses (eliminates noise).

•Eliminate Edge Responses

- Uses **Hessian matrix** to discard unstable edge-like points (similar to Harris corner detection).



One pixel in an image is compared with its 8 neighbors as well as 9 pixels in the next scale and 9 pixels in previous scales. This way, a total of 26 checks are made. If it is a local extrema, it is a potential keypoint.

Scale Invariant Feature Transform (SIFT)

Step 2: Keypoint Localization

• Remove Low-Contrast Points

- Only keep keypoints with strong DoG responses (eliminates noise).

• Eliminate Edge Responses

- Uses **Hessian matrix** to discard unstable edge-like points (similar to Harris corner detection).

- Reject flats:

$$\square \quad |D(\hat{\mathbf{x}})| < 0.03$$

- Reject edges:

$$\mathbf{H} = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$$

Let α be the eigenvalue with larger magnitude and β the smaller.

$$\text{Tr}(\mathbf{H}) = D_{xx} + D_{yy} = \alpha + \beta,$$

$$\text{Det}(\mathbf{H}) = D_{xx}D_{yy} - (D_{xy})^2 = \alpha\beta.$$

Let $r = \alpha/\beta$.
So $\alpha = r\beta$

$$\frac{\text{Tr}(\mathbf{H})^2}{\text{Det}(\mathbf{H})} = \frac{(\alpha + \beta)^2}{\alpha\beta} = \frac{(r\beta + \beta)^2}{r\beta^2} = \frac{(r + 1)^2}{r},$$

$(r+1)^2/r$ is at a min when the 2 eigenvalues are equal.

$$\square \quad r < 10$$

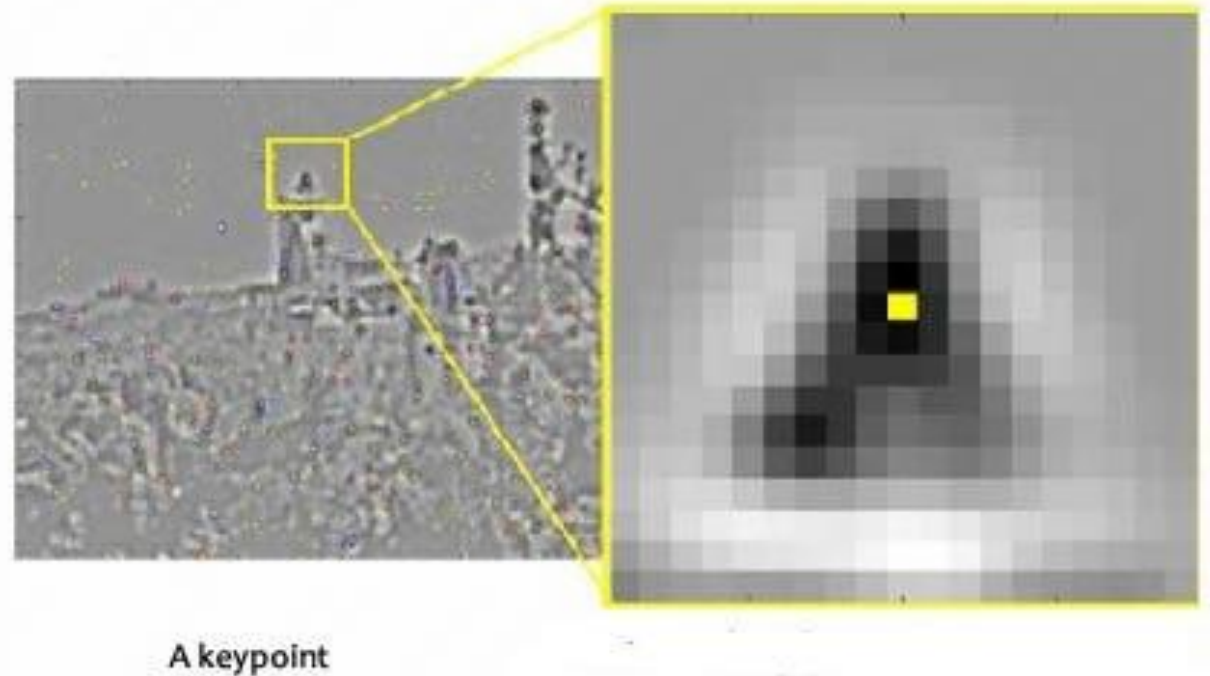
if the intensity at this extrema is less than a threshold value (0.03 as per the paper), it is rejected. DoG has a higher response for edges, so edges also need to be removed. A 2x2 Hessian matrix ($\mathbf{H}$) is used to compute the principal curvature

Scale Invariant Feature Transform (SIFT)

Step 3: Orientation Assignment

•Compute Dominant Orientation

- In a patch around each keypoint, calculate gradient directions and magnitudes.
- Build a **36-bin orientation histogram** (covering 360°).
- The peak direction is assigned as the **keypoint's dominant orientation** (ensures rotation invariance).



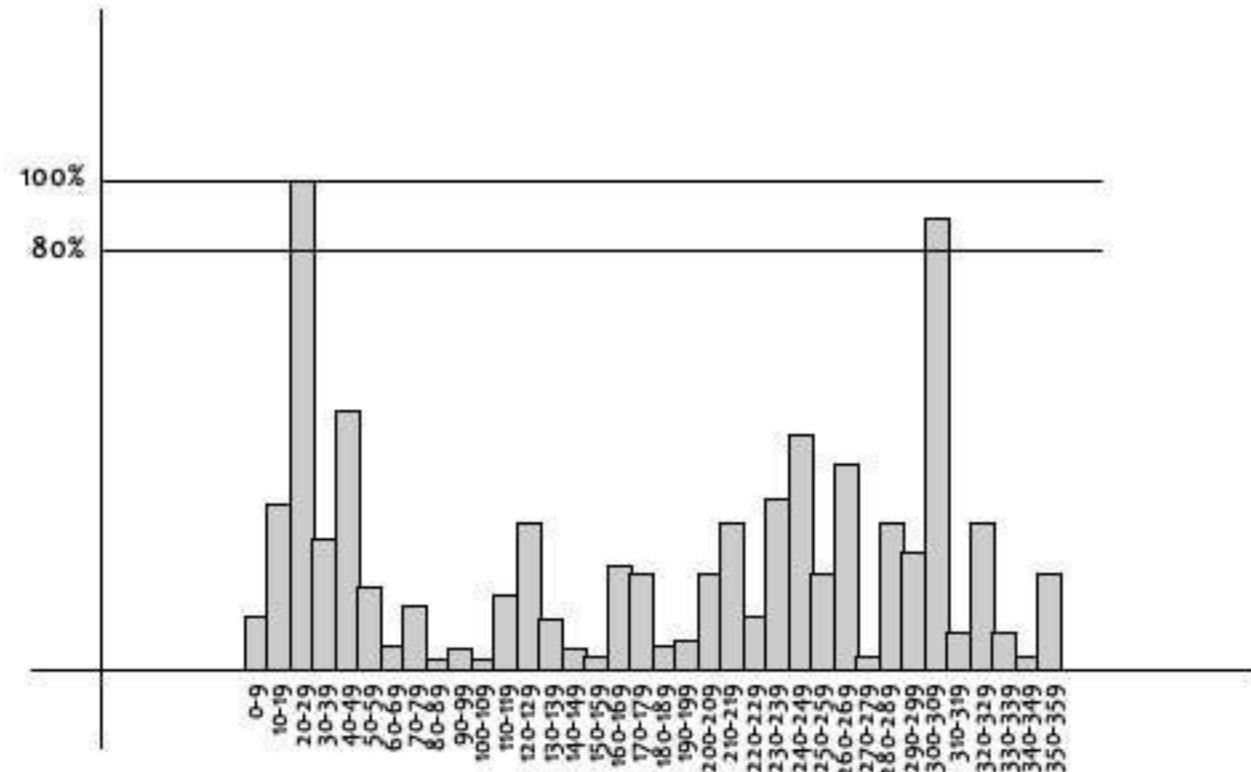
An orientation histogram with 36 bins covering 360 degrees is created. Let's say the gradient direction at a certain point (in the “orientation collection region”) is 18.759 degrees, then it will go into the 10–19-degree bin. And the “amount” that is added to the bin is proportional to the magnitude of the gradient at that point. Once you’ve done this for all pixels around the keypoint, the histogram will have a peak at some point.

Scale Invariant Feature Transform (SIFT)

Step 3: Orientation Assignment

•Compute Dominant Orientation

- In a patch around each keypoint, calculate gradient directions and magnitudes.
- Build a **36-bin orientation histogram** (covering 360°).
- The peak direction is assigned as the **keypoint's dominant orientation** (ensures rotation invariance).



The highest peak in the histogram is taken and any peak above 80% of it is also considered to calculate the orientation. It creates keypoints with same location and scale, but different directions. It contributes to the stability of matching.

Scale Invariant Feature Transform (SIFT)

Step 4: Descriptor Generation (128-D Vector)

•Divide the Keypoint Neighborhood

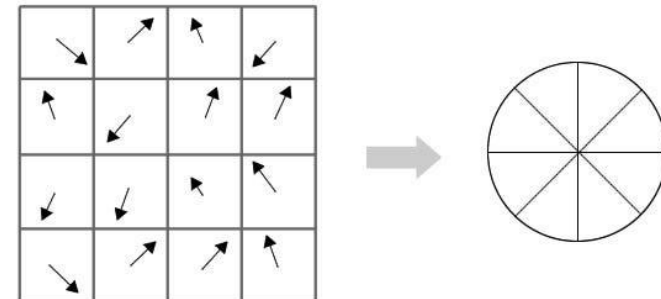
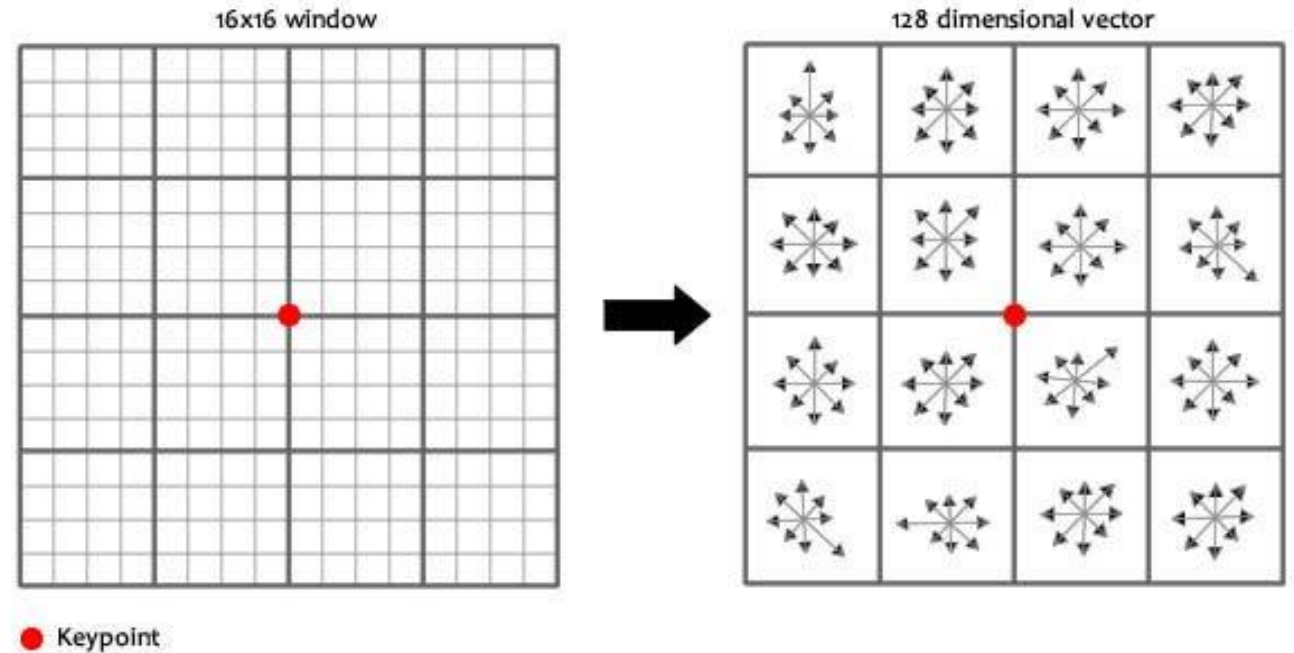
- Split the region into 4×4 sub-blocks (16 total).

•Compute Gradient Histograms per Block

- Each sub-block computes an **8-bin orientation histogram** (8 directions).
- Concatenate all histograms $\rightarrow$ **128-dimensional feature vector** ($16 \times 8 = 128$).

•Normalization for Robustness

- Normalize the vector to reduce sensitivity to lighting changes.



Scale Invariant Feature Transform (SIFT)

✓ Advantages

- Highly robust to **scale, rotation, and lighting changes**.
- Works well under **partial occlusion and viewpoint changes**.
- Widely used in **image stitching, object recognition, and 3D reconstruction**.

✗ Limitations

- **Computationally expensive** (slow for real-time applications).
- **Not ideal for non-rigid deformations** (e.g., facial expressions).
- **Largely replaced by deep learning** (e.g., CNN features, SuperPoint).

Other Local Feature Descriptors

Method	Type	Speed	Robustness	Best Use Case
SIFT	Traditional	Slow	High (scale/rotation)	Image stitching, 3D reconstruction
SURF	Faster SIFT	Medium	High	Real-time applications
ORB	Binary (FAST+BRIEF)	Fast	Medium (no scale invariance)	Mobile/AR apps
SuperPoint	Deep Learning	Fast	High (learned features)	Real-time SLAM, robotics

Why we still learn local feature descriptors: taking SIFT as an example, although SIFT is gradually being replaced by deep learning, its design concepts (such as scale space and orientation histograms) continue to influence modern feature extraction methods.

Image Classification

- Traditional Methods (Pre-Deep Learning)
 - Key Handcrafted Feature Extraction Methods
 - Local Feature Descriptors
 - **Global Feature Descriptors**
 - Bag-of-Visual-Words (BoVW) Representation
 - Limitations on Handcrafted Features
- Image Classification in Deep Learning Era
 - From Perceptrons to CNNs
 - Key CNN Architectures
 - Lightweight Models

Global Feature Descriptors

Objective: Encode entire image into a fixed-length vector

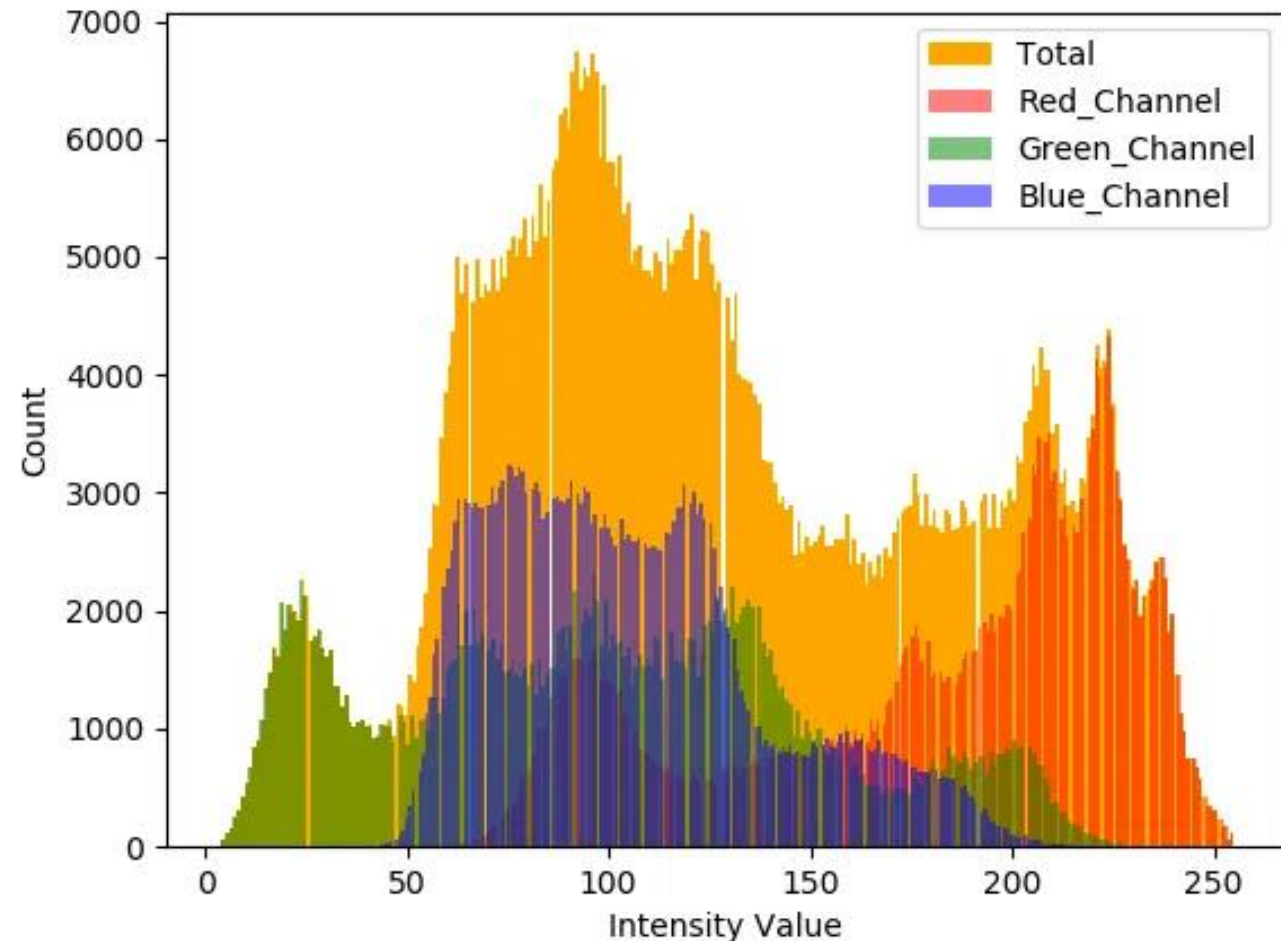
- HOG (Histogram of Oriented Gradients)
 - Captures edge structure (used in pedestrian detection). **Will be introduced in the session of object detection**
- LBP (Local Binary Patterns)
 - Texture descriptor using pixel neighborhood comparisons.
- Color Histograms
 - RGB/HSV histograms for color-based recognition.

Key Properties:

- Whole-image representation** (vs. local features like SIFT, which describe patches).
- Computationally efficient** (faster than aggregating local features).
- Less robust to occlusions** but effective for **scene-level** tasks.

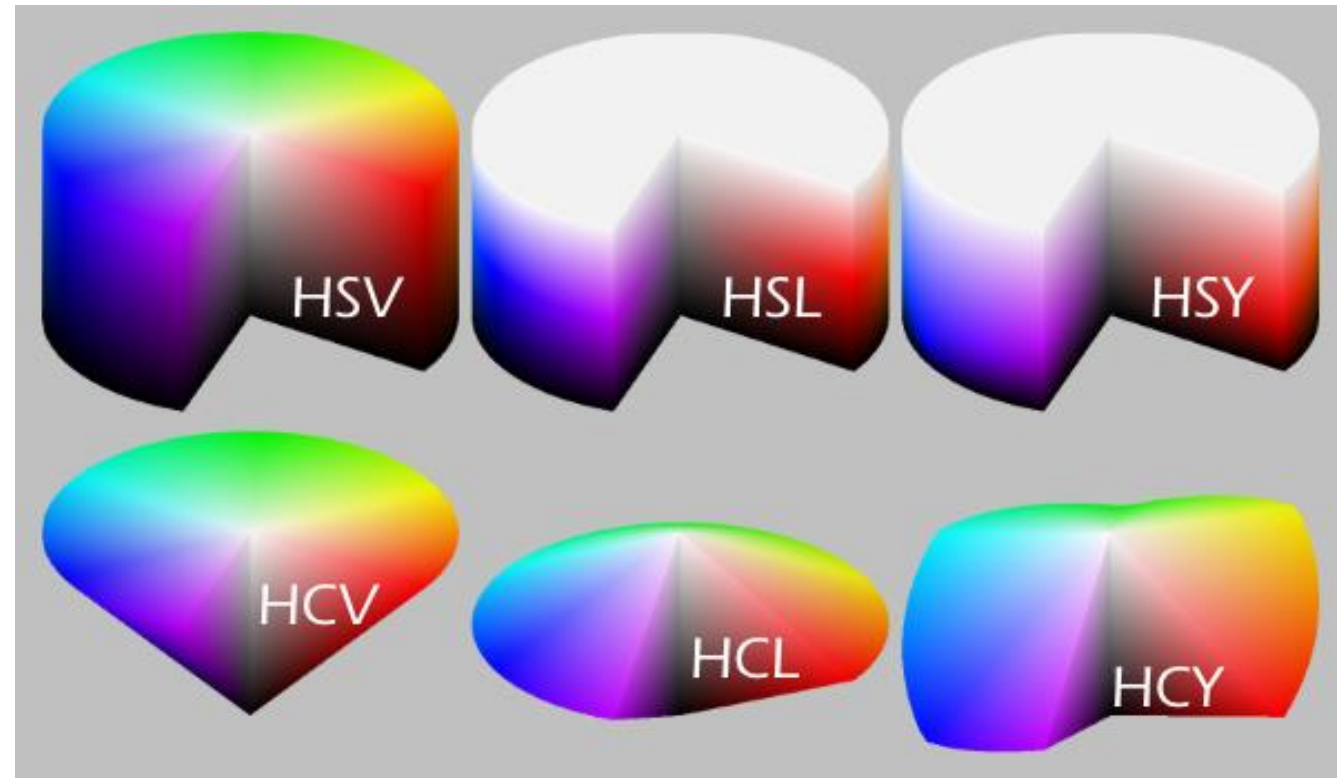
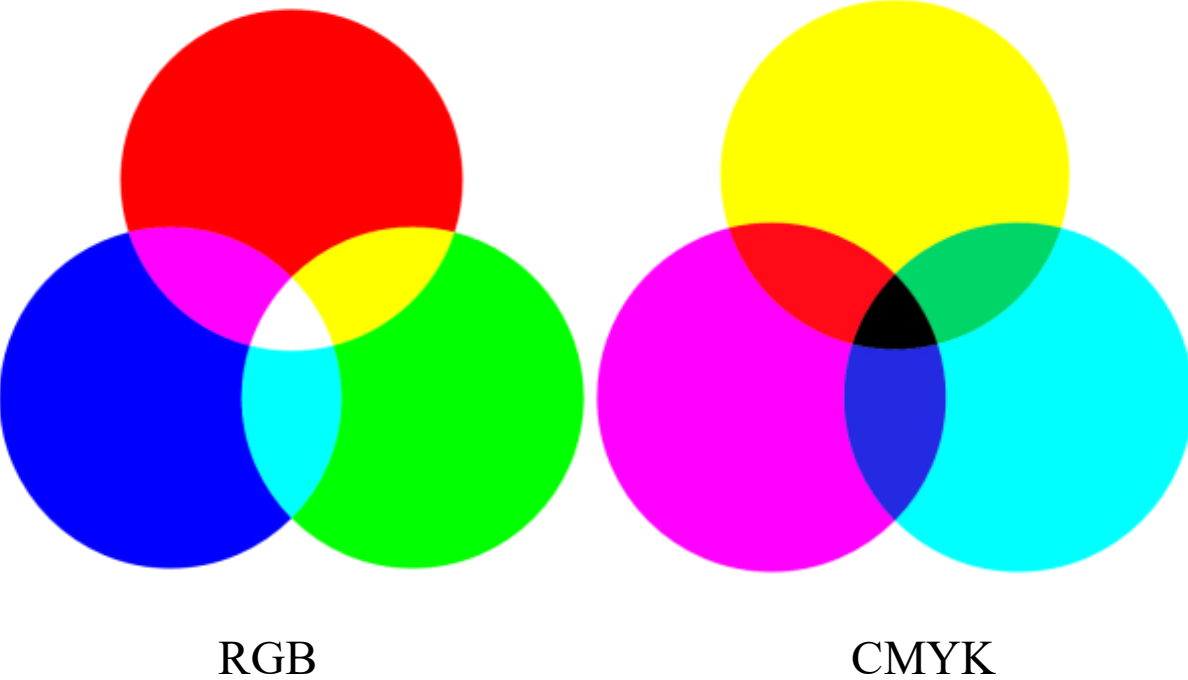
Global Feature Descriptors – Color Histograms

A color histogram represents the distribution of colors within a digital image by counting the number of pixels within specific color ranges. It's essentially a graphical depiction of how frequently different colors appear in the image.



Global Feature Descriptors – Color Histograms

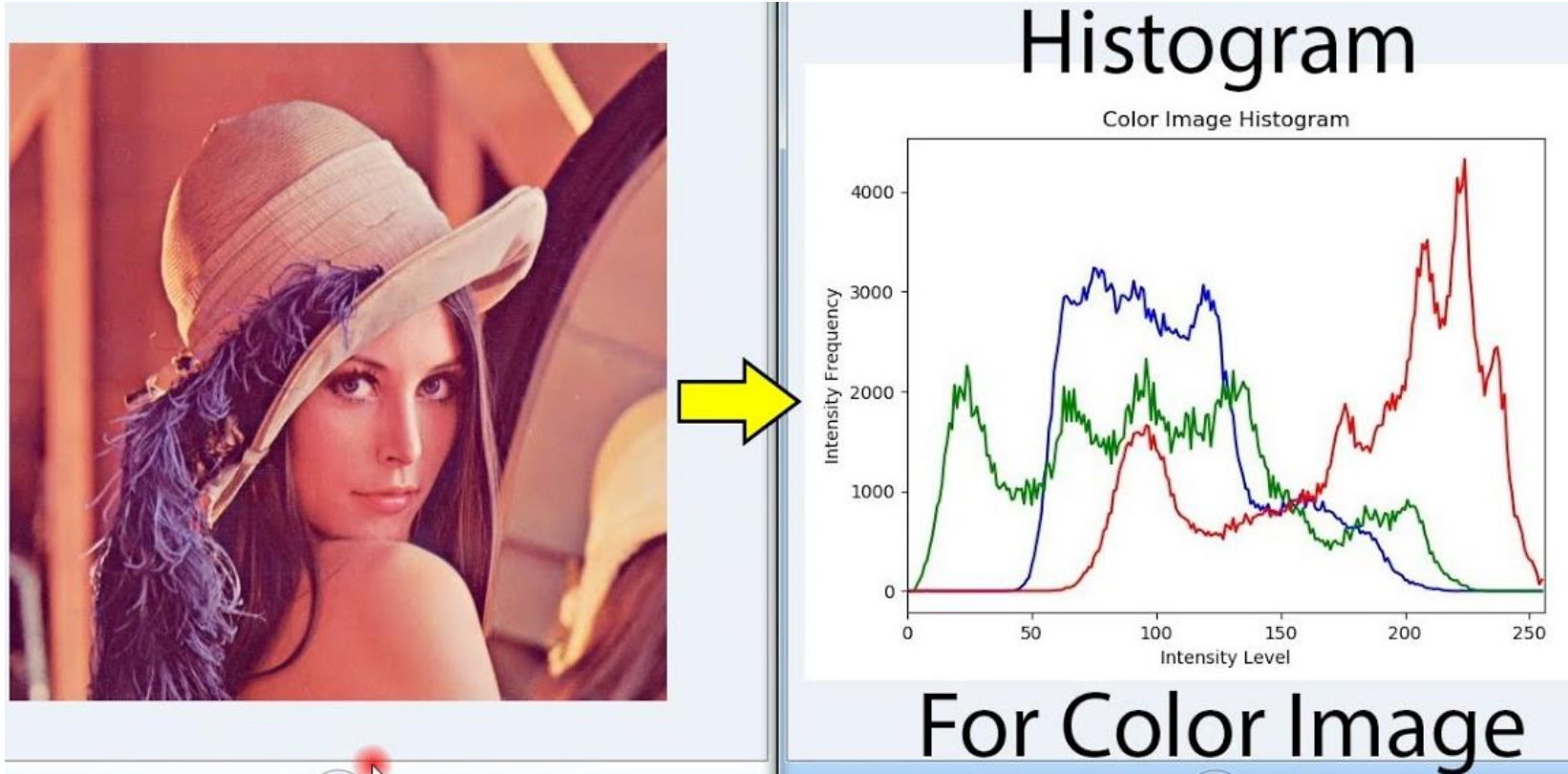
Different types of color space



Global Feature Descriptors – Color Histograms

How it works (taking RGB space as an example):

- Digital images are composed of pixels, each with a red, green, and blue (RGB) value.
- A color histogram analyzes these RGB values to determine the frequency of each color or color range.
- The x-axis of the histogram represents the color values, and the y-axis represents the number of pixels with that color.
- A high bar in the histogram indicates a large number of pixels with that particular color or range of colors.



Global Feature Descriptors – Local Binary Pattern (LBP)

LBP is based on **appearance features**. It is a way to describe the local structure of an image in a way that is invariant to changes in illumination

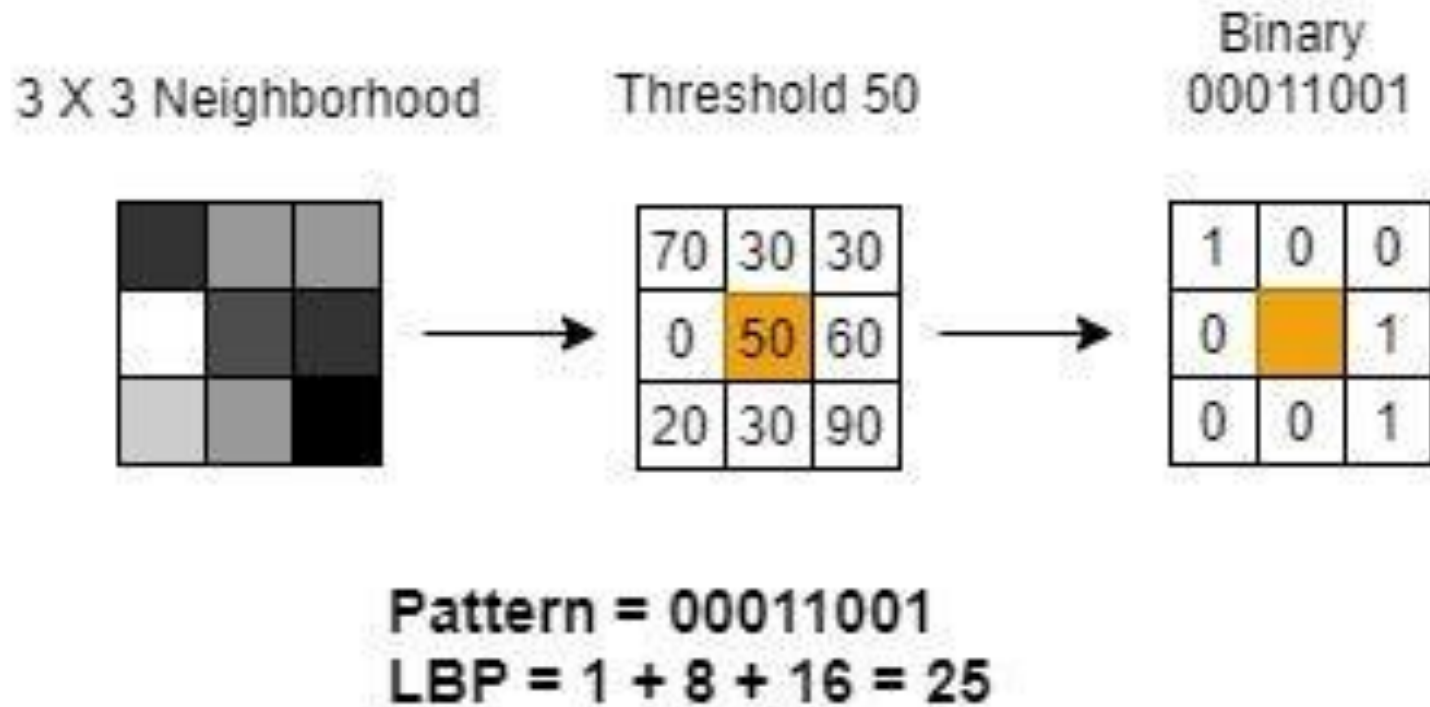
LBP works by comparing the intensity of a central pixel in a small neighborhood with the intensity of its surrounding pixels.

Each pixel in the neighborhood is assigned a binary value based on whether its intensity is greater than or less than the intensity of the central pixel (threshold).

These binary values are then concatenated into a binary number, which represents the texture of that neighborhood.

These binary values can be then used to construct a histogram of the texture distribution within an image.

Global Feature Descriptors – Local Binary Pattern (LBP)

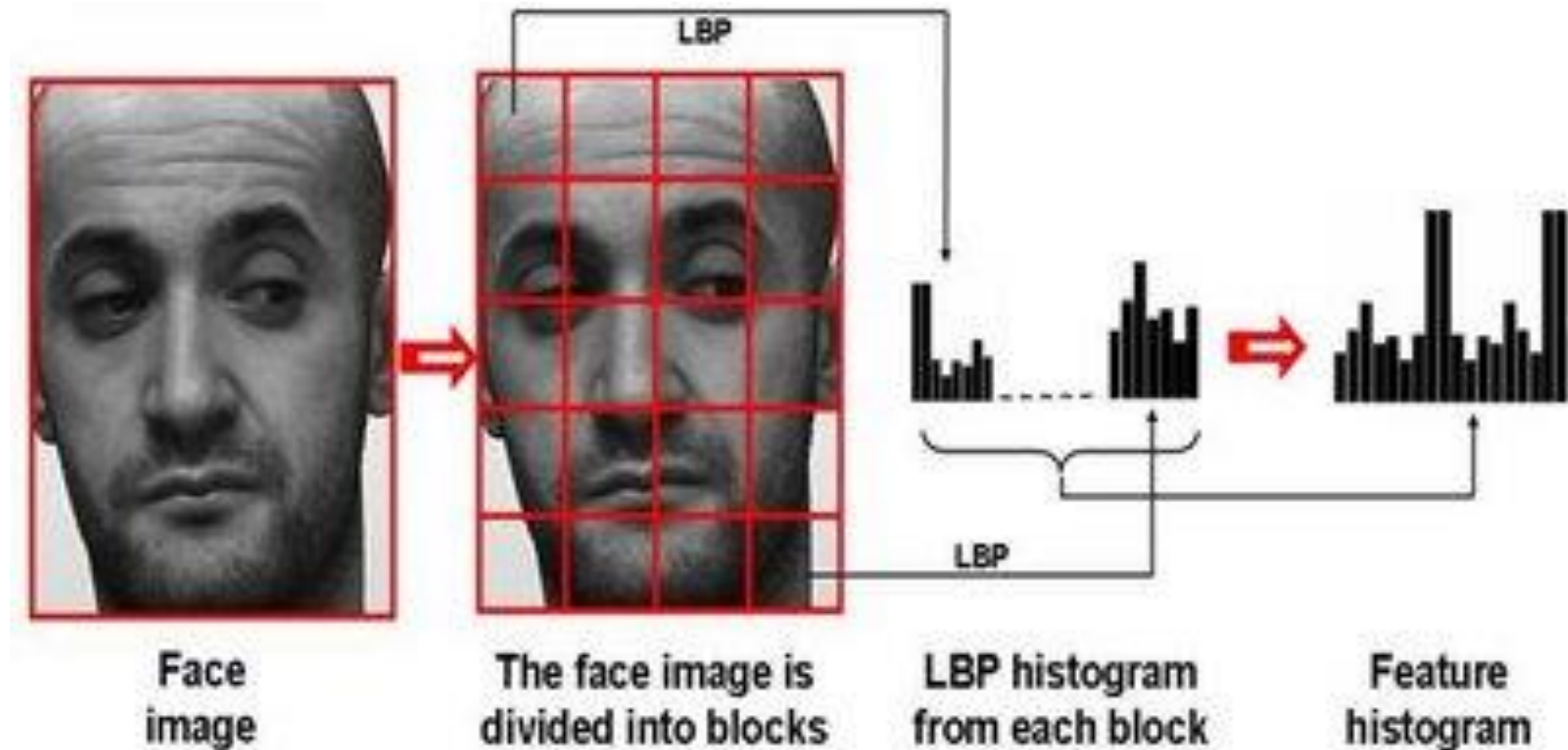


Repeat these steps for each pixel in the image to obtain a binary code for each pixel

Now use these LBP values to construct the histogram. **By constructing a histogram of the LBP patterns, we can capture the frequency of occurrence of different texture patterns in the image.**

This histogram can then be used as a feature vector for texture classification tasks

Global Feature Descriptors – Local Binary Pattern (LBP)



Global Feature Descriptors – Local Binary Pattern (LBP)

Advantages of LBP

- 1.LBP is robust to illumination variations. This makes it particularly useful for applications such as facial recognition and object detection, where lighting conditions can vary significantly.
- 2.LBP is a computationally efficient, suitable for processing large datasets and real-time applications.
- 3.LBP is invariant to image rotation and scale.
- 4.LBP has been shown to be highly discriminative

Disadvantages of LBP

- 1.LBP is sensitive to noise in the image.
- 2.LBP only captures local texture information in the immediate vicinity of each pixel, which can limit its ability to capture more global texture information in the image.
- 3.While LBP is invariant to image rotation, it does not capture rotational information in the texture patterns. This can limit its ability to distinguish between textures that are similar but differ in their rotational patterns.

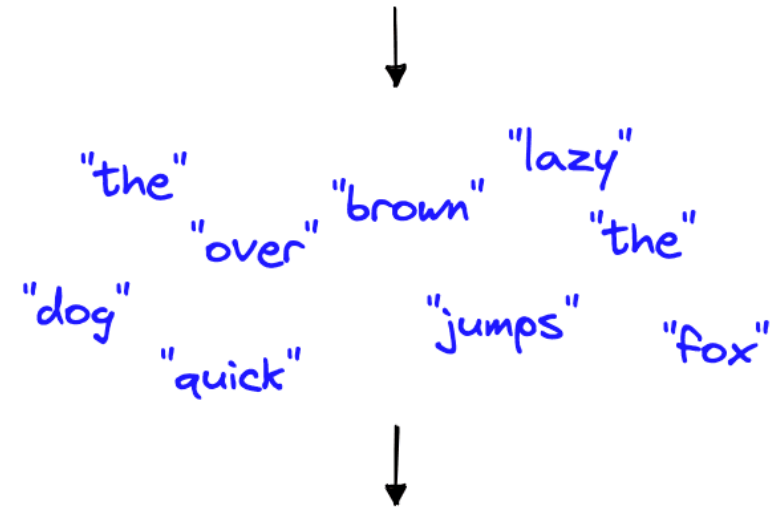
Image Classification

- Traditional Methods (Pre-Deep Learning)
 - Key Handcrafted Feature Extraction Methods
 - Local Feature Descriptors
 - Global Feature Descriptors
 - Bag-of-Visual-Words (BoVW) Representation
 - Limitations on Handcrafted Features
- Image Classification in Deep Learning Era
 - From Perceptrons to CNNs
 - Key CNN Architectures
 - Lightweight Models

Bag-of-Visual-Words (BoVW) Representation

The model derives from **bag of words** in natural language processing (NLP), where a chunk of text is split into words or sub-words and those components are collated into an unordered list, the so-called “bag of words” (BoW).

"the quick brown fox jumps over the lazy dog"



Bag-of-Visual-Words (BoVW) Representation

Similarly, in bag of *visual* words the images are represented by patches, and their unique patterns (or *visual features*) are extracted from the image.



Bag-of-Visual-Words (BoVW) Representation

Visual features consist of two items:

- **Keypoints** are points in an image, which do **not** change if the image is rotated, expanded, or scaled and
- **Descriptors** are vector representations of an image patch found at a given keypoint.

These visual features can be detected and extracted using a feature detector, such as **SIFT (Scale Invariant Feature Transform)**, **ORB (Oriented FAST and Rotated BRIEF)**, or **SURF (Speeded Up Robust Features)**.

The most common is **SIFT** as it is invariant to scale, rotation, translation, illumination, and blur. SIFT converts each image patch into a 128-dimensional vector (i.e., the **descriptor** of the visual feature).

A single image will be represented by many SIFT vectors. The order of these vectors is not important, only their presence within an image.

Bag-of-Visual-Words (BoVW) Representation

Codebook and Visual Words

After extracting visual features we build a **codebook**, also called a dictionary or vocabulary. This codebook acts as a repository of all existing **visual words** (similar to an **actual** dictionary, like the Oxford English Dictionary).

We use this codebook as a way to translate a potentially infinite variation of visual features into a predefined set of visual words.

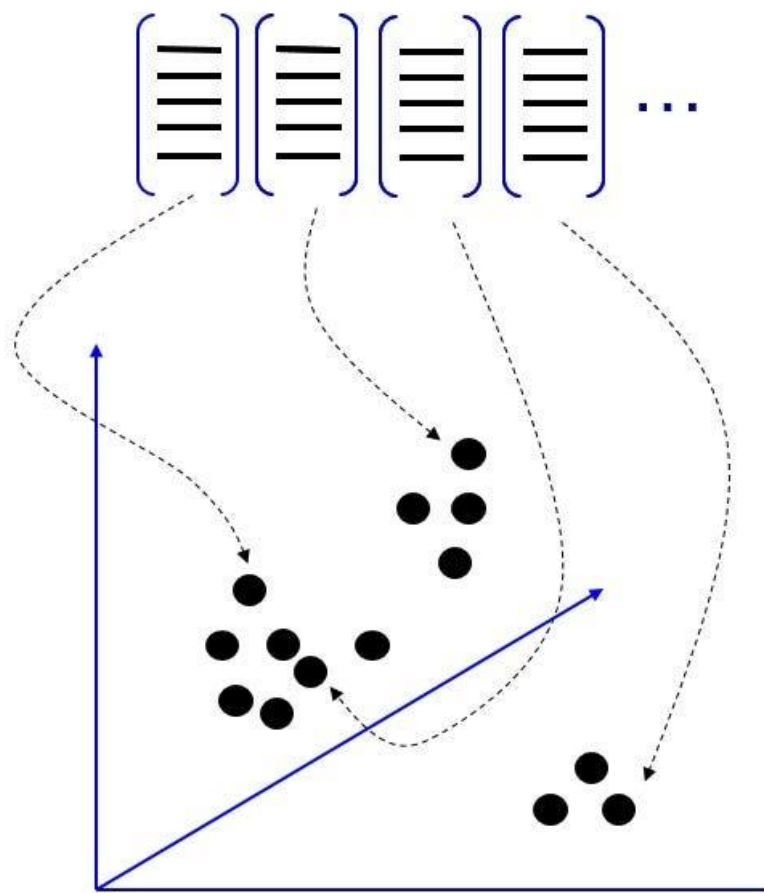
How? K-means clustering

If we choose $k=150$, k-means will generate 150 centroids and, therefore, 150 visual words.

When we perform the mapping from new visual feature vectors to the nearest centroid (i.e., visual word), we categorize visual features into a more limited set of visual words. This process of reducing the number of possible unique vectors is called **vector quantization**.

Bag-of-Visual-Words (BoVW) Representation

Codebook and Visual Words



Bag-of-Visual-Words (BoVW) Representation

Using a limited set of visual words allows us to compress our image descriptions into a set of visual word IDs. And, more importantly, it helps us represent similar features across images using a shared set of visual words.

After those steps, our images will be represented by a varying number of visual words. From here we move on to the next step of transforming these visual words into image-level frequency vectors.

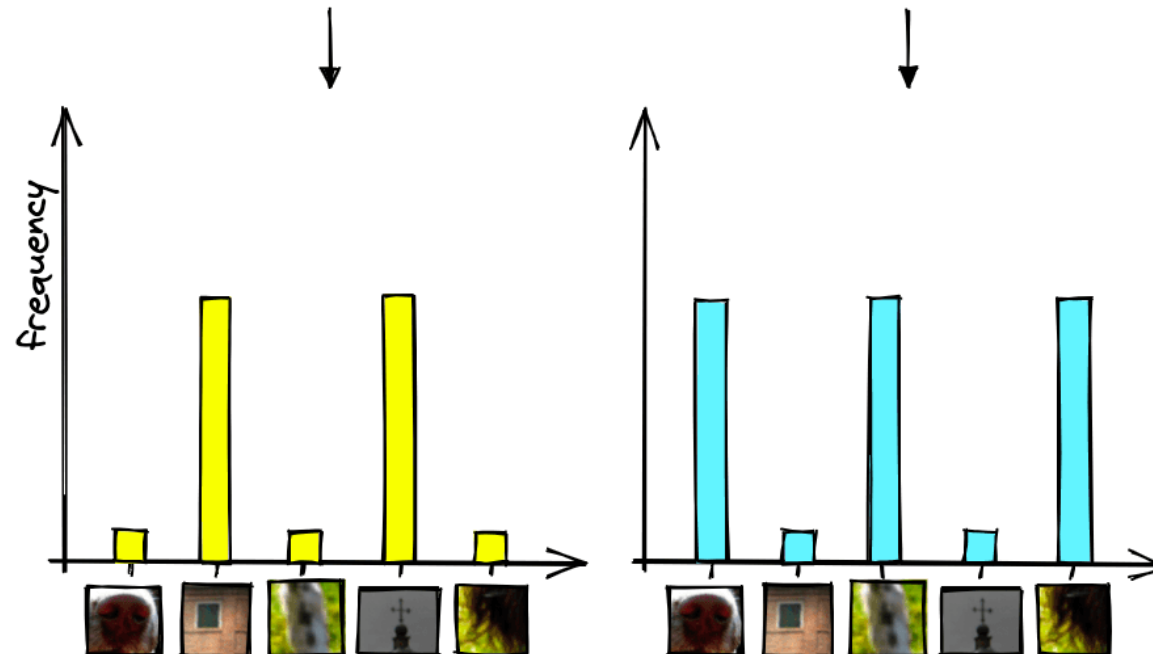


Bag-of-Visual-Words (BoVW) Representation

Frequency Vectors

We can count the frequency of these visual words and visualize them with histograms.

The **x-axis** of the histogram is the codebook, while the **y-axis** is the frequency of each visual word (in the codebook) for that image.



Bag-of-Visual-Words (BoVW) Representation

Tf-idf

In language there are some words that are more important than others in that they give us more information. If we used the sentence “the history of Rome” to search through a set of articles, the words “the” and “of” should not be given the same importance as “history” or “Rome”.

This problem is also found in images. A visual word extracted from a patch of sky in an image is unlikely to tell us whether this image is of a church or a dog. Some visual words are more relevant than others.

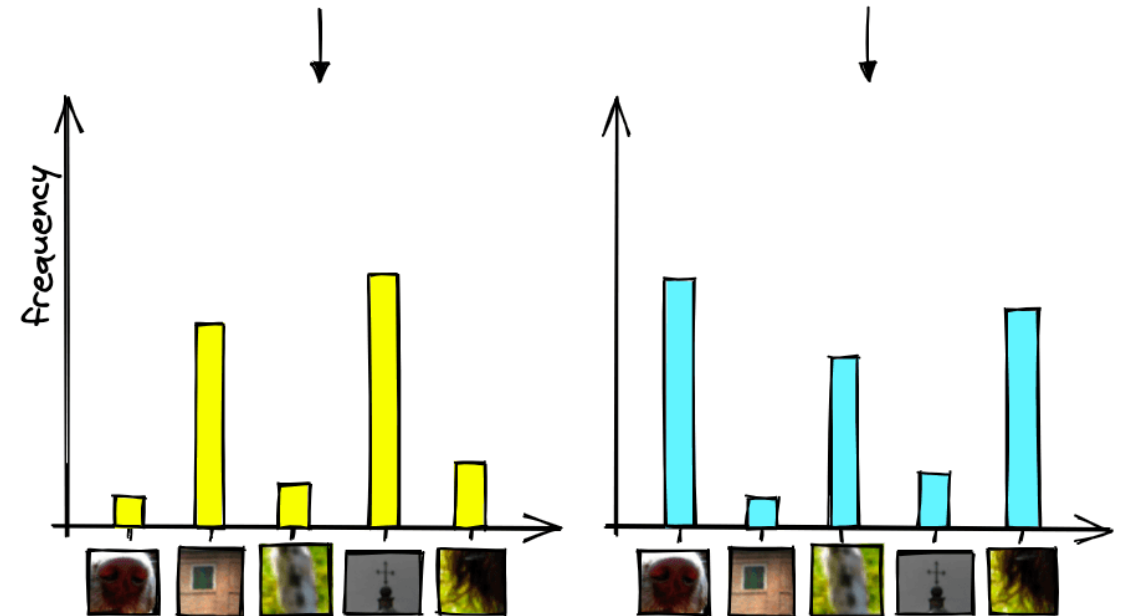


Bag-of-Visual-Words (BoVW) Representation

Tf-idf

$$tf-idf_{t,d} = tf_{t,d} * idf_t = tf_{t,d} * \log \frac{N}{df_t}$$

- $tf_{t,d}$ is the term frequency of the visual word t in the image d (the number of times t occurs in d),
- N is the total number of images,
- df_t number of images containing visual word t ,
- $\log \frac{N}{df_t}$ measures how common the visual word t is across all images in the database. This is low if the visual word d occurs many times in the image, high otherwise.

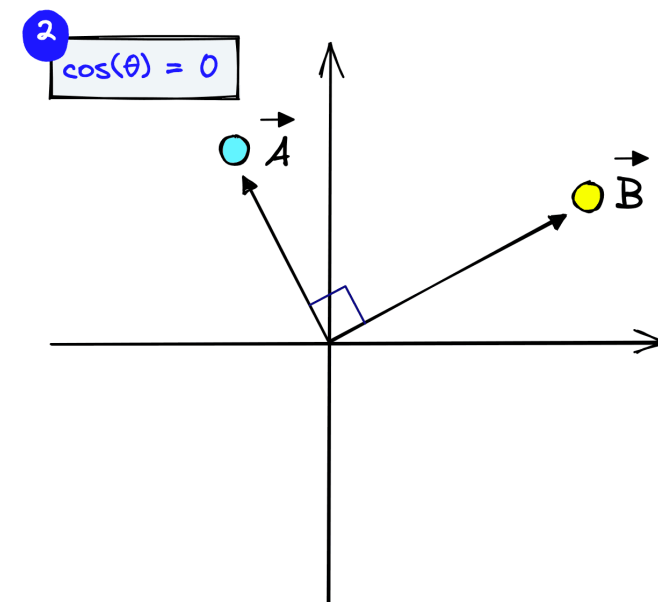
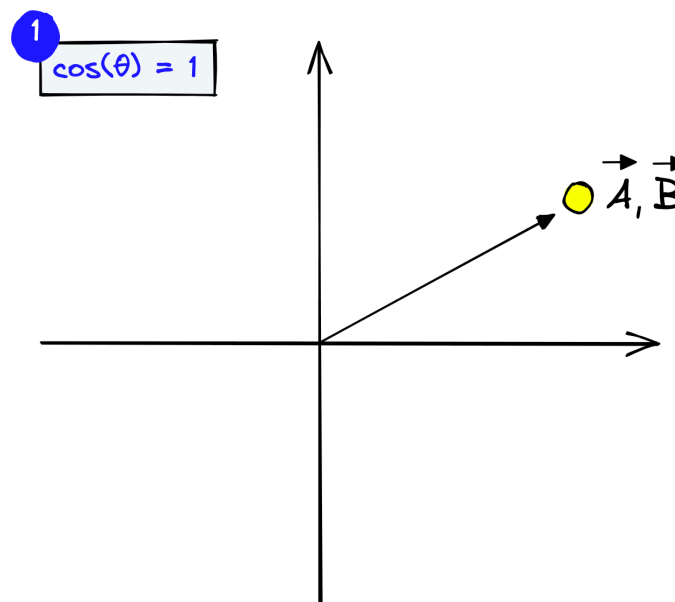


Bag-of-Visual-Words (BoVW) Representation

Measure Similarity

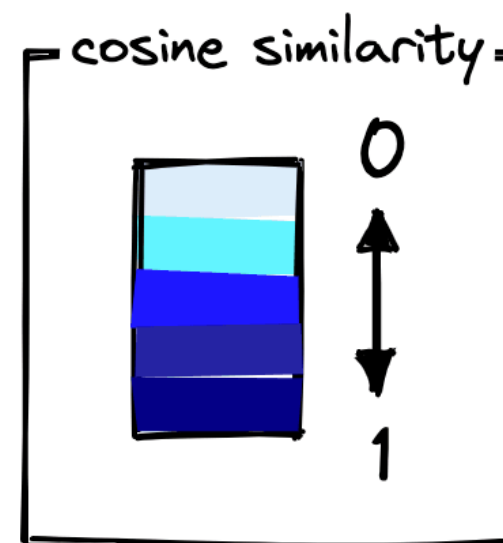
1. Cosine similarity,
2. Euclidean distance, and
3. Dot product similarity.

$$\text{cossim}(A, B) = \cos(\theta) = \frac{A \cdot B}{||A|| ||B||}$$



Bag-of-Visual-Words (BoVW) Representation

Measure Similarity



Bag-of-Visual-Words (BoVW) Representation

Search Image



2: 0.6527



3: 0.6196



4: 0.6186



5: 0.616



Image Classification

- Traditional Methods (Pre-Deep Learning)
 - Key Handcrafted Feature Extraction Methods
 - Local Feature Descriptors
 - Global Feature Descriptors
 - Bag-of-Visual-Words (BoVW) Representation
 - Limitations on Handcrafted Features
- Image Classification in Deep Learning Era
 - From Perceptrons to CNNs
 - Key CNN Architectures
 - Lightweight Models

Limitations on Handcrafted Features

Domain Dependency: Handcrafted features are highly dependent on expert knowledge of the specific domain (e.g., SIFT for image keypoints). If the domain changes, the features may no longer be effective, requiring new manual engineering.

Time-Consuming and Labor-Intensive: Designing effective features requires extensive trial and error, making the process slow and expensive. Experts must iteratively refine features based on empirical results.

Suboptimal for Complex Patterns: Handcrafted features may miss subtle or high-level patterns in data that could be automatically learned by deep learning models (e.g., CNNs for images). They often rely on simplifications that discard useful information.

Limitations on Handcrafted Features

When Are Handcrafted Features Still Useful?

Small datasets (where deep learning may overfit).

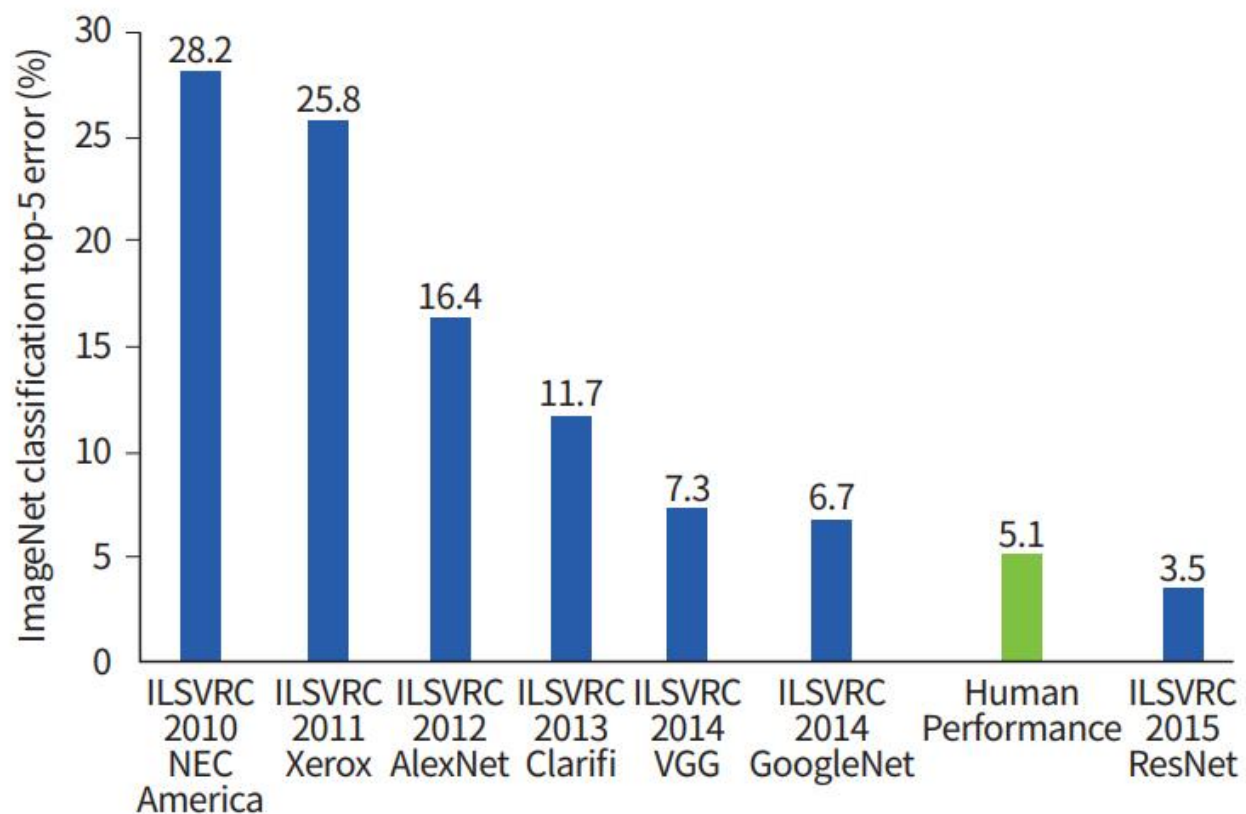
Interpretability (engineered features are more explainable).

Resource-constrained environments (lightweight feature extractors vs. large neural networks).

Image Classification

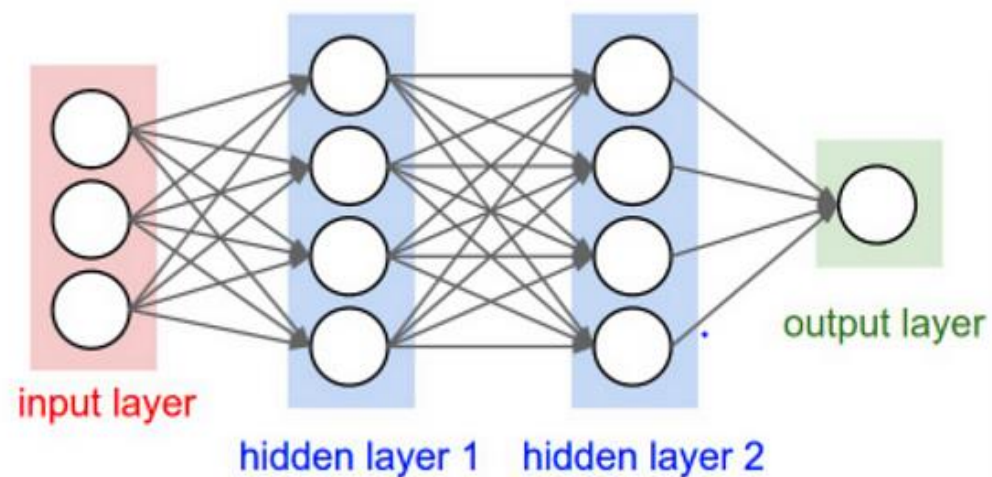
- Traditional Methods (Pre-Deep Learning)
 - Key Handcrafted Feature Extraction Methods
 - Local Feature Descriptors
 - Global Feature Descriptors
 - Bag-of-Visual-Words (BoVW) Representation
 - Limitations on Handcrafted Features
- Image Classification in Deep Learning Era
 - From Perceptrons to CNNs
 - Key CNN Architectures
 - Lightweight Models

Top-5 error(%) in ImageNet

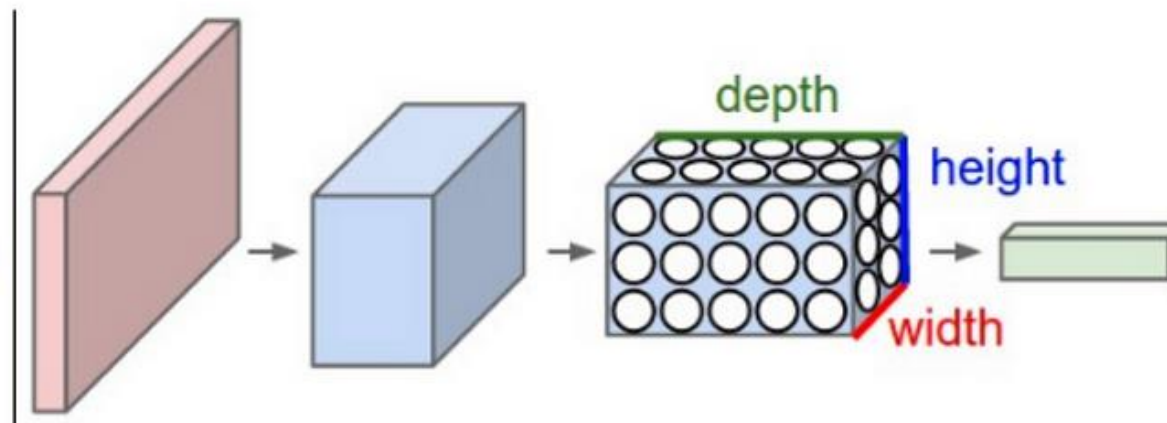


ImageNet testing images

From Perceptrons to CNNs



MLP

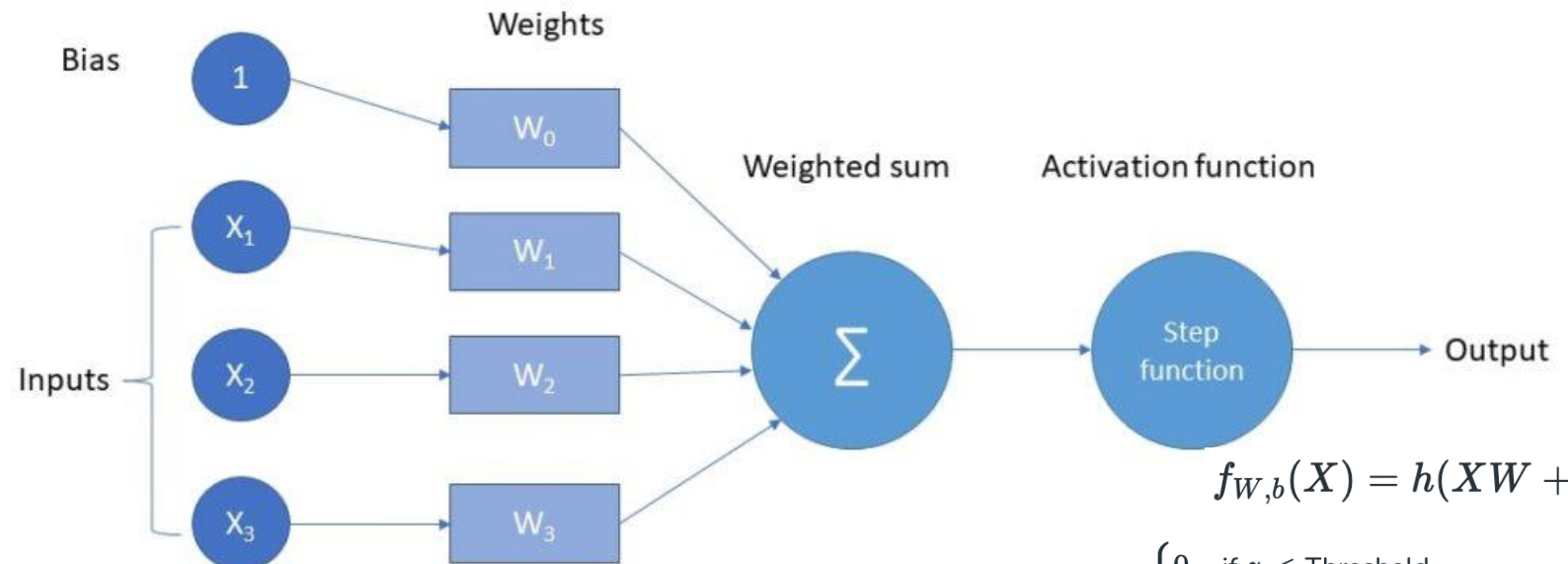


CNN

From Perceptrons to CNNs

What is perceptron

Basic Unit: A perceptron is a simple model that takes **inputs**, multiplies them by **weights**, sums them, and applies an **activation function** to produce an output.



$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n = X^TW$$

$$f_{W,b}(X) = h(XW + b)$$

$$h(z) = \begin{cases} 0 & \text{if } z < \text{Threshold} \\ 1 & \text{if } z \geq \text{Threshold} \end{cases}$$

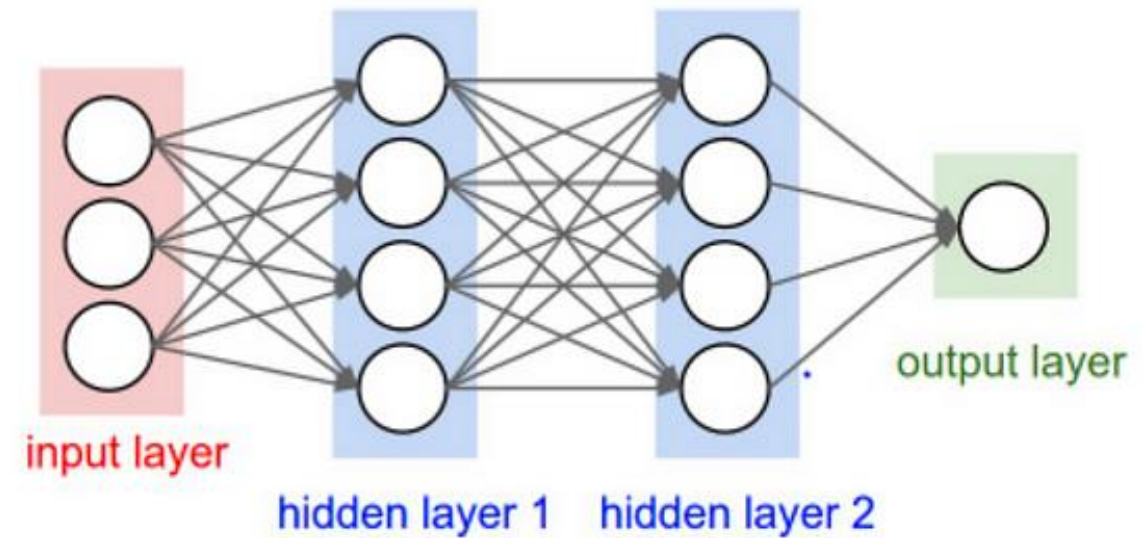
bias: The bias term helps the perceptron make adjustments independent of the input, improving its flexibility in learning

From Perceptrons to CNNs

Multilayer Perceptrons (MLPs):

Single-layer perceptrons can only learn linearly separable data and struggle with complex patterns

A multilayer perceptron (MLP) is a class of feedforward artificial neural network. A MLP consists of at least three layers of nodes: an input layer, a hidden layer and an output layer.

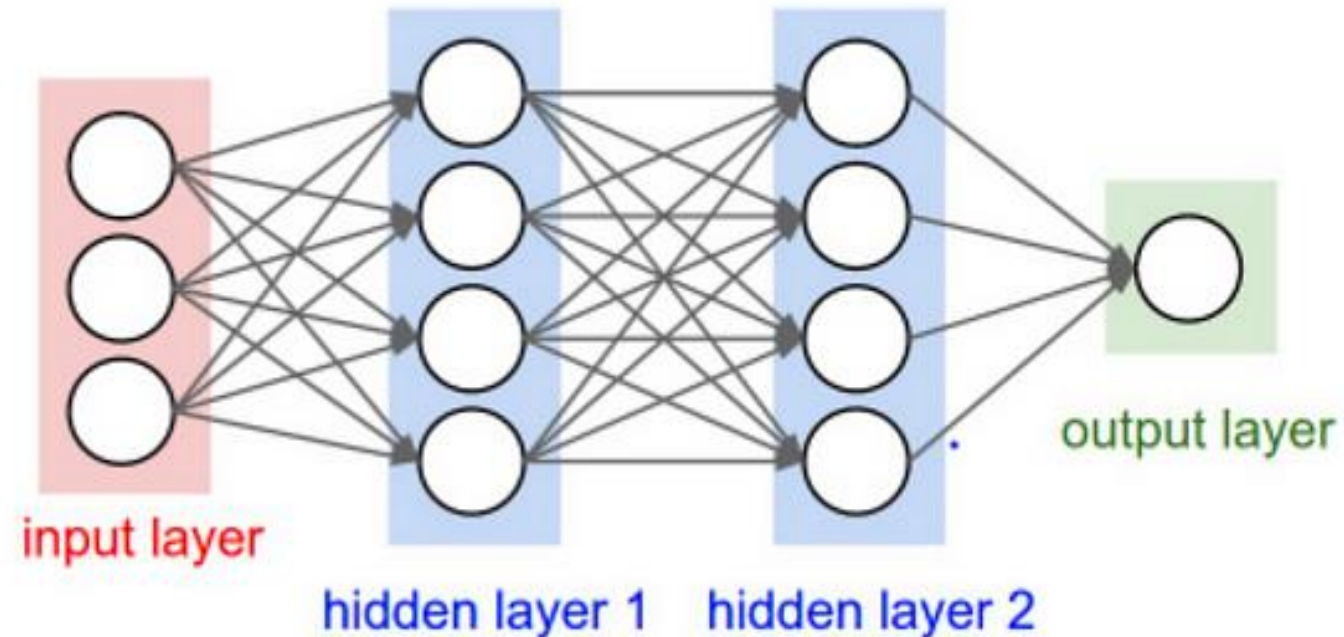


MLP

From Perceptrons to CNNs

Two limitations of MLPs:

It has the characteristic of **fully connected layers**, where each perceptron is connected with every other perceptron. The disadvantage is that the number of total parameters can grow to very high (number of perceptron in layer 1 multiplied by # of p in layer 2 multiplied by # of p in layer 3...). This is inefficient because there is redundancy in such high dimensions

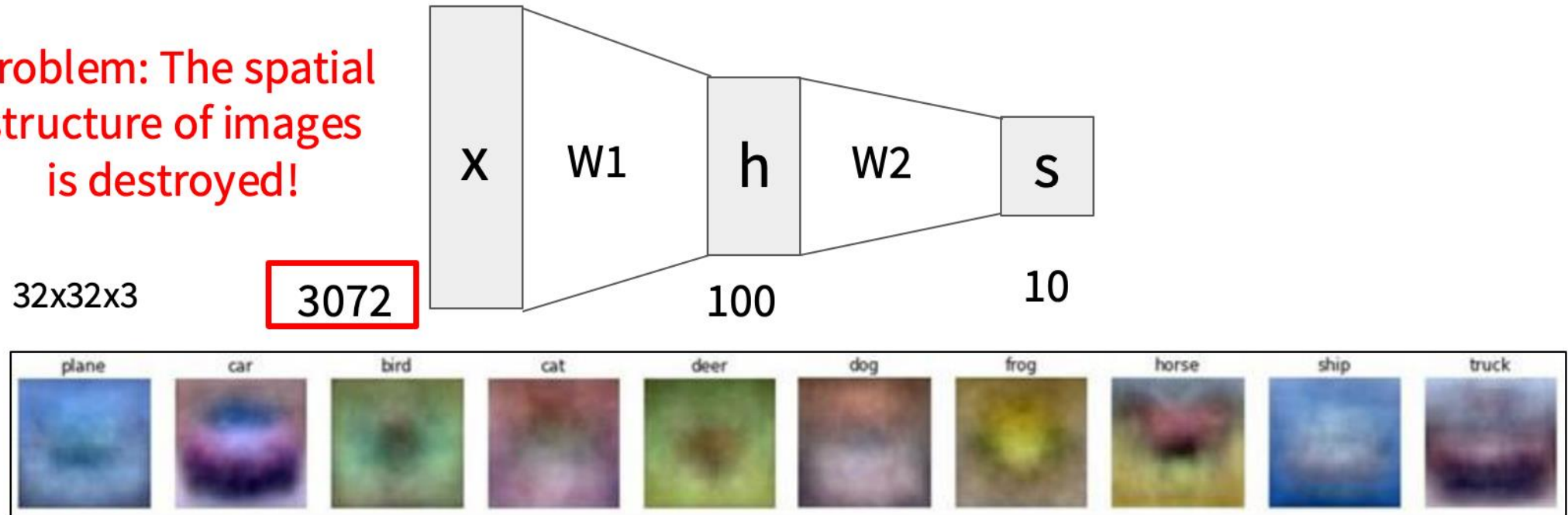


MLP

From Perceptrons to CNNs

Two limitations of MLPs:

Problem: The spatial structure of images is destroyed!

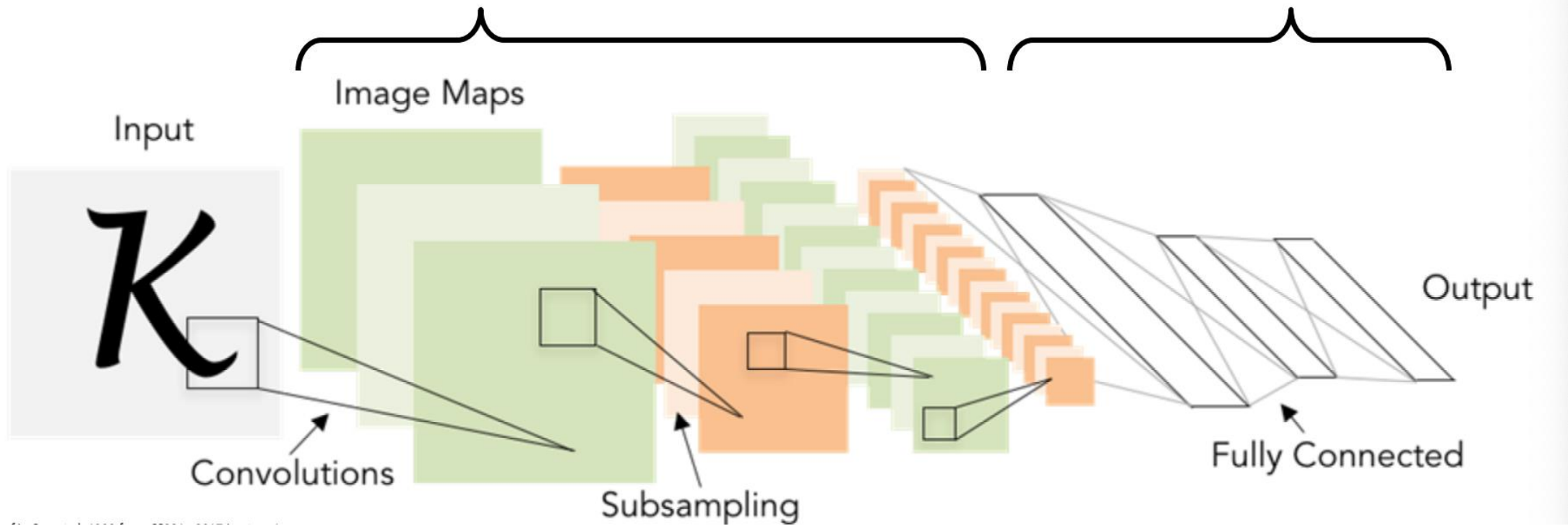


Another disadvantage is that it disregards spatial information. It takes flattened vectors as inputs.

From Perceptrons to CNNs

Convolution and **pooling** operators extract features while respecting 2D image structure

Fully-Connected layers form an MLP at the end to predict scores



CNNs build upon the perceptron concept but introduce convolutional layers that utilize filters (also called **kernels**) to convolve over input data (like images).

From Perceptrons to CNNs

- **Feature Extraction:** Each filter detects specific patterns or features in local regions of the input, and the convolution operation slides this filter across the entire input, creating a feature map.
- **Shared Weights:** The same filter is applied across different locations in the input, sharing weights and reducing the number of parameters, making CNNs more efficient than MLPs for image processing.
- **Spatial Hierarchies:** CNNs learn hierarchical representations of features, where lower layers detect simple patterns (edges, textures), and higher layers combine these to recognize more complex objects.
- **Pooling:** Pooling layers (like max-pooling) further reduce the dimensionality of feature maps, making the network more robust to variations in input.
- **Fully Connected Layers:** The final layers of a CNN are often fully connected layers (similar to perceptrons) that take the pooled feature maps and produce the final classification or prediction.

From Perceptrons to CNNs

Trained end-to-end with backprop + gradient descent

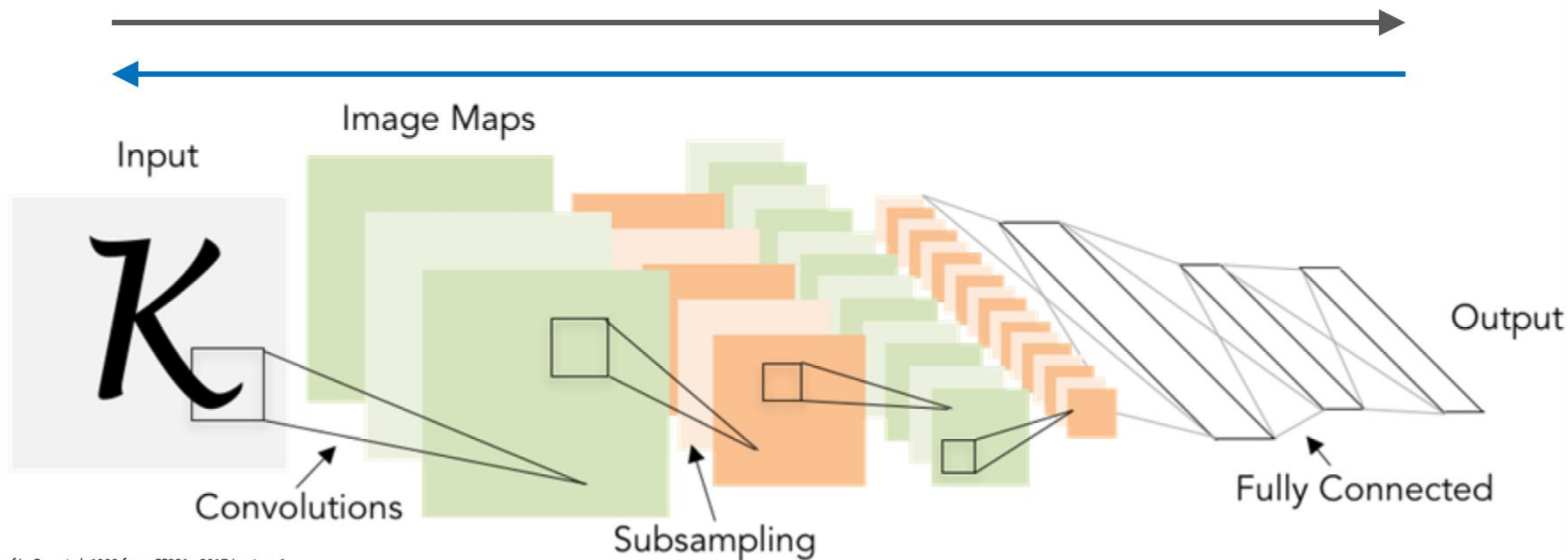


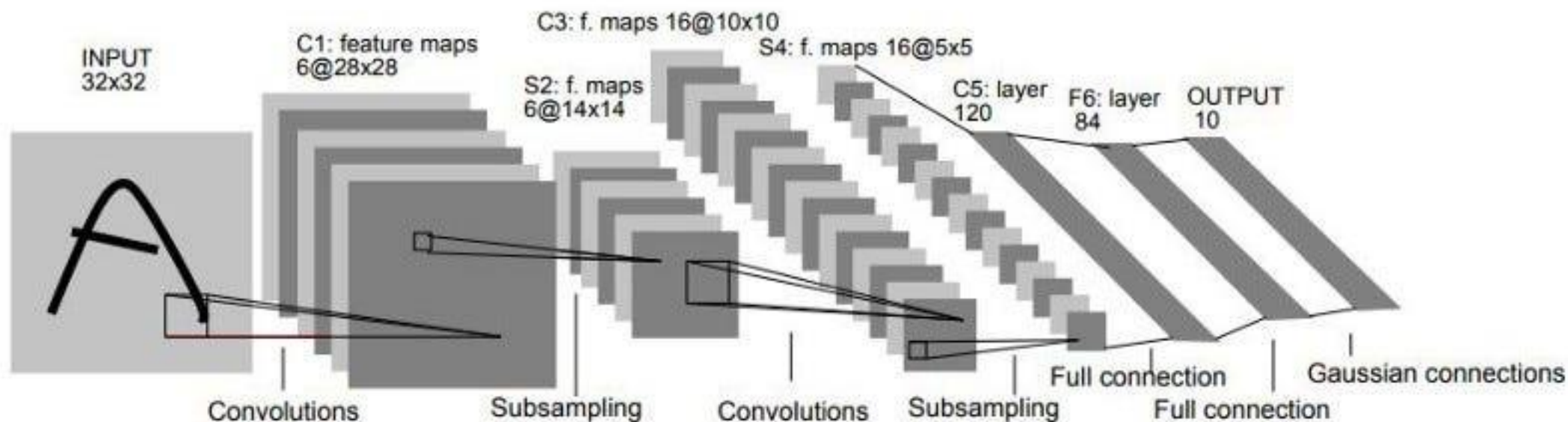
Image Classification

- Traditional Methods (Pre-Deep Learning)
 - Key Handcrafted Feature Extraction Methods
 - Local Feature Descriptors
 - Global Feature Descriptors
 - Bag-of-Visual-Words (BoVW) Representation
 - Limitations on Handcrafted Features
- Image Classification in Deep Learning Era
 - From Perceptrons to CNNs
 - **Key CNN Architectures**
 - Lightweight Models

Key CNN Architectures - LeNet

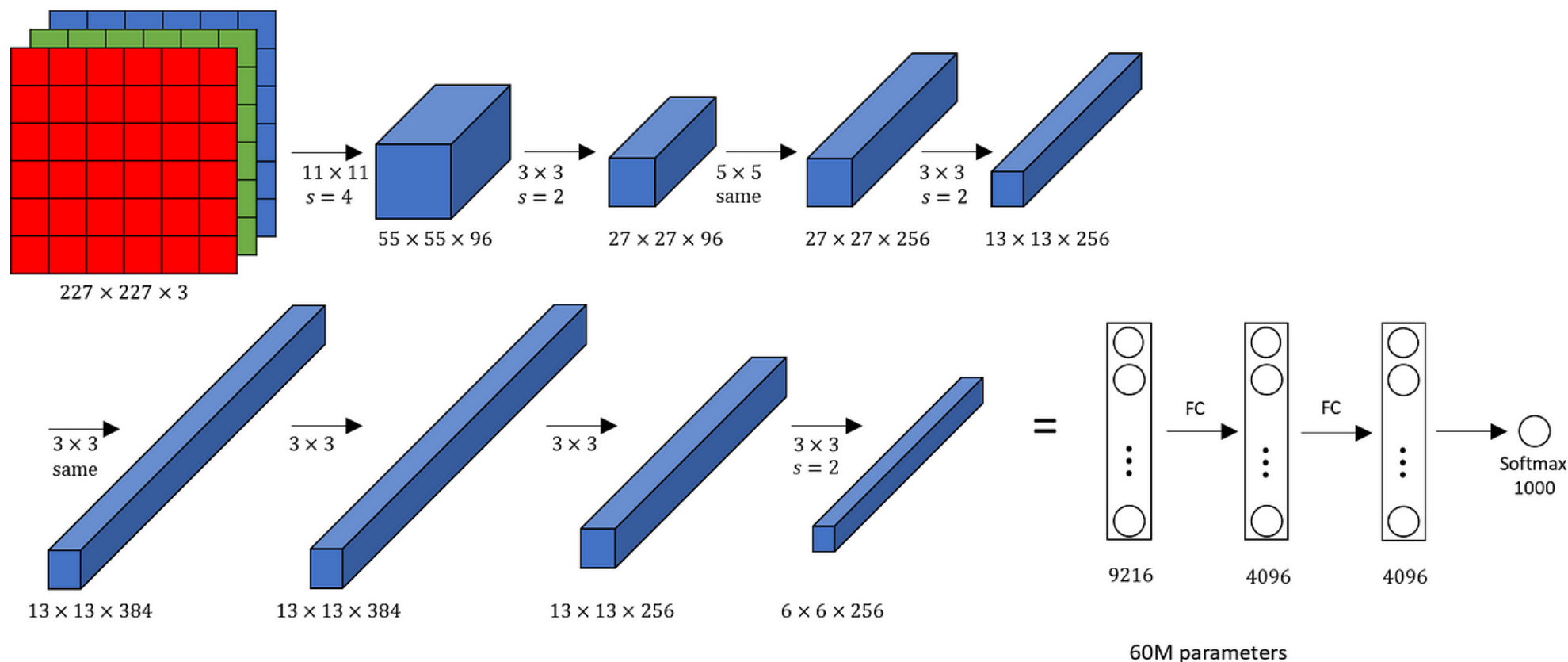
LeNet (1998) – A bit of history

‘**tanh**’ is
used as the
activation
function

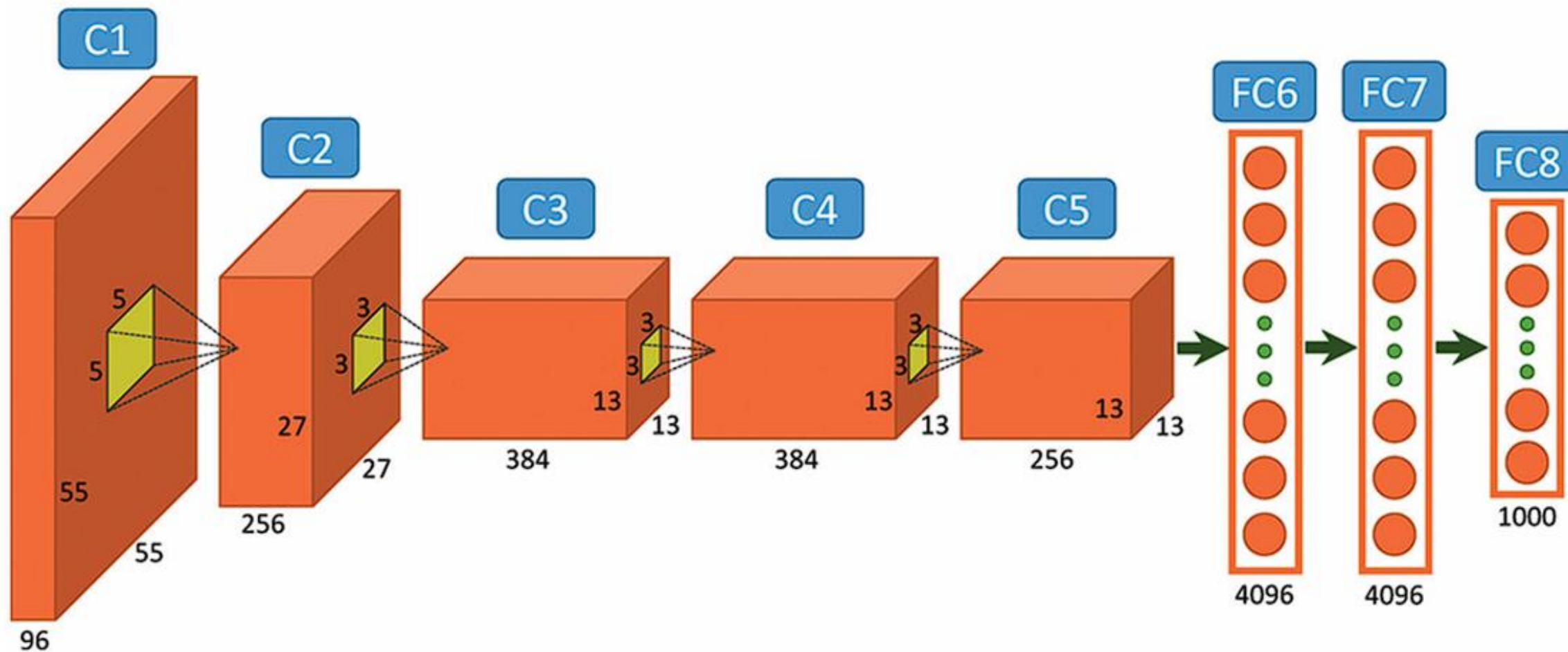


Key CNN Architectures - AlexNet

AlexNet (2012) – Deep Learning Breakthrough



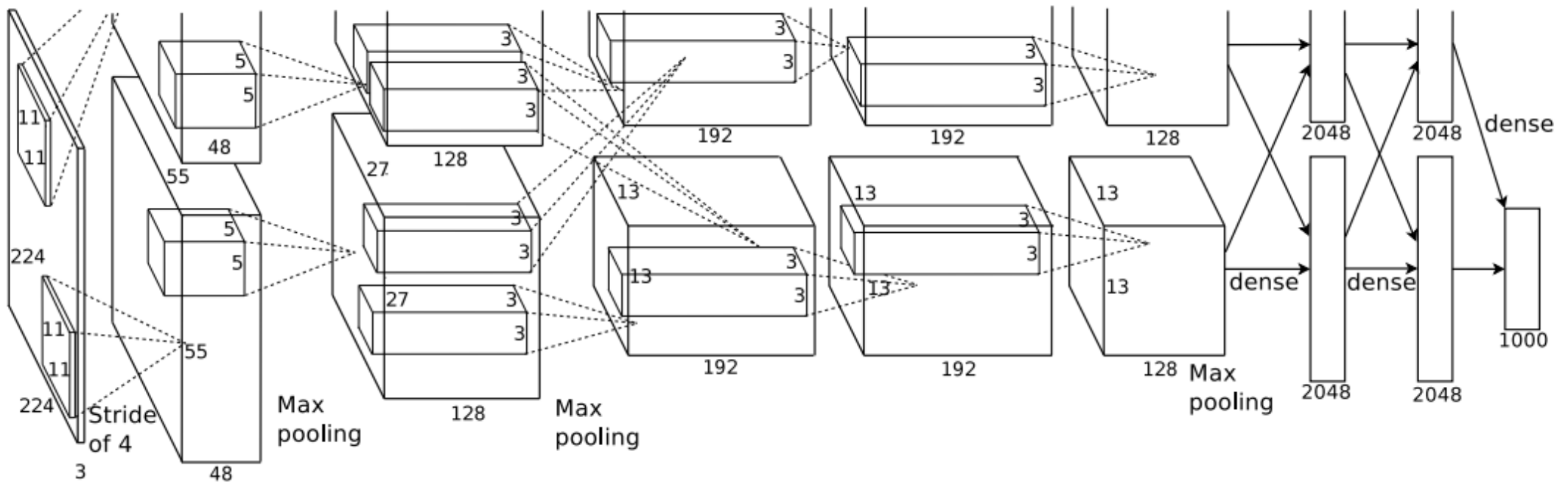
Key CNN Architectures - AlexNet



AlexNet: architecture

Key CNN Architectures - AlexNet

AlexNet was trained on a GTX 580 GPU with only 3 GB of memory which couldn't fit the entire network. So the network was split across 2 GPUs, with half of the neurons(feature maps) on each GPU



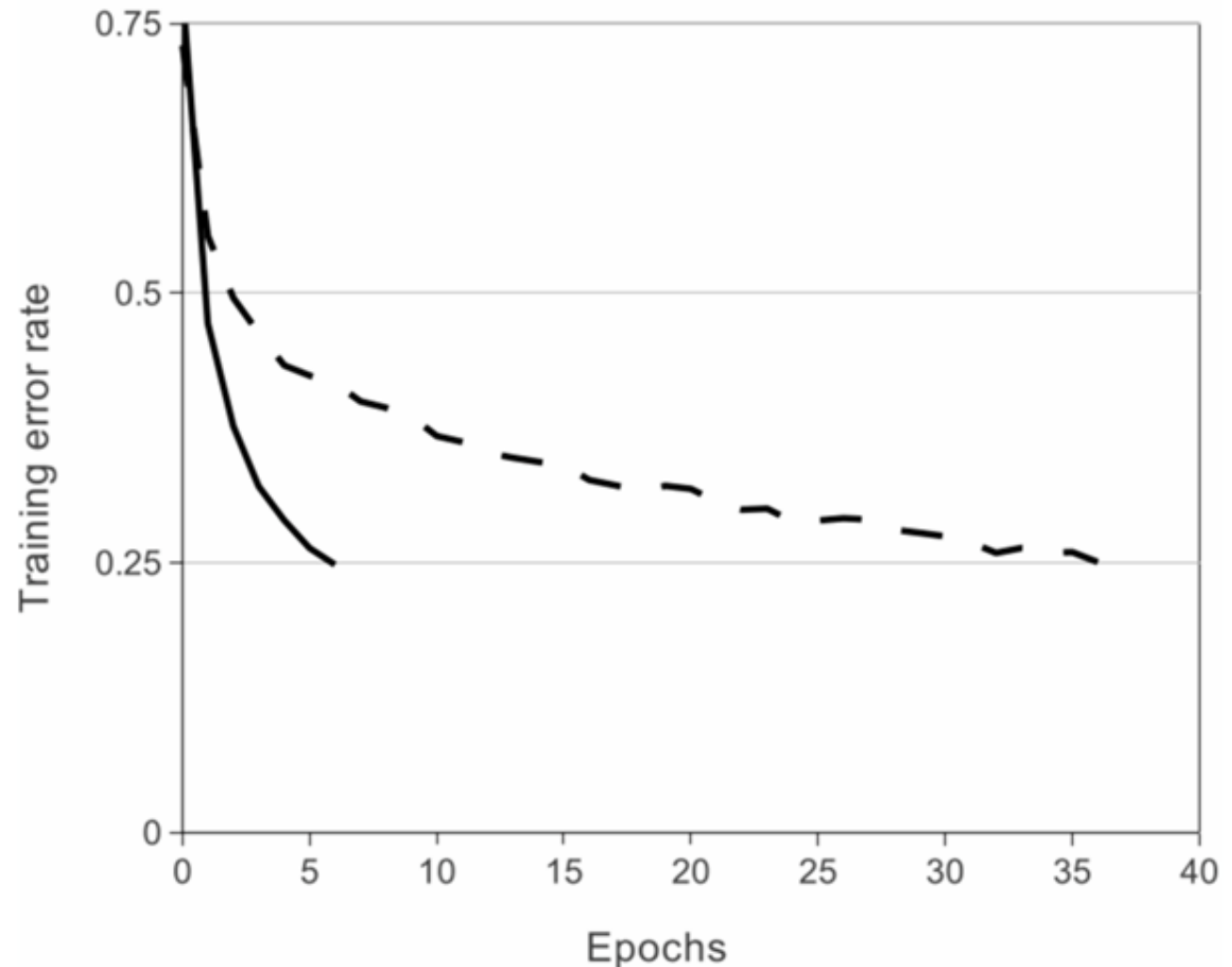
The original illustration of the network in the paper

Key CNN Architectures - AlexNet

Key features of AlexNet

ReLU is used as an activation function rather than 'tanh'

AlexNet can achieve a training error rate of 25% with the aid of ReLUs (solid curve). Compared to a network using tanh, this is six times faster (dotted curve).



Key CNN Architectures - AlexNet

Key features of AlexNet

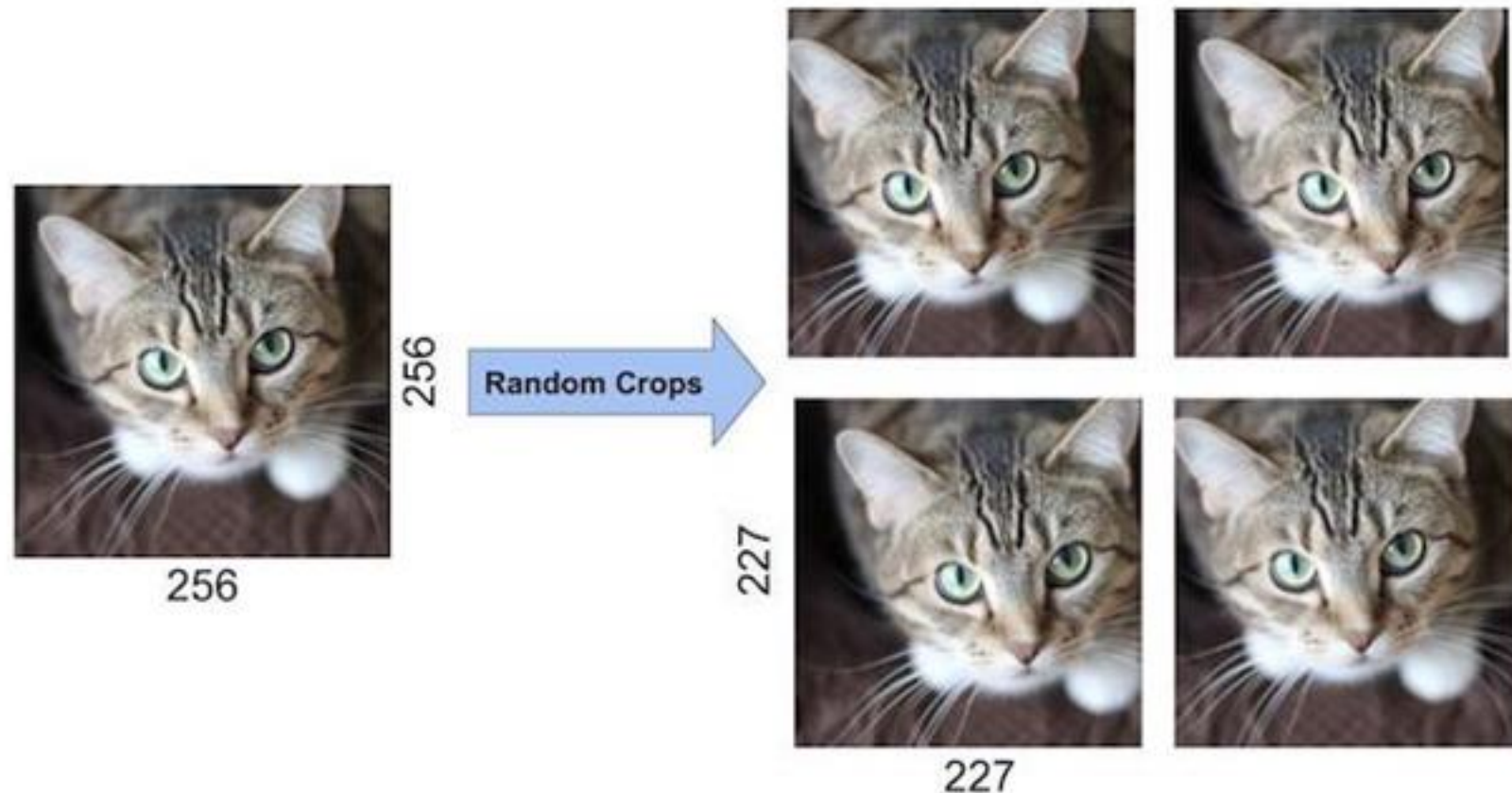
Data Augmentation is
been carried out like
flipping, jittering,
cropping, colour
normalization, etc.



Key CNN Architectures - AlexNet

Key features of AlexNet

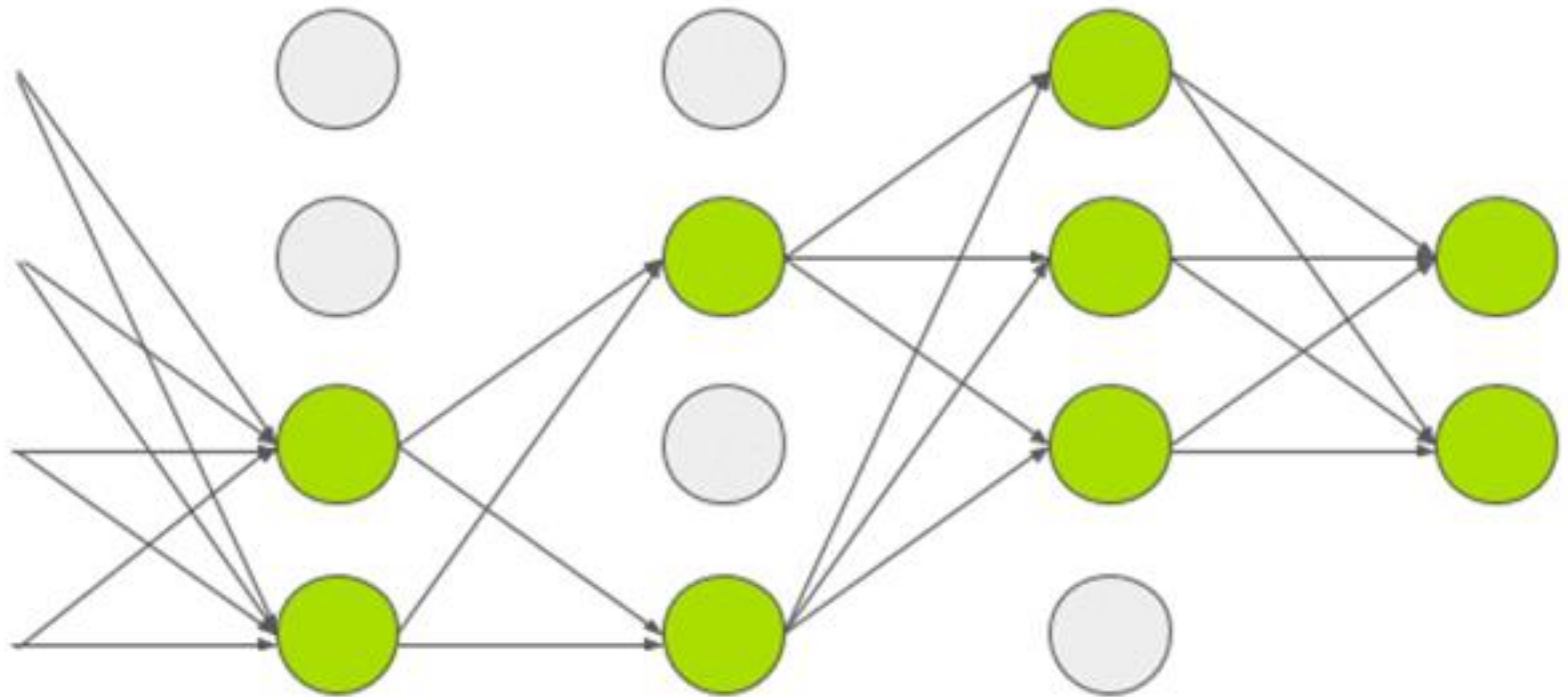
Data Augmentation is carried out like flipping, jittering, cropping, colour normalization, etc.



Key CNN Architectures - AlexNet

Key features of AlexNet

Dropout is carried out in the first two fully-connected layers to reduce overfit



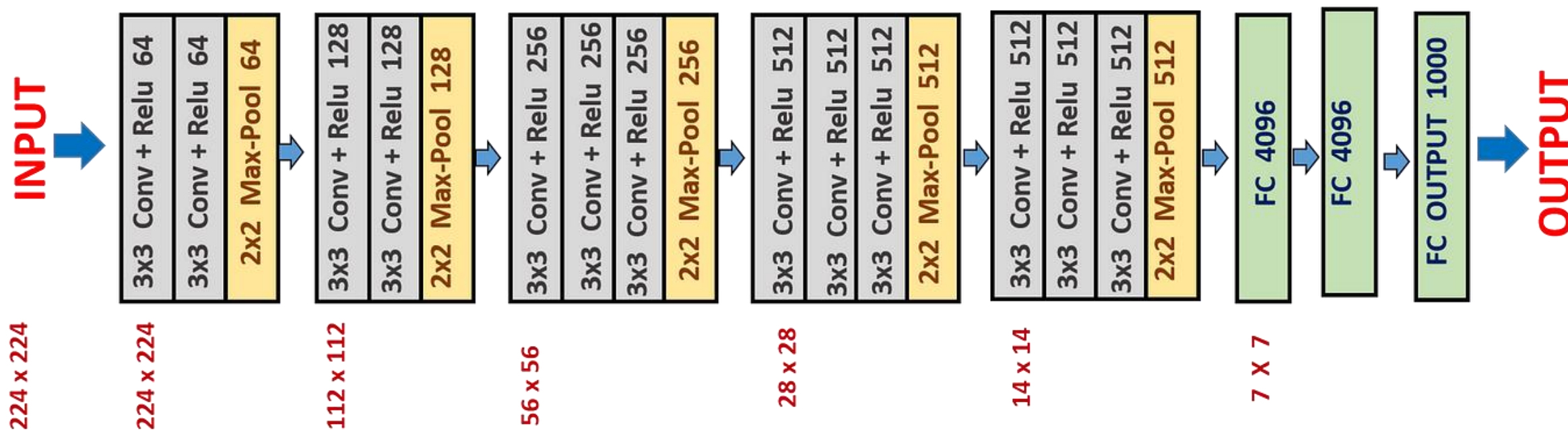
A neuron is removed from the neural network during dropout with a probability of 0.5. A neuron that is dropped does not make any contribution to either forward or backward propagation. As seen in the graphic below, each input is processed by a separate Neural Network design

Key CNN Architectures – VGGNet

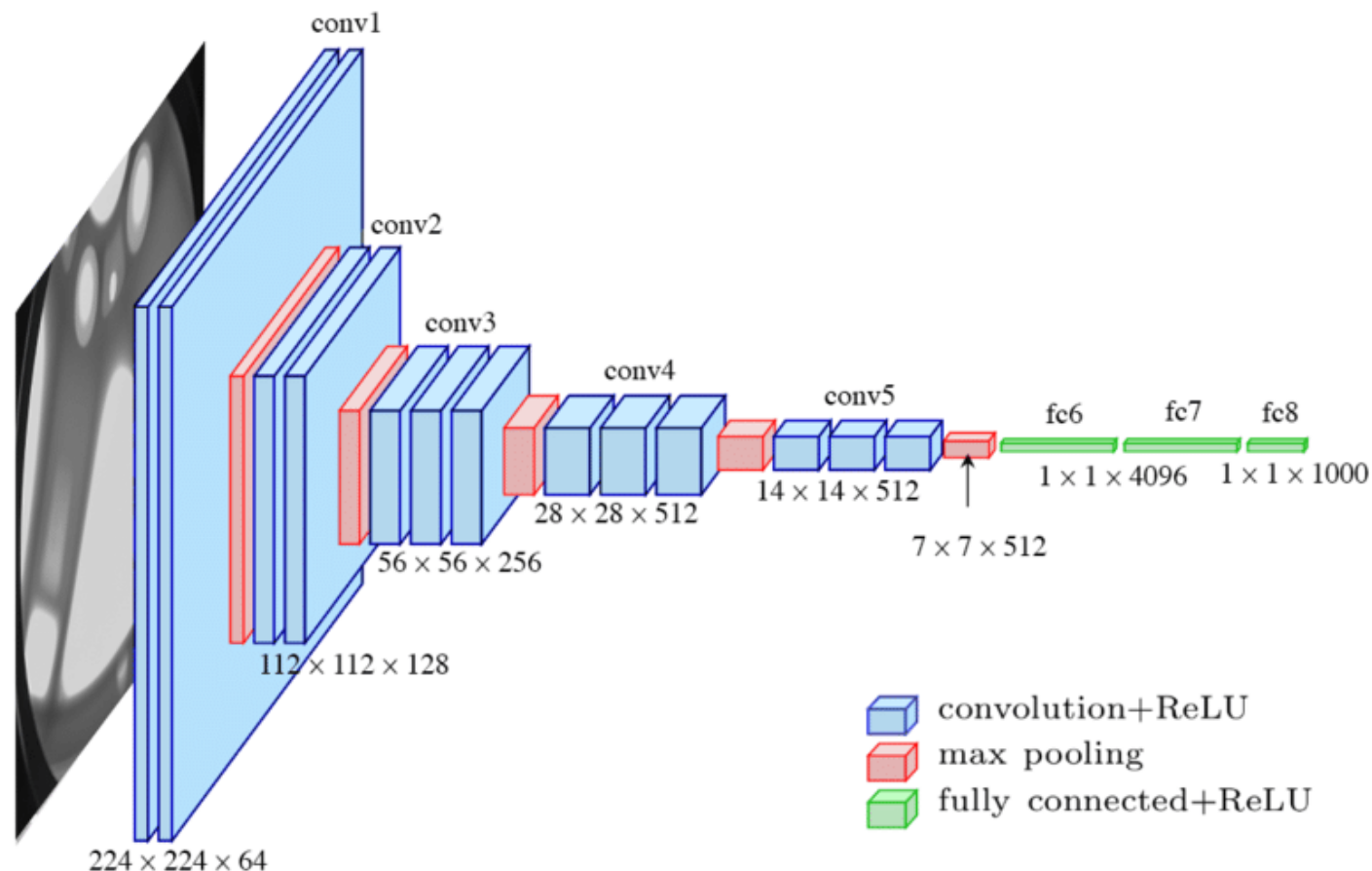
VGGNet (2014) – Depth Matters

VGG stands for Visual Geometry Group. The term “deep” describes the number of layers, with VGG-16 or VGG-19 having 16 or 19 weight layers, respectively.

VGG-16



Key CNN Architectures – VGGNet



Key CNN Architectures – VGGNet

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

Different configurations of VGG-Net

Key CNN Architectures – VGGNet

Key design: VGG-16/VGG-19: Small 3x3 filters, deeper networks.

Impact on design philosophy (simplicity vs. performance)

Limitations Of VGG 16:

It is very slow to train (the original VGG model was trained on Nvidia Titan GPU for 2–3 weeks).

The size of VGG-16 trained imageNet weights is 528 MB. So, it takes quite a lot of disk space and bandwidth which makes it inefficient.

138 million parameters lead to exploding gradients problem.

Key CNN Architectures – GoogLeNet

Inception (GoogLeNet, 2014) – Efficiency

GoogLeNet, also known as Inception-v1, **introduces the concept of inception modules**

Key Features and Innovations:

Inception Modules: GoogLeNet's core innovation is the use of Inception modules. These modules allow the network to learn features at different scales by using multiple filter sizes (1x1, 3x3, and 5x5) in parallel within the same layer.

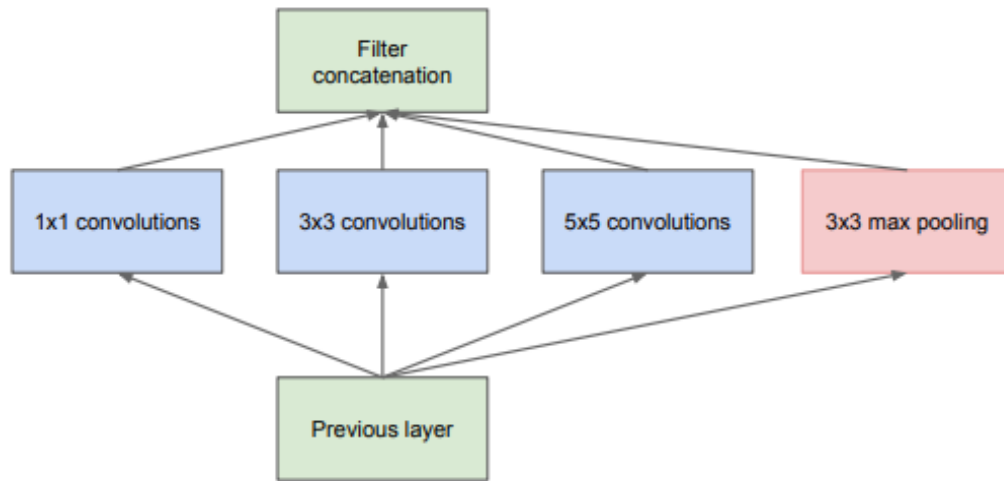
Global Average Pooling: Replacing fully connected layers with global average pooling at the end of the network reduced the number of parameters and improved accuracy.

Auxiliary Classifiers: helps to overcome the issues that are faced commonly by neural networks known as vanishing gradient problem

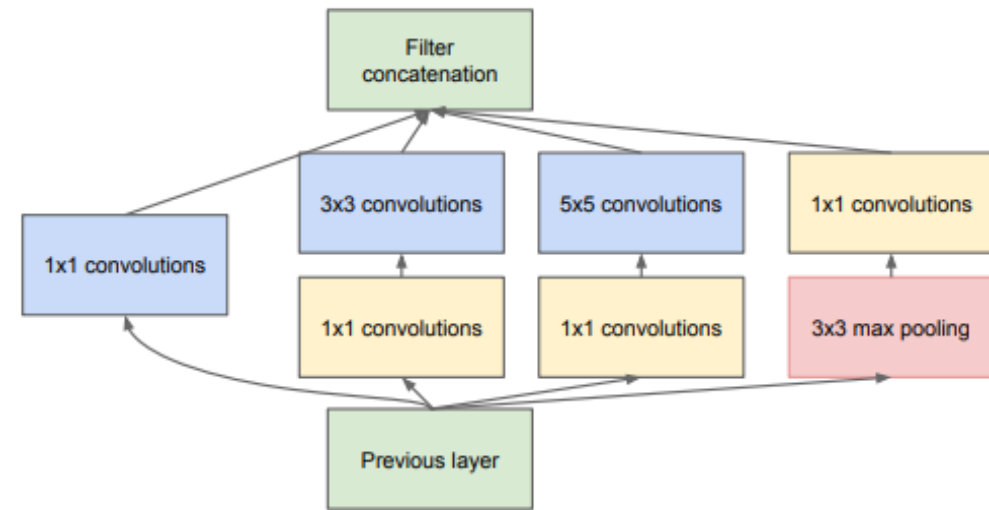


Key CNN Architectures – GoogLeNet

An inception module typically consists of several convolutional layers with different filter sizes. These layers are arranged in parallel, so that the network can process the input data at multiple resolutions simultaneously.



(a) Inception module, naïve version



(b) Inception module with dimension reductions

Key CNN Architectures – GoogLeNet

The inception module is made up of four paths

- **1x1 convolution:** This path applies a 1x1 convolution to the input. This reduces the number of channels in the input, which helps to reduce the computational complexity of the network.
- **3x3 convolution:** This path applies a 3x3 convolution to the input. This is a standard convolutional operation that is used to extract features from the input image.
- **5x5 convolution:** This path applies a 5x5 convolution to the input. This path is used to capture larger-scale features from the input image.
- **Max pooling:** This path applies a max pooling operation to the input. This operation reduces the size of the input by keeping the maximum value in each 2x2 window.

Bottleneck
layers

The inception module is made up of four paths

ception module

Diagram illustrating the Inception module architecture, showing the flow from the Previous Activation to the Channel Concat output.

The input is the Previous Activation ($28 \times 28 \times 192$).

The module branches into four parallel paths:

- Path 1: 1×1 CONV (96) $\rightarrow$ 3×3 CONV ($28 \times 28 \times 128$)
- Path 2: 1×1 CONV (16) $\rightarrow$ 5×5 CONV ($28 \times 28 \times 32$)
- Path 3: MAXPOOL ($3 \times 3, s=1$, same) $\rightarrow$ 1×1 CONV ($28 \times 28 \times 32$)
- Path 4: 1×1 CONV ($28 \times 28 \times 64$)

The outputs of these four paths are combined in the Channel Concat block, resulting in the final output: $28 \times 28 \times 25.6$.

Key CNN Architectures – GoogLeNet

In GoogLeNet architecture, replacing fully connected layers with global average pooling improved the top-1 accuracy by about 0.6%

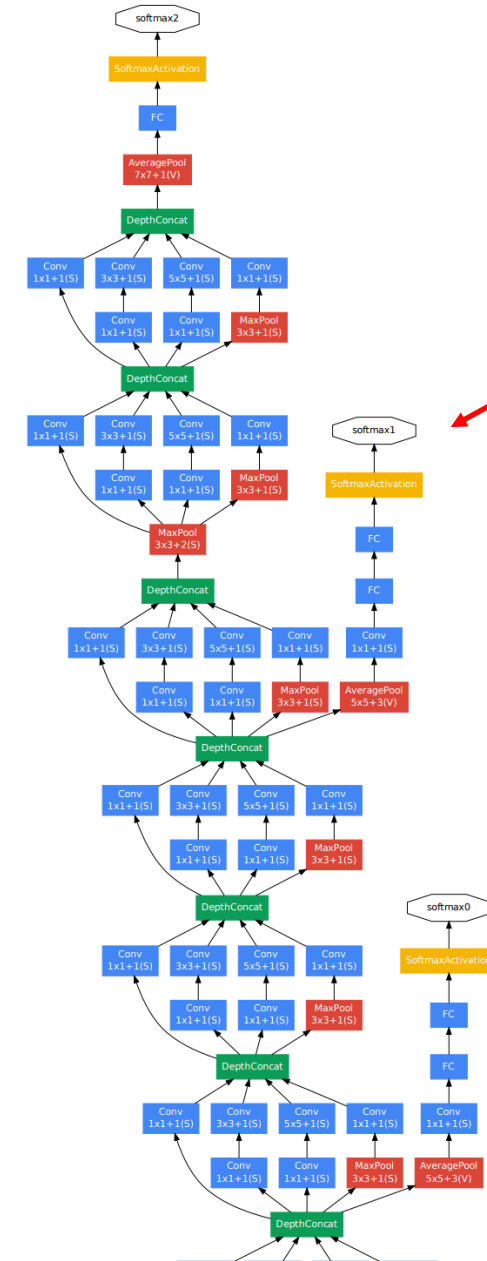
type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

Key CNN Architectures – GoogLeNet

Auxiliary Classifiers are placed on the side of the network and are only included in training and not during inferences or testing.

In the GoogLeNet architecture, there are two auxiliary classifiers in the network. They are placed strategically, where the depth of the feature extracted is sufficient to make a meaningful impact, but before the final prediction from the output classifier.

- An average pooling layer with a 5×5 window and stride 3.
- A 1×1 convolution for dimension reduction with 128 filters.
- Two fully connected layers, the first layer with 1024 units, followed by a dropout layer and the final layer corresponding to the number of classes in the task.
- A SoftMax layer to output the prediction probabilities.



Key CNN Architectures – GoogLeNet

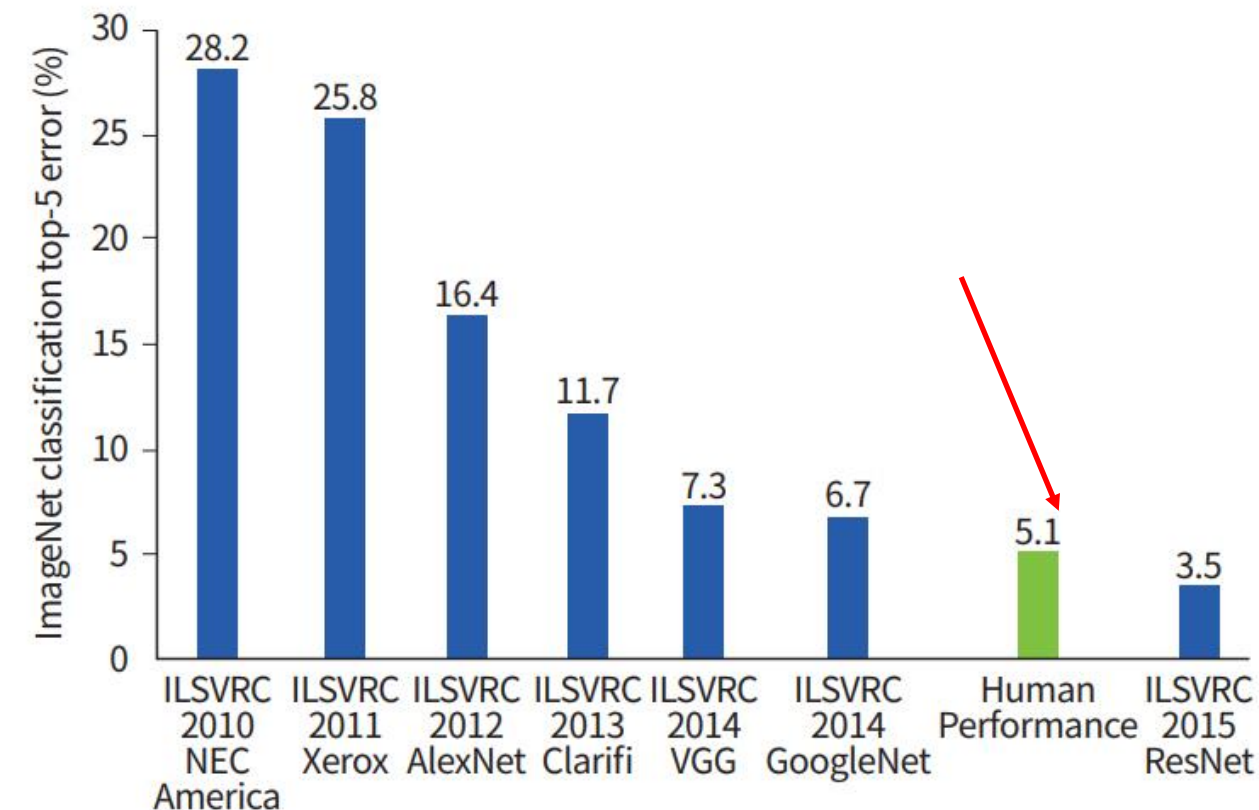
GoogLeNet Versions

Inception v2 (2015): The second version of Inception was modified with improvements such as **batch normalisation** and **shortcut connections**. It also refined the inception modules by replacing larger convolutions with smaller, more efficient ones. These changes improved accuracy and reduced training time.

Inception v3 (2015): The v3 model further refined Inception v2 by using **atrous convolution** (dilated convolutions that expand the network's receptive field without sacrificing resolution and significantly increasing network parameters).

Inception v4, Inception-ResNet v2 (2016): This version of Inception introduced **residual connections** (inspired by ResNet) into the Inception lineage, which led to further performance improvements.

Key CNN Architectures – ResNet



Kaiming He

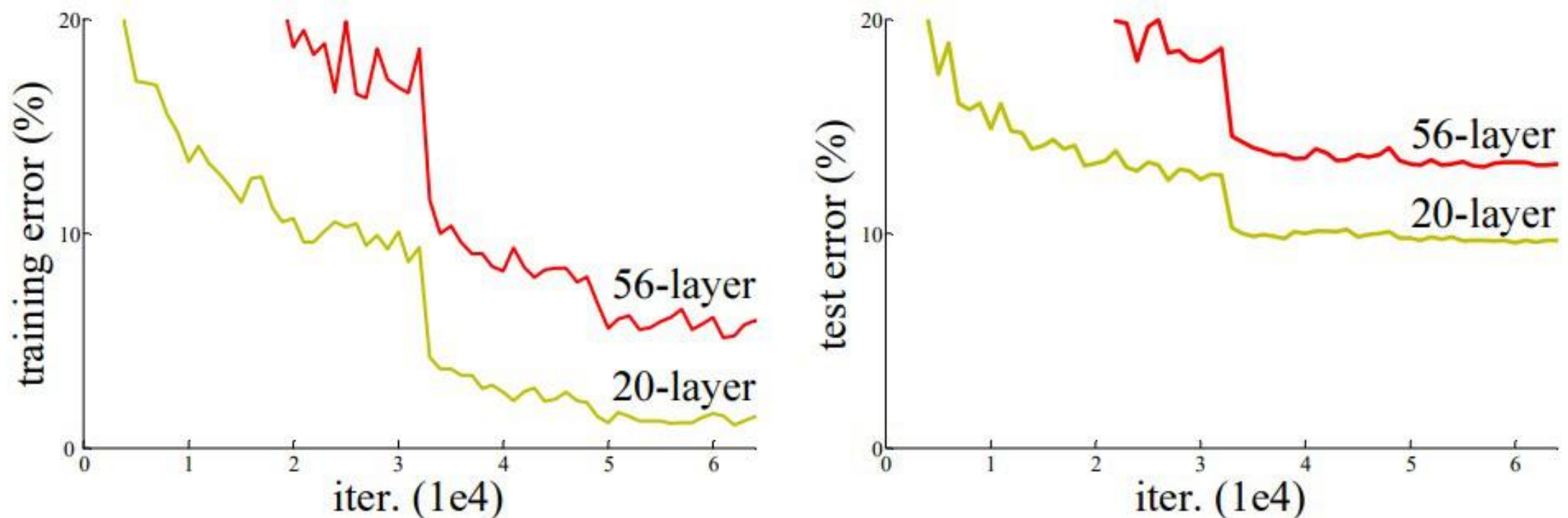
Associate Professor, EECS, [MIT](#)
Verified email at mit.edu - [Homepage](#)
[Computer Vision](#) [Machine Learning](#)

[FOLLOW](#)

TITLE	CITED BY	YEAR
Deep Residual Learning for Image Recognition K He, X Zhang, S Ren, J Sun Computer Vision and Pattern Recognition (CVPR), 2016	279300	2016
Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks S Ren, K He, R Girshick, J Sun Neural Information Processing Systems (NIPS), 2015	91673	2015

Key CNN Architectures – ResNet

When we add more layers to neural network: **Vanishing/Exploding gradient**. This results in the gradient becoming zero or being overly large. Therefore, the training and test error rate similarly increases as the number of layers is increased, as shown by the following figure



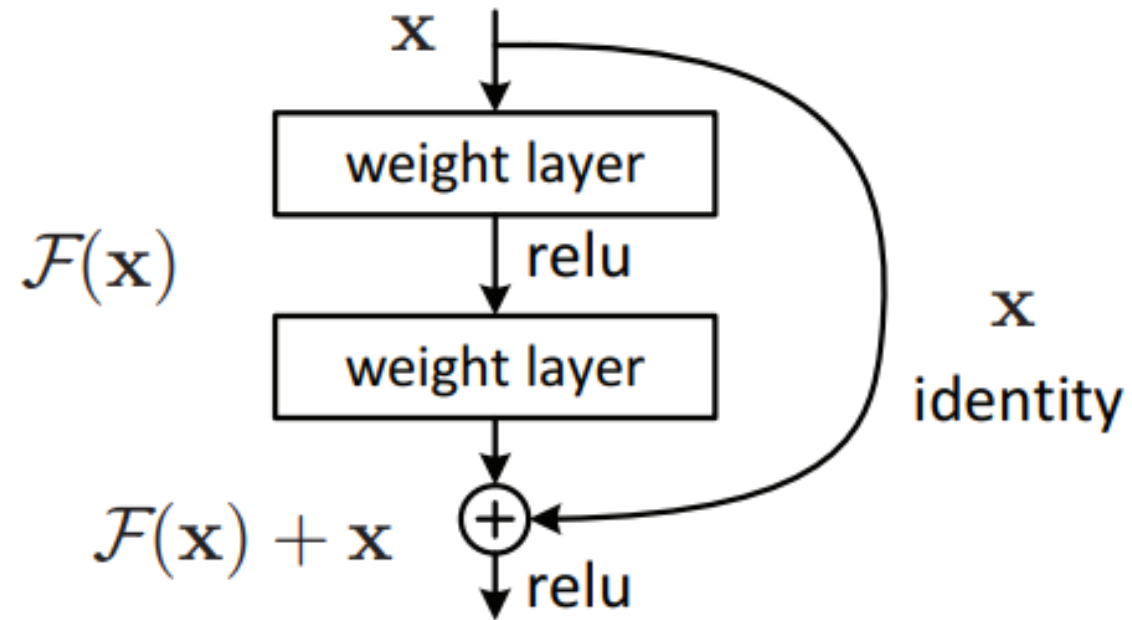
Key CNN Architectures – ResNet

ResNet introduces the **Residual Blocks** to address the issue of the vanishing/exploding gradient

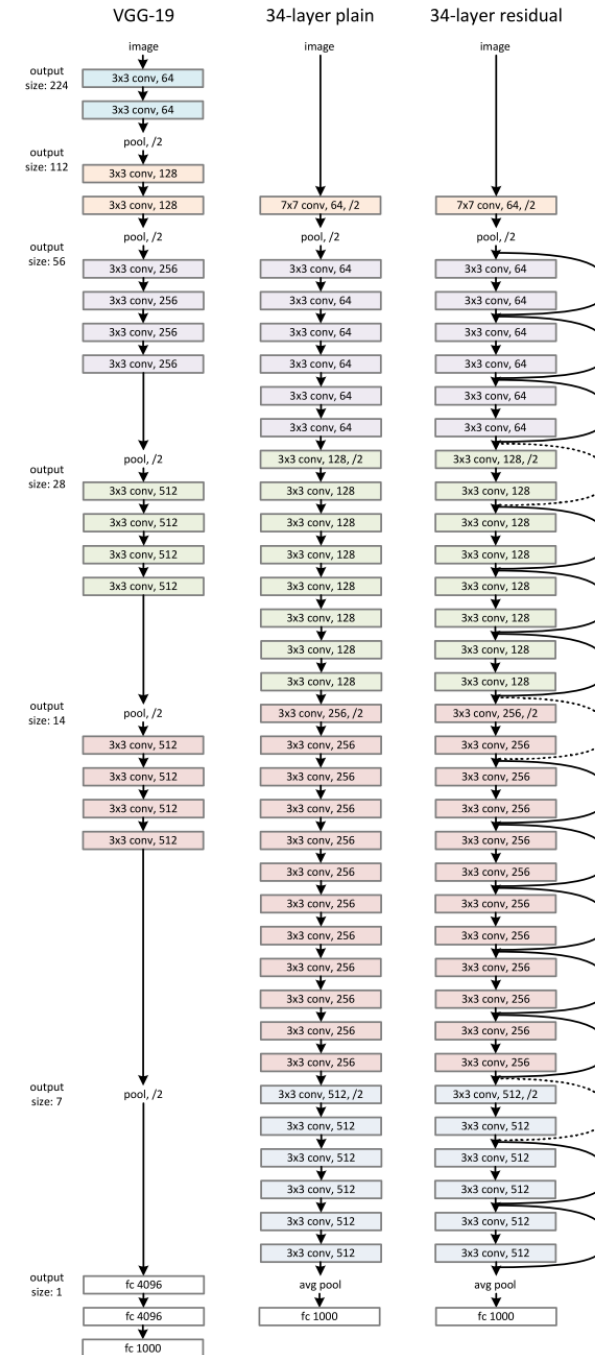
Skip connections is applied in this network. The skip connection bypasses some levels in between to link-layer activations to subsequent layers. This creates a leftover block. These leftover blocks are stacked to create Resnet

The strategy behind this network is to let the network fit the **residual mapping** rather than have layers learn the underlying mapping

The benefit of including this kind of skip link is that regularisation will skip any layer that degrades architecture performance. As a result, training an extremely deep neural network is possible without encountering issues with vanishing or expanding gradients.



Key CNN Architectures – ResNet



Key CNN Architectures – ResNet

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Different variants of ResNet for ImageNet

Key CNN Architectures – EfficientNet

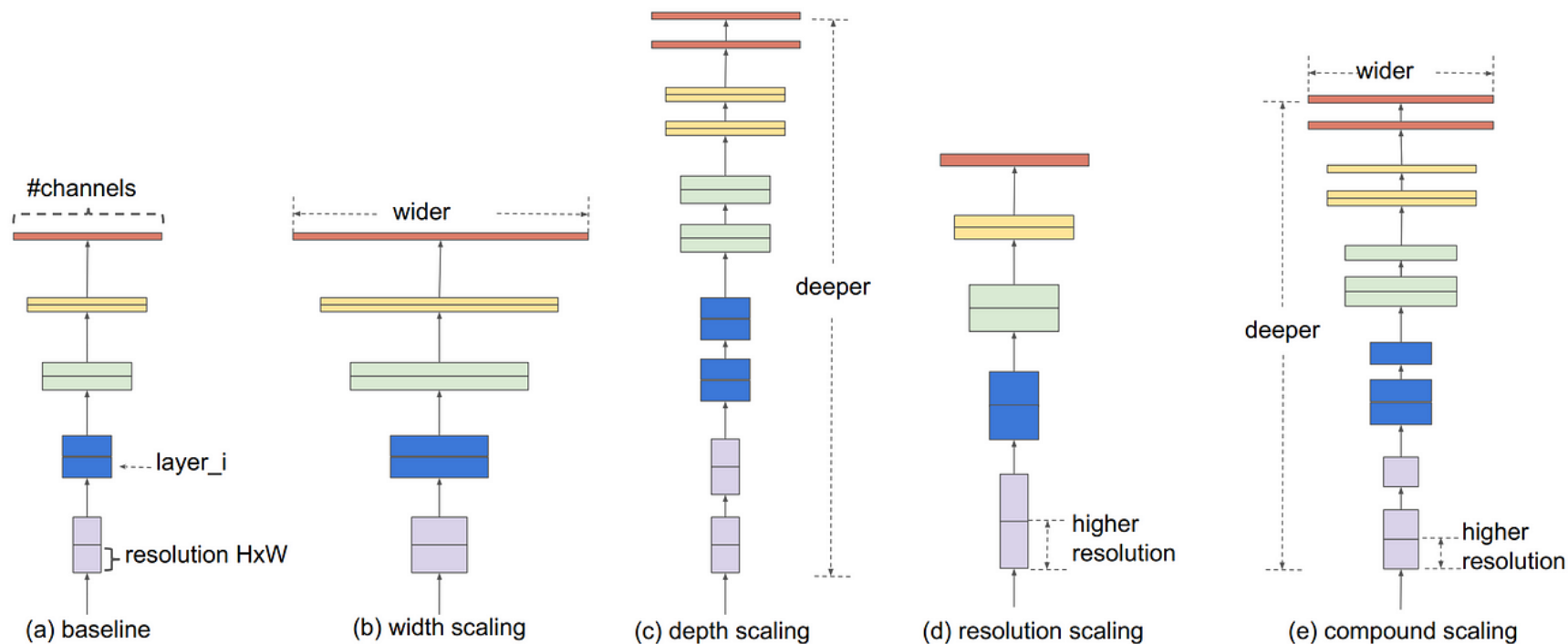
EfficientNet (2019) – Compound Scaling

EfficientNet uses a technique called **compound coefficient** to scale up models in a simple but effective manner. Instead of randomly scaling up width, depth or resolution, compound scaling uniformly scales each dimension with a certain fixed set of scaling coefficients

Using the scaling method and AutoML, the authors of efficient developed seven models of various dimensions, which surpassed the state-of-the-art accuracy of most convolutional neural networks, and with much better efficiency.

Key CNN Architectures – EfficientNet

While scaling single dimensions helps improve model performance, balancing the scale in all the three dimensions — width, depth, and image resolution — considering the variable available resources best improve the overall model performance.



Key CNN Architectures – EfficientNet

The compound scaling method is based on the idea of balancing dimensions of width, depth, and resolution by scaling with a constant ratio

$$\text{Depth } d = \alpha^\phi, \text{ Width } w = \beta^\phi, \text{ Resolution } r = \gamma^\phi, (1)$$

$$\text{such that } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

Where the values of α, β, γ are determined using a grid search algorithm. ϕ determines the increase in computational resources to the network. This parameter is user defined. They are bound by the factor that denotes that $\alpha \cdot \beta^2 \cdot \gamma^2$ is close to 2. FLOPS (Floating Point Operations Per Second) of a convolution operation is proportional to d, w^2, r^2 , which denotes that the FLOPS will double on doubling the network depth. Still, on doubling network width or resolution, the FLOPS will increase four times. In CNN's, the convolutional operations control the computational costs. Scaling the network using the above equation will increase the FLOPS by $(\alpha \cdot \beta^2 \cdot \gamma^2)^\phi$. Since $\alpha \cdot \beta^2 \cdot \gamma^2$ is approximately 2, so for any new ϕ , the total FLOPS increase by 2^ϕ .

The intuition for the networks is, if **the input image is bigger**, then the network needs **more layers** to increase the receptive field and **more channels** to capture more fine-grained patterns on the bigger image.

Key CNN Architectures – EfficientNet

Architecture

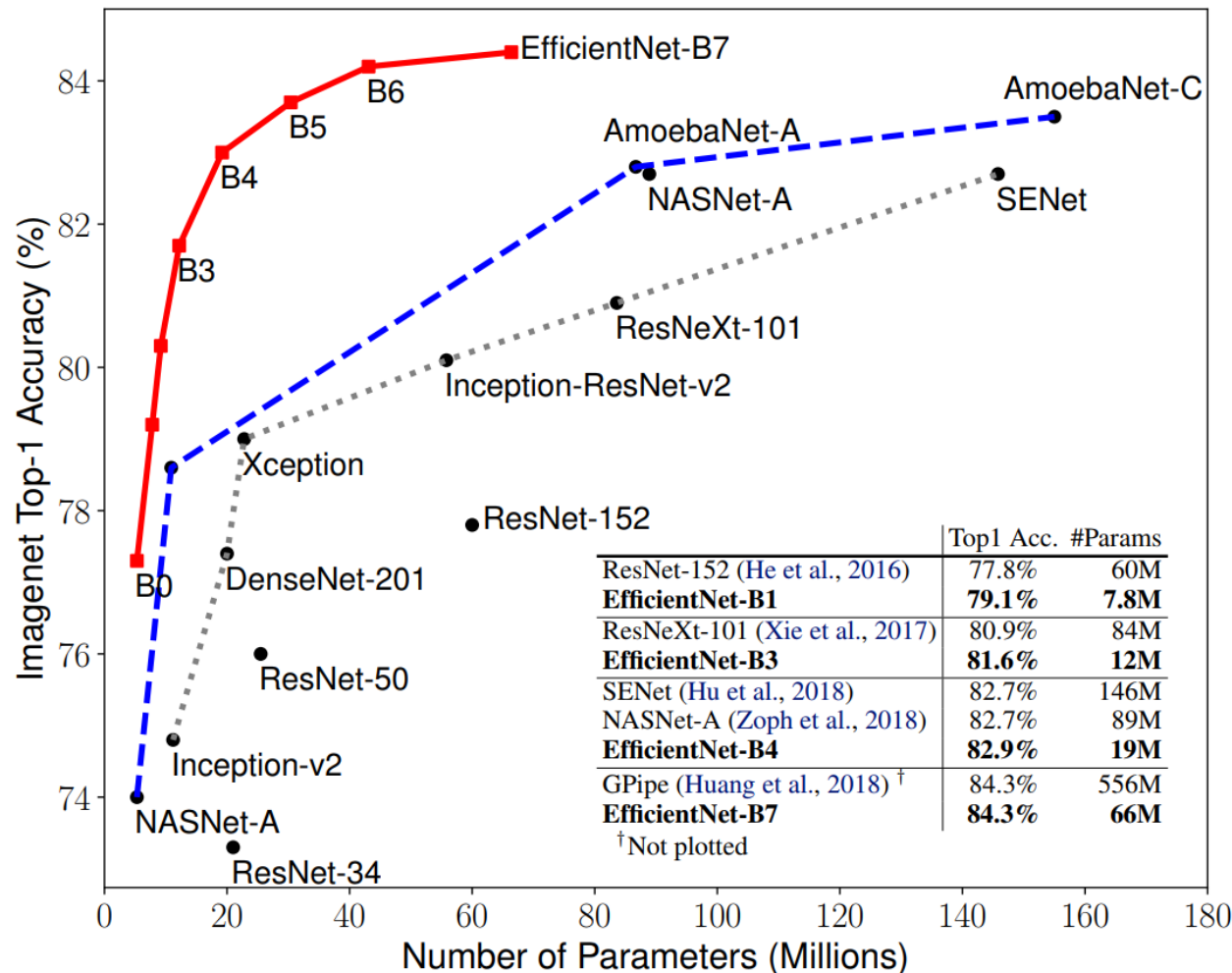
EfficientNet is based on the baseline network developed by the neural architecture search using the AutoML MNAS framework.

The network is fine-tuned for obtaining maximum accuracy but is also penalized if the network is very computationally heavy.

It is also penalized for slow inference time when the network takes a lot of time to make predictions.

The architecture uses a mobile inverted bottleneck convolution similar to MobileNet V2 but is much larger due to the increase in FLOPS. This baseline model is scaled up to obtain the family of EfficientNets.

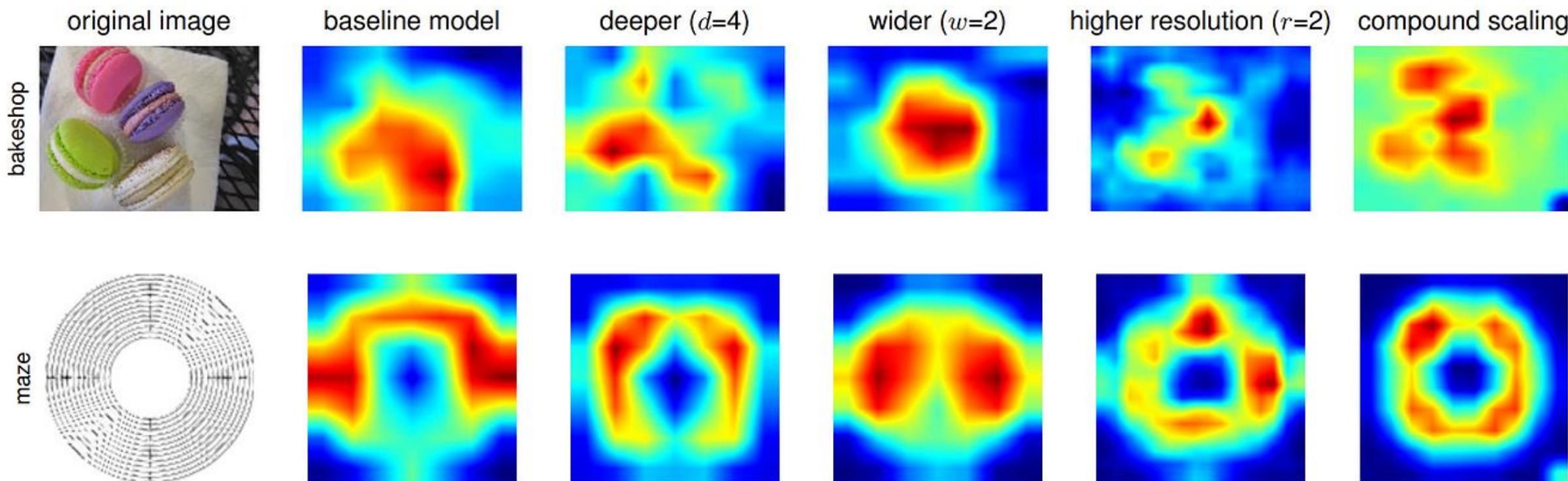
Key CNN Architectures – EfficientNet



EfficientNet size and performance on the ImageNet dataset

Key CNN Architectures – EfficientNet

EfficientNet is also seen to provide better Class Activation Maps (CAM), which focused more on the relevant regions with more object details, paving the way towards better model explainability



The compound scaling method allows the scaled model (last column) to focus on more relevant regions with more object details

Image Classification

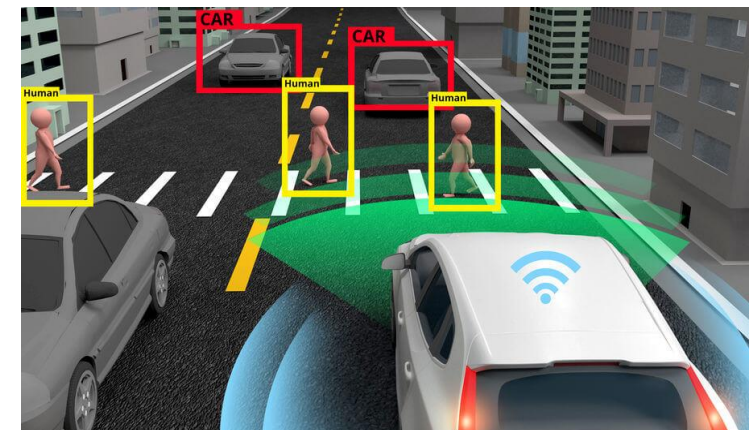
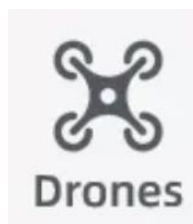
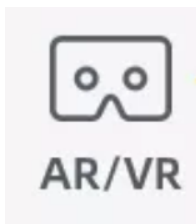
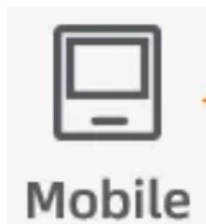
- Traditional Methods (Pre-Deep Learning)
 - Key Handcrafted Feature Extraction Methods
 - Local Feature Descriptors
 - Global Feature Descriptors
 - Bag-of-Visual-Words (BoVW) Representation
 - Limitations on Handcrafted Features
- Image Classification in Deep Learning Era
 - From Perceptrons to CNNs
 - Key CNN Architectures
 - **Lightweight Models**

Lightweight Models

Why Lightweight Models?

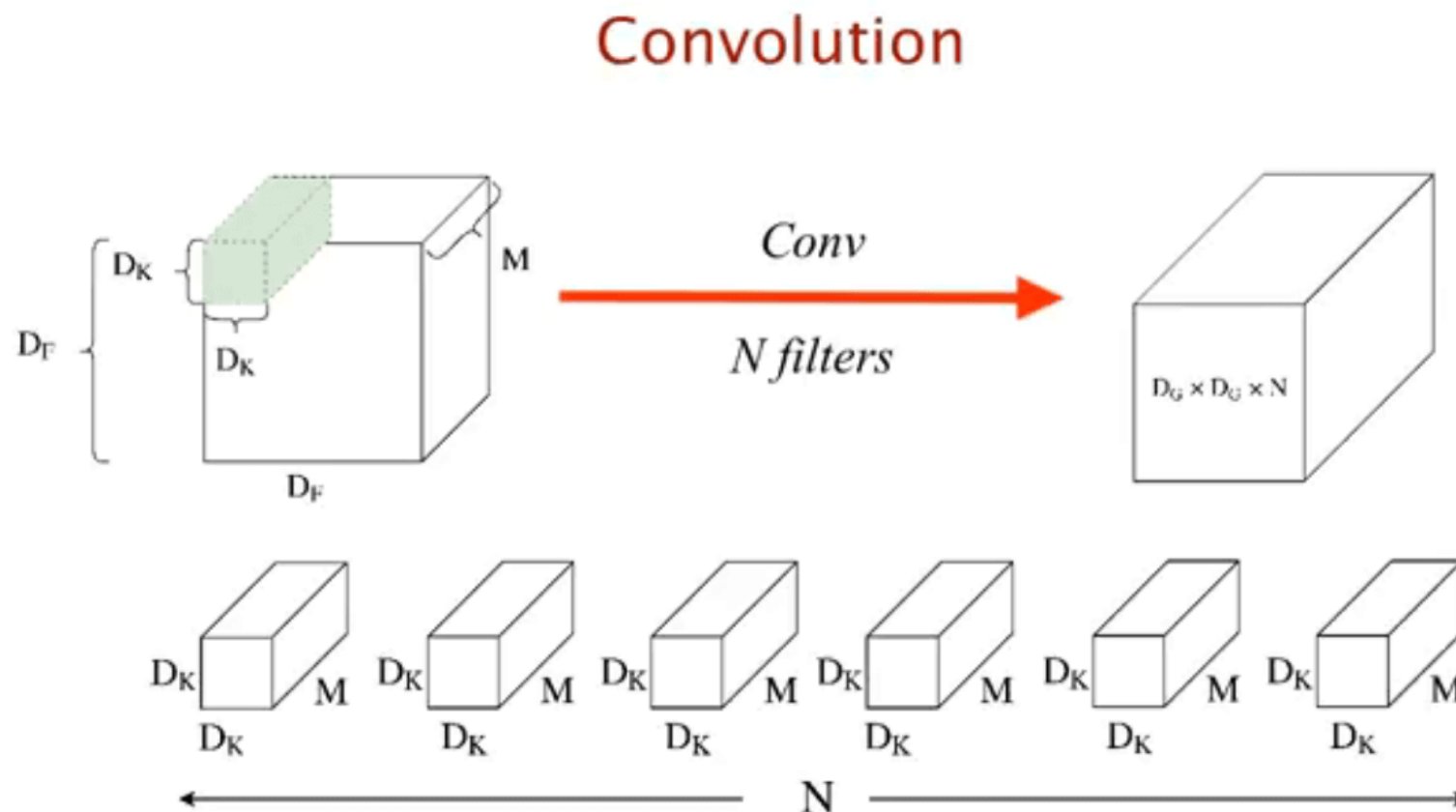
Edge devices (phones, drones, IoT) →
constraints on compute, memory,
power

Latency-critical applications (AR,
autonomous systems)



Lightweight Models

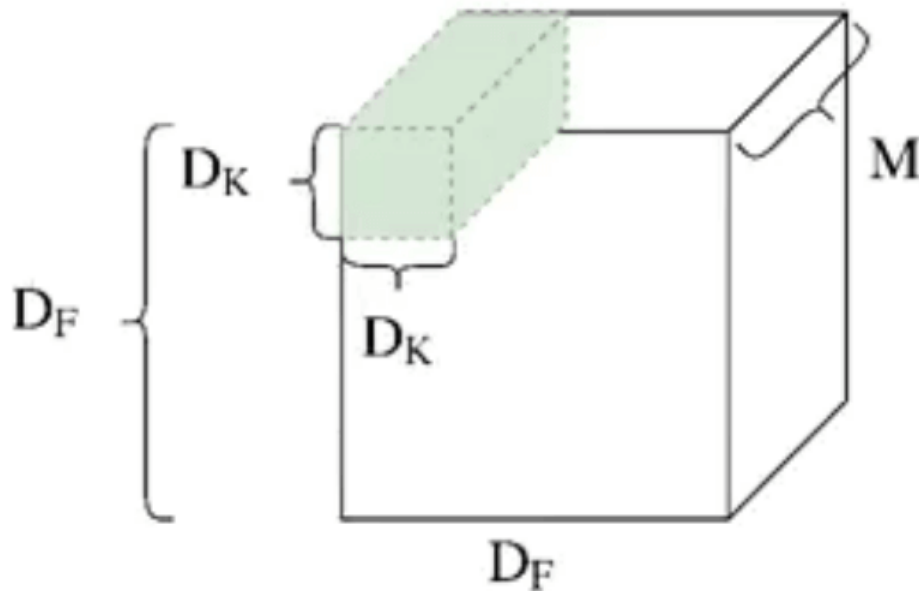
Key Design Strategies for Efficiency: Depthwise separable convolutions (Xception, MobileNet)



Lightweight Models

Computational cost in standard convolution

Convolution



$$\text{Mults once} = D_K^2 \times M$$

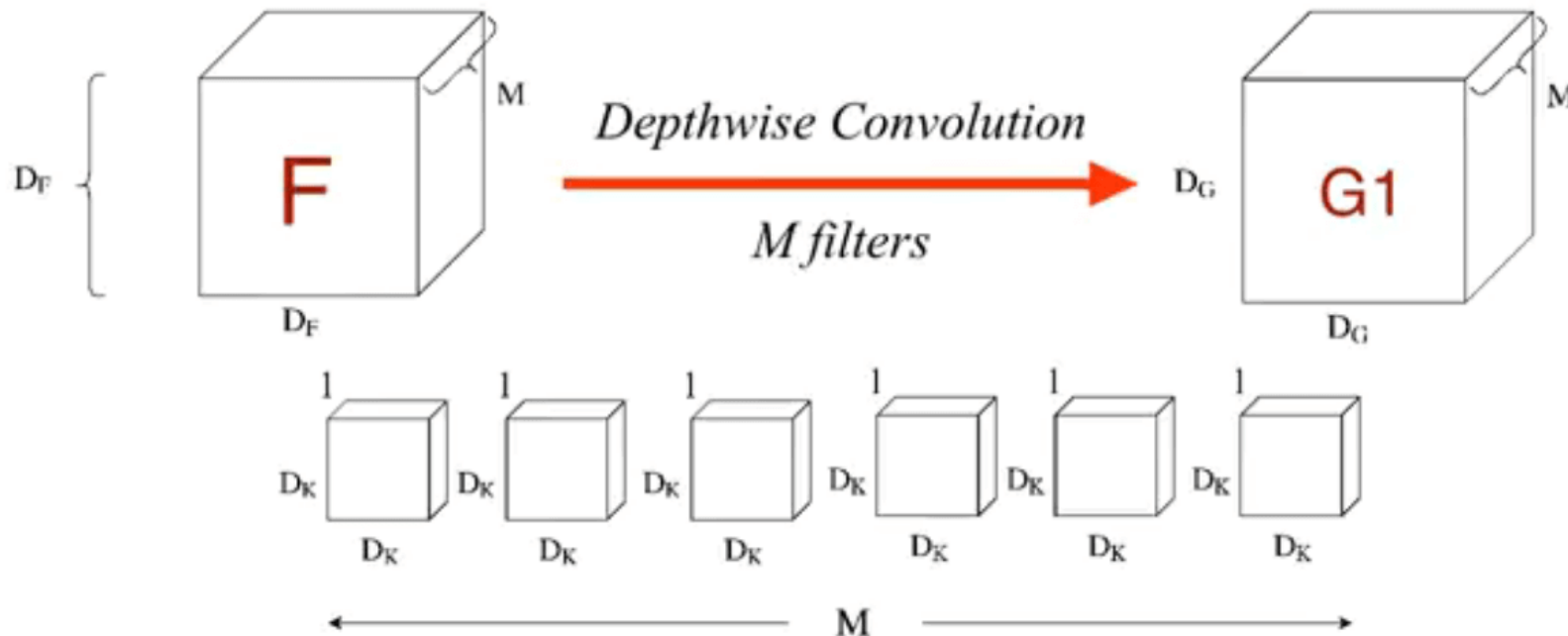
$$\text{Mults per Kernel} = D_G^2 \times D_K^2 \times M$$

$$\text{Mults N Kernels} = N \times D_G^2 \times D_K^2 \times M$$

Lightweight Models

Depthwise Separable Convolution --> **depthwise convolution (spatial filtering per channel)** + pointwise convolution (channel mixing)

1. Depthwise Convolution: Filtering Stage



Applies a single filter per input channel (unlike standard convolution, which applies multiple filters across all channels).

Does not mix channels—each channel is processed independently.

$$\text{Mults once} = D_K^2$$

$$\text{Mults 1 Channel} = D_G^2 \times D_K^2$$

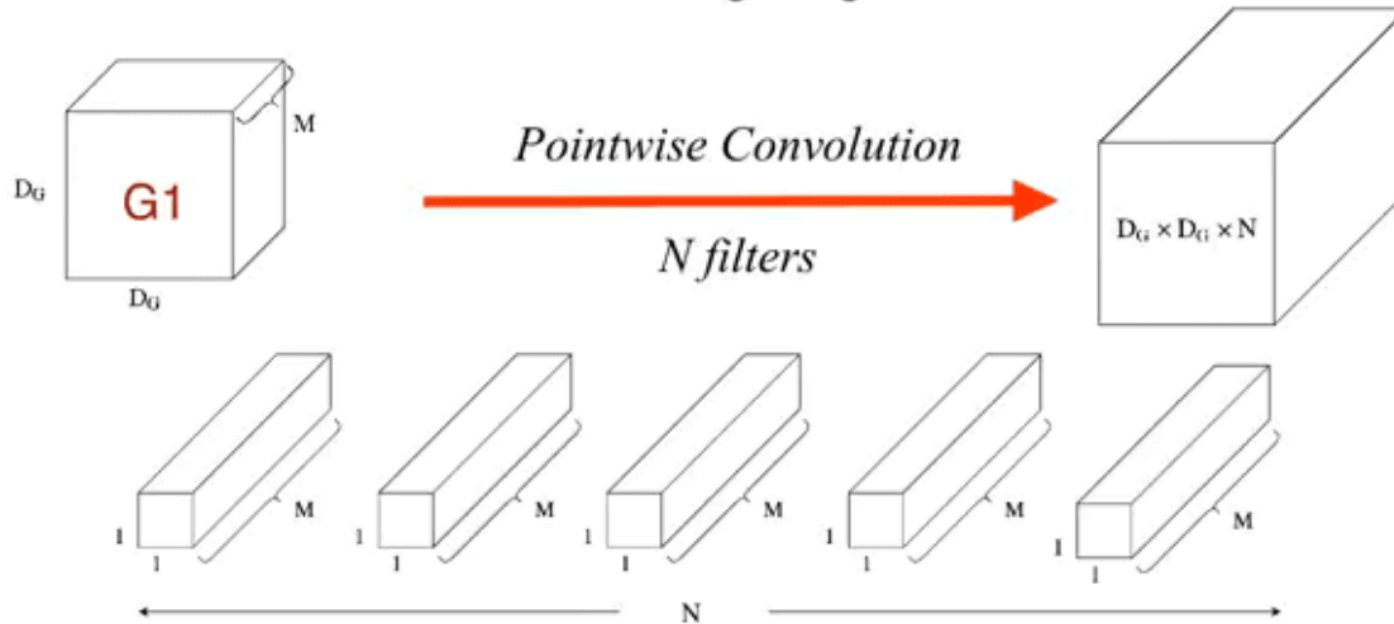
$$\text{DC Mults} = M \times D_G^2 \times D_K^2$$



Lightweight Models

Depthwise Separable Convolution --> depthwise convolution (spatial filtering per channel) + **pointwise convolution (channel mixing)**

2. Pointwise Convolution: Filtering Stage



Mixes channels using a 1×1 convolution.

Mults once = M

Mults 1 Kernel = $D_G \times D_G \times M$

PC Mults = $N \times D_G \times D_G \times M$

Lightweight Models

Standard Convolution

$$\text{Mults once} = D_K^2 \times M$$

$$\text{Mults per Kernel} = D_G^2 \times D_K^2 \times M$$

$$\text{Mults N Kernels} = N \times D_G^2 \times D_K^2 \times M$$



$$\frac{\text{No. params in Depthwise Separable Conv}}{\text{No. params in Standard Conv}} = \frac{M \times (D_K^2 + N)}{N \times D_K^2 \times M} = \frac{1}{N} + \frac{1}{D_K^2}$$

Depthwise Separable Convolution

$$\text{Mults once} = D_K^2$$

$$\text{Mults 1 Channel} = D_G^2 \times D_K^2$$

$$\text{DC Mults} = M \times D_G^2 \times D_K^2$$

$$\text{Mults once} = M$$

$$\text{Mults 1 Kernel} = D_G \times D_G \times M$$

$$\text{PC Mults} = N \times D_G \times D_G \times M$$

$$\text{Total} = \text{DC Mults} + \text{PC Mults}$$

$$M \times D_G^2 \times D_K^2 + N \times D_G^2 \times M$$

$$M \times D_G^2 (D_K^2 + N)$$

Lightweight Models

Why Does This Work?

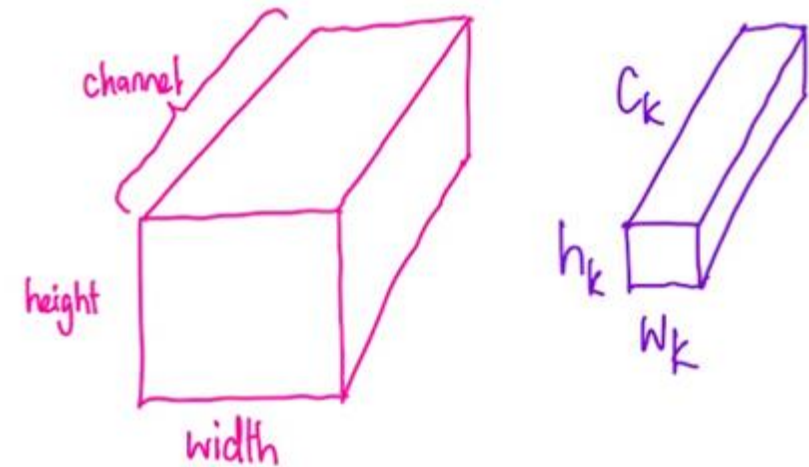
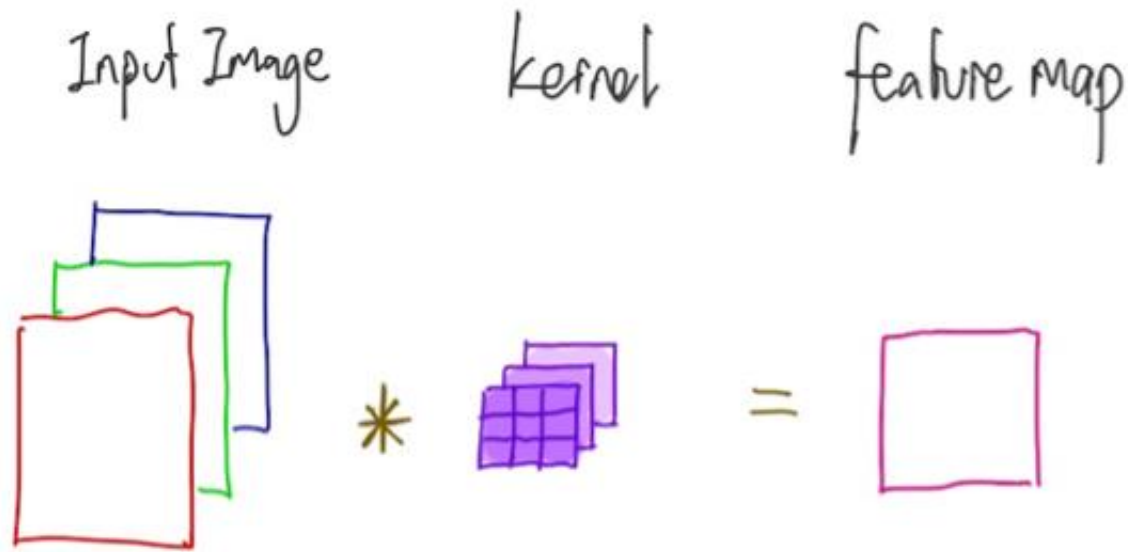
Decouples spatial and channel-wise operations:

- Depthwise convolution captures **spatial features** (edges, textures).
- Pointwise convolution combines **channel-wise information**.

Reduces redundancy by avoiding full channel mixing in the spatial step.

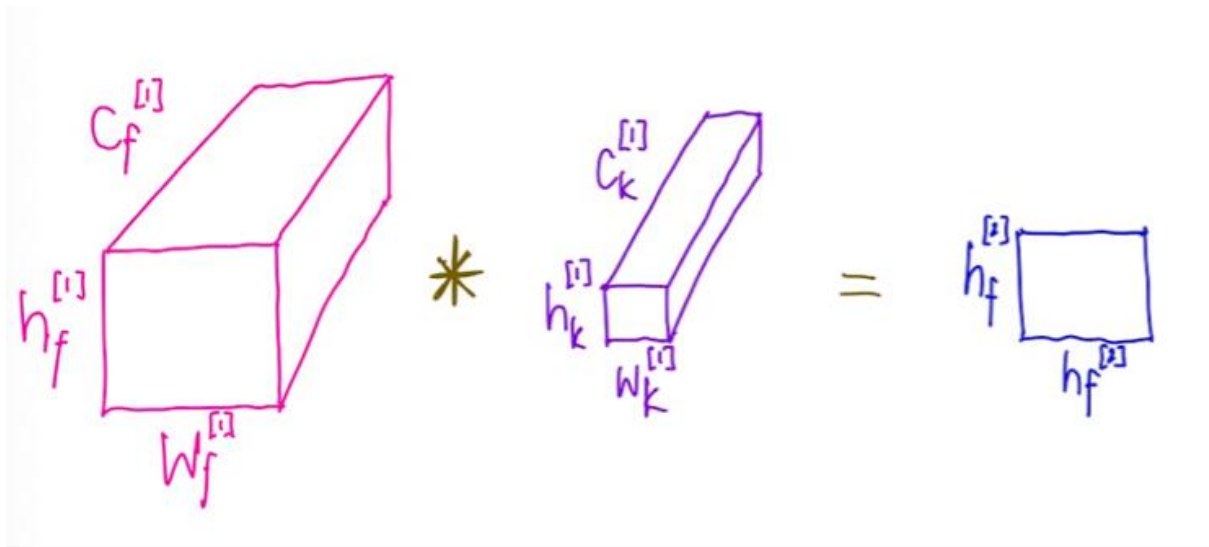
Lightweight Models

Key Design Strategies for Efficiency: Channel pruning

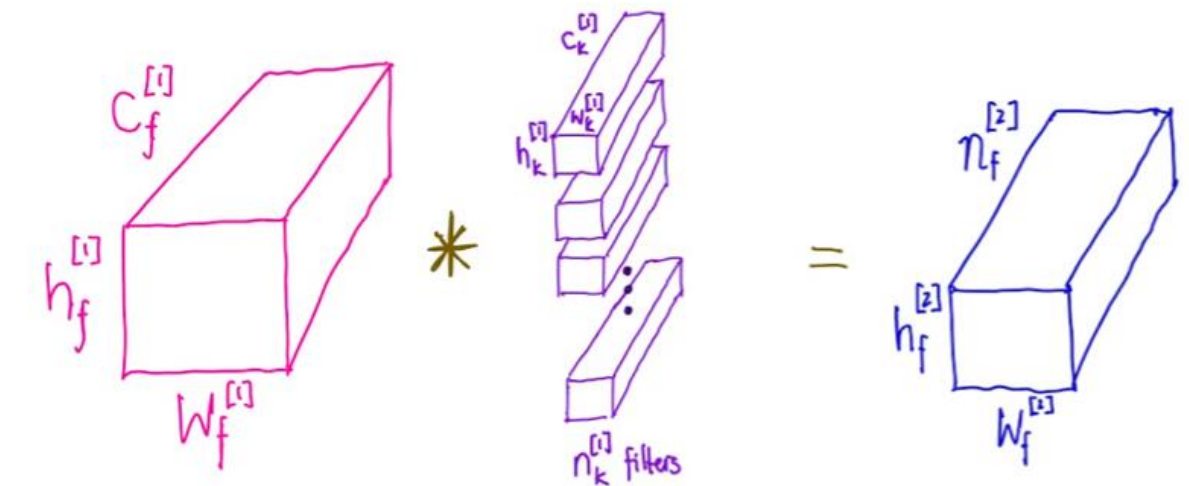


Lightweight Models

Key Design Strategies for Efficiency: Channel pruning



In a 2D-convolutional forward pass, the feature map will be convolved by a filter block with matching number of channels which will yield a single channel feature map.

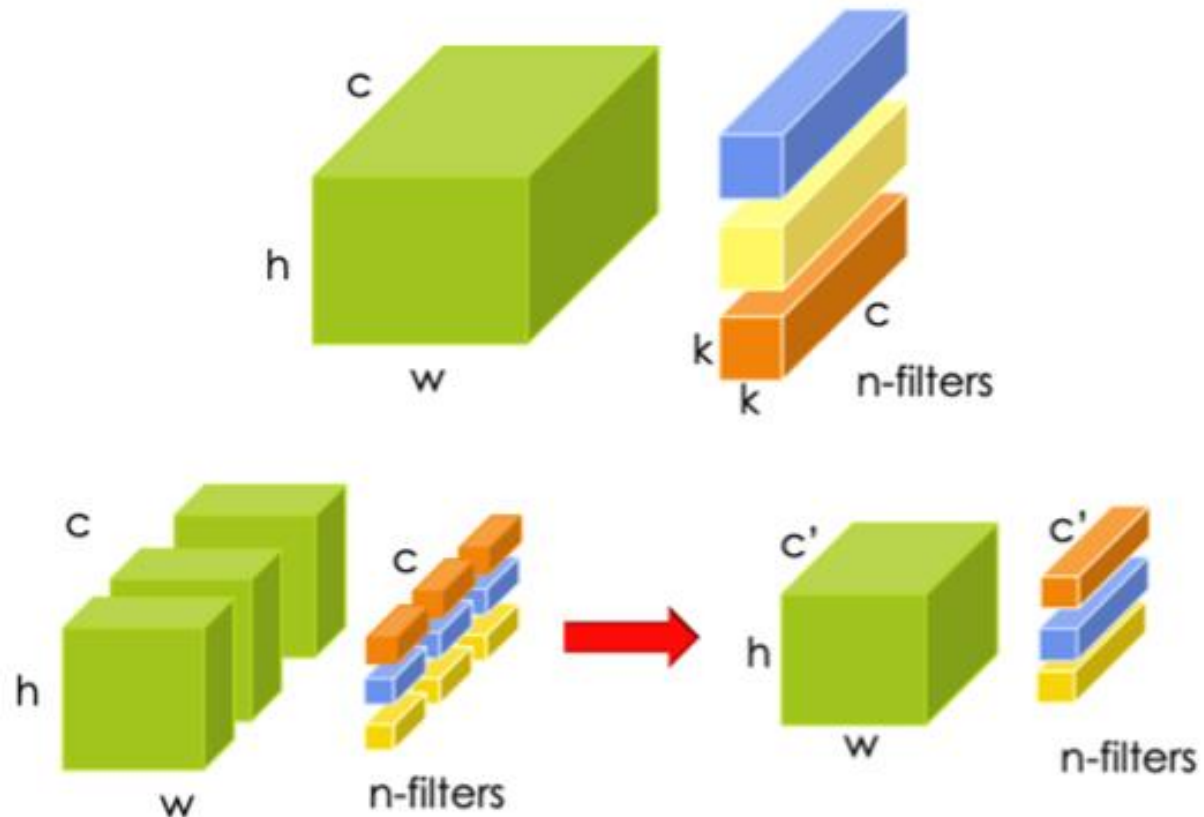


The width and height of the feature map will most likely be smaller or the same size as the input image, but the number of feature map's channels will equal to the number filters.

Lightweight Models

Key Design Strategies for Efficiency: Channel pruning

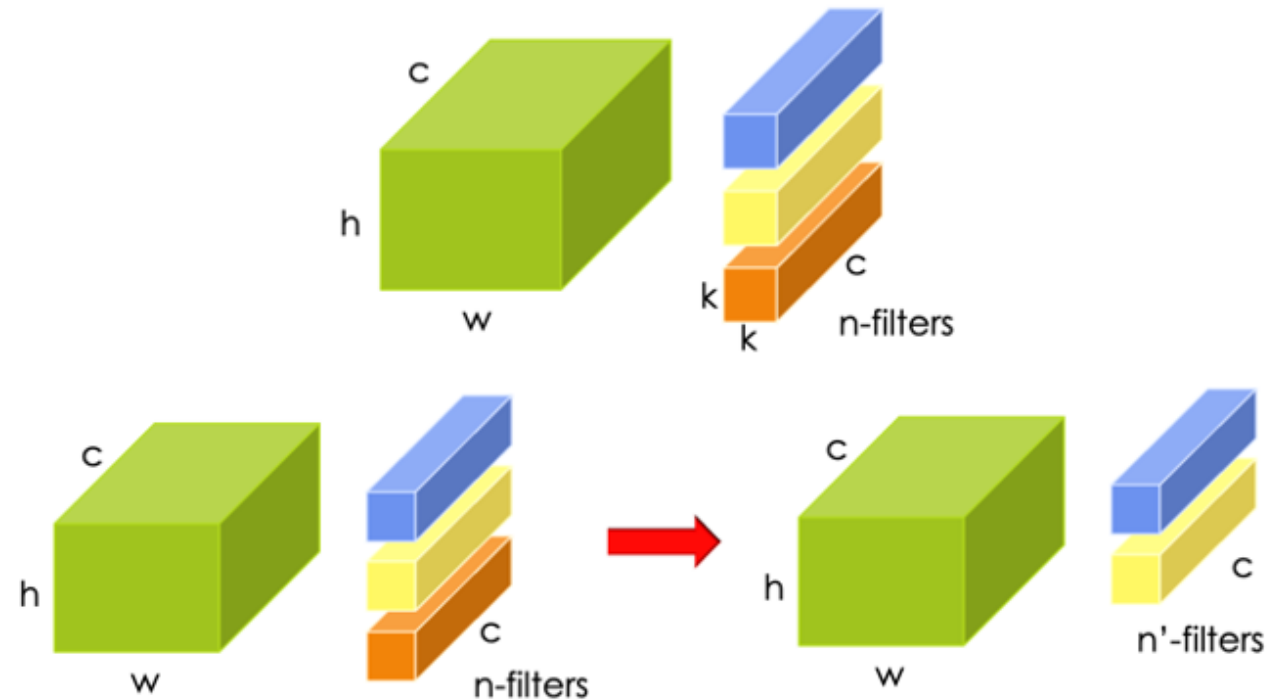
Prune the least significant channels. Channels that are similar or redundant in a filter can be eliminated without affecting the overall generalizability of the entire network. This also means that the corresponding feature map channels in the current and next layer can also be eliminated, thus reducing the computation of the network even more.



Lightweight Models

Key Design Strategies for Efficiency: Channel pruning

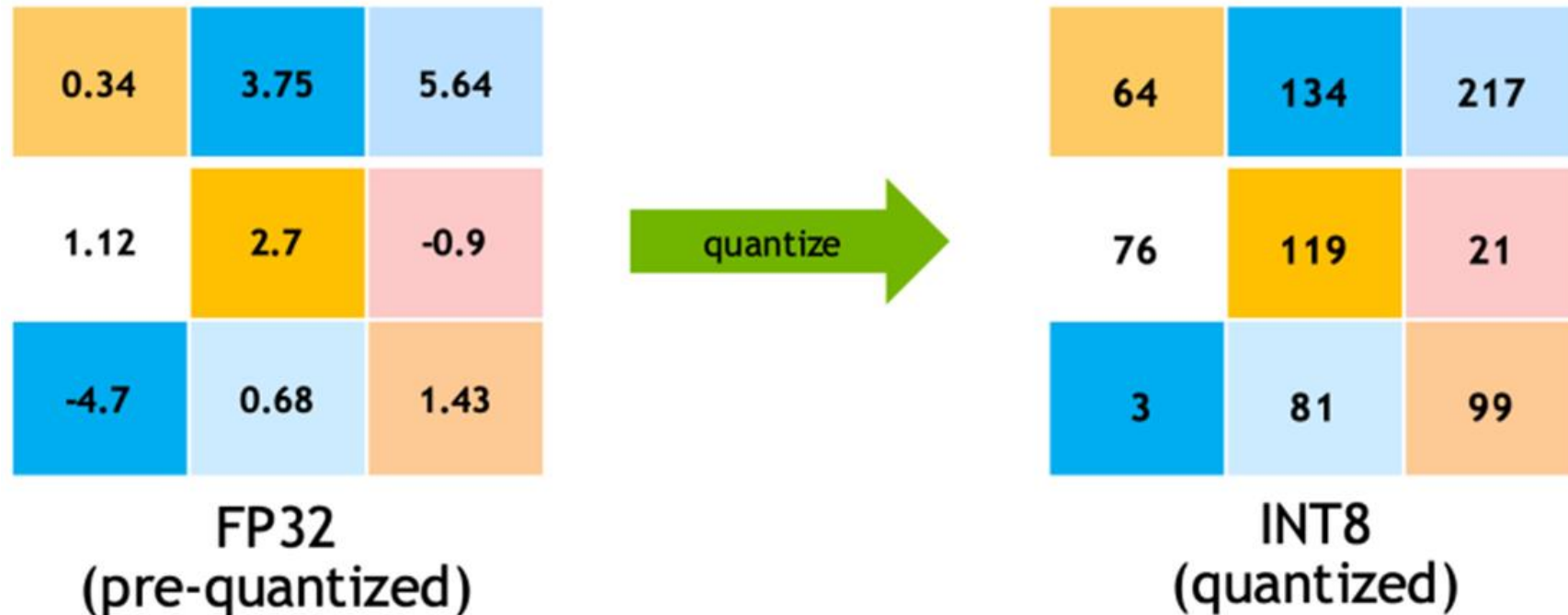
Prune filters within a layer of the network. The general steps are as follow: identify which filters are the least important, prune those filters, retrained the model, and repeat. There are many metrics on how to determine the importance of a filter. The simplest is calculating the sum of the absolute kernel weights for each kernel, and then sort them



Lightweight Models

Key Design Strategies for Efficiency: quantization

In neural networks, we often store weights in floating-point. Quantization is a method that converts model weights from high-precision floating-point representation to low-precision floating-point (FP) or integer (INT) representations.



Lightweight Models

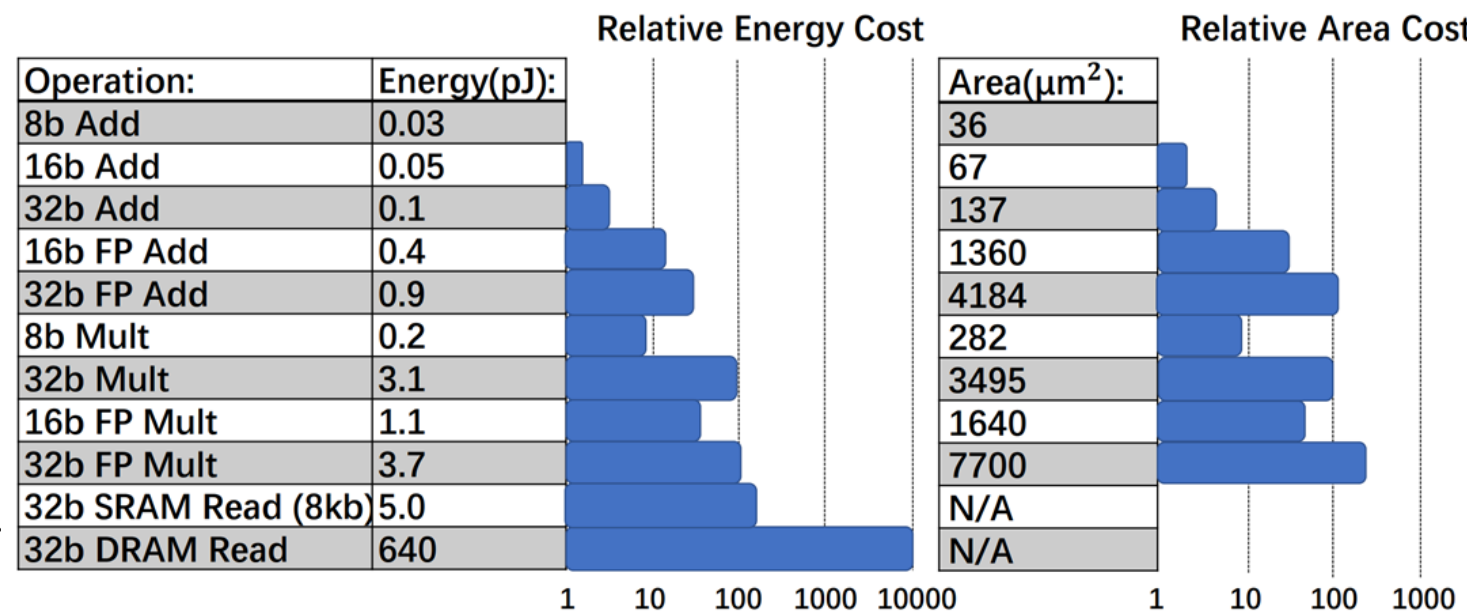
Key Design Strategies for Efficiency: quantization

Benefits when using quantization:

1.Reduced Memory Usage: Quantization reduces the amount of memory required to store the neural network model. For example, converting 32-bit floating-point weights to 8-bit integers can reduce the memory footprint by a factor of four.

2.Faster Computation: Lower precision arithmetic operations typically require less computational power and can be executed faster on many hardware platforms.

3.Lower Power Consumption: Significant reduction in energy for computations and memory access.

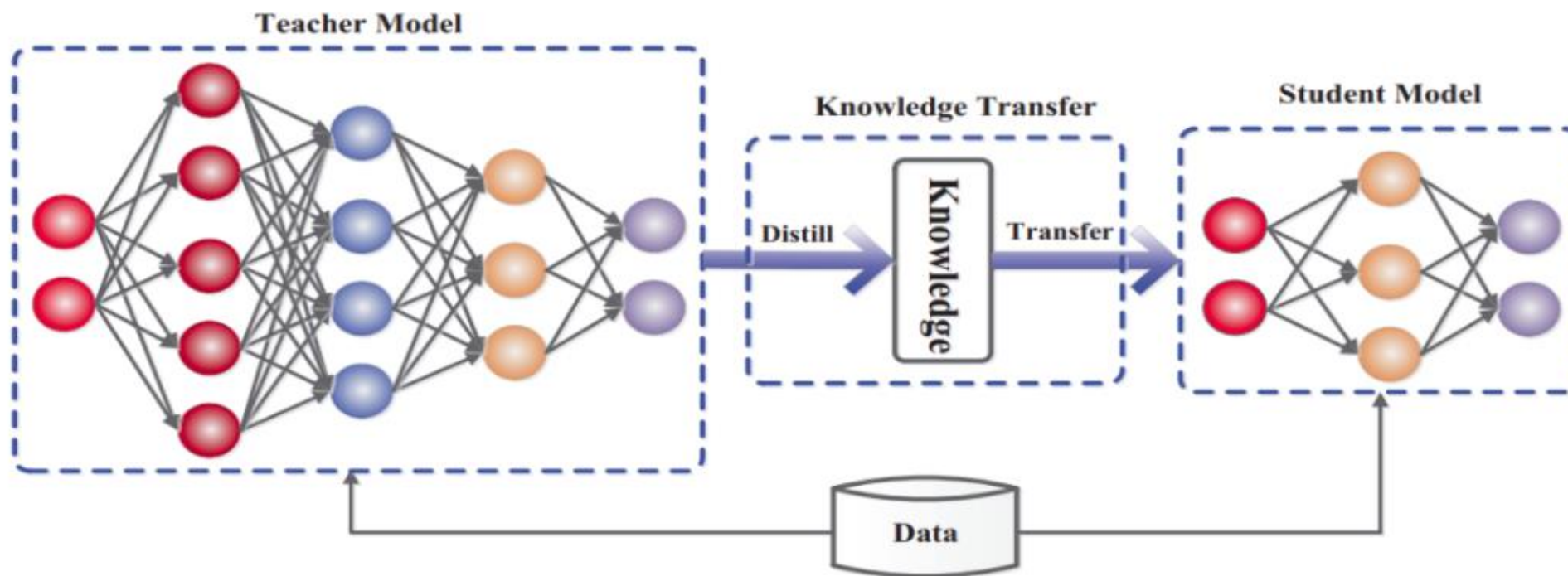


Energy cost and relative area cost for different precision in Titan RTX GPU

Lightweight Models

Key Design Strategies for Efficiency: knowledge distillation

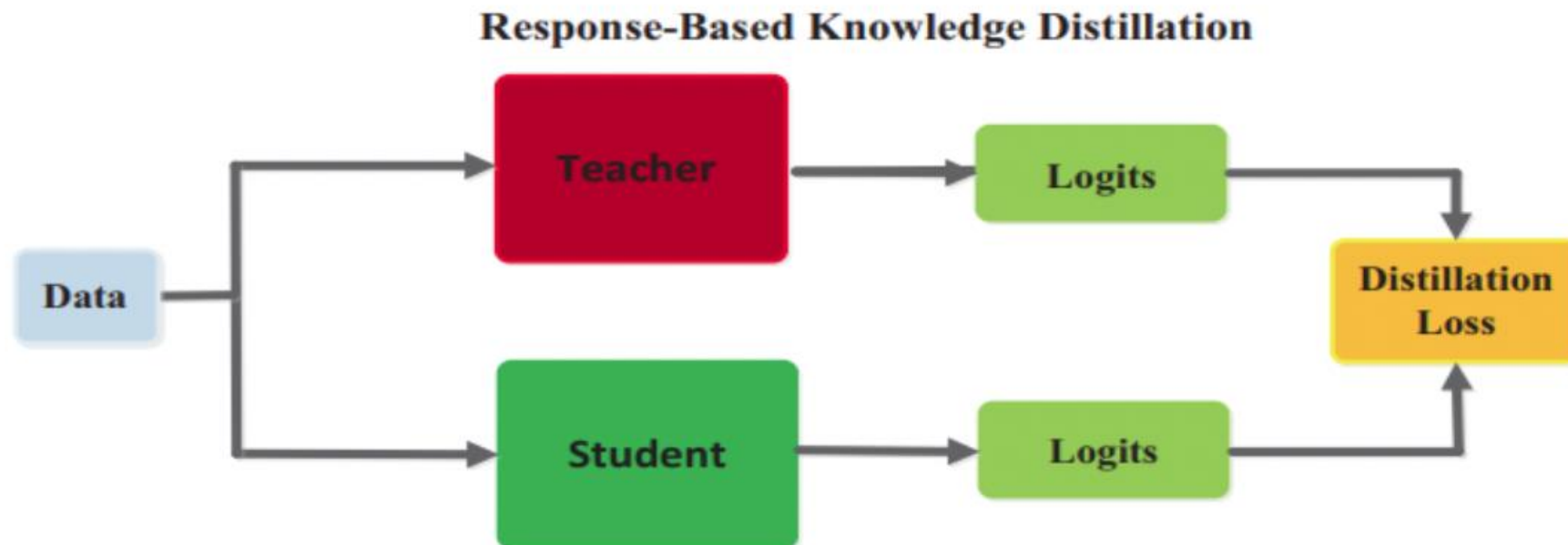
Knowledge distillation refers to the process of transferring knowledge from a large model to a smaller one



Lightweight Models

Key Design Strategies for Efficiency: knowledge distillation

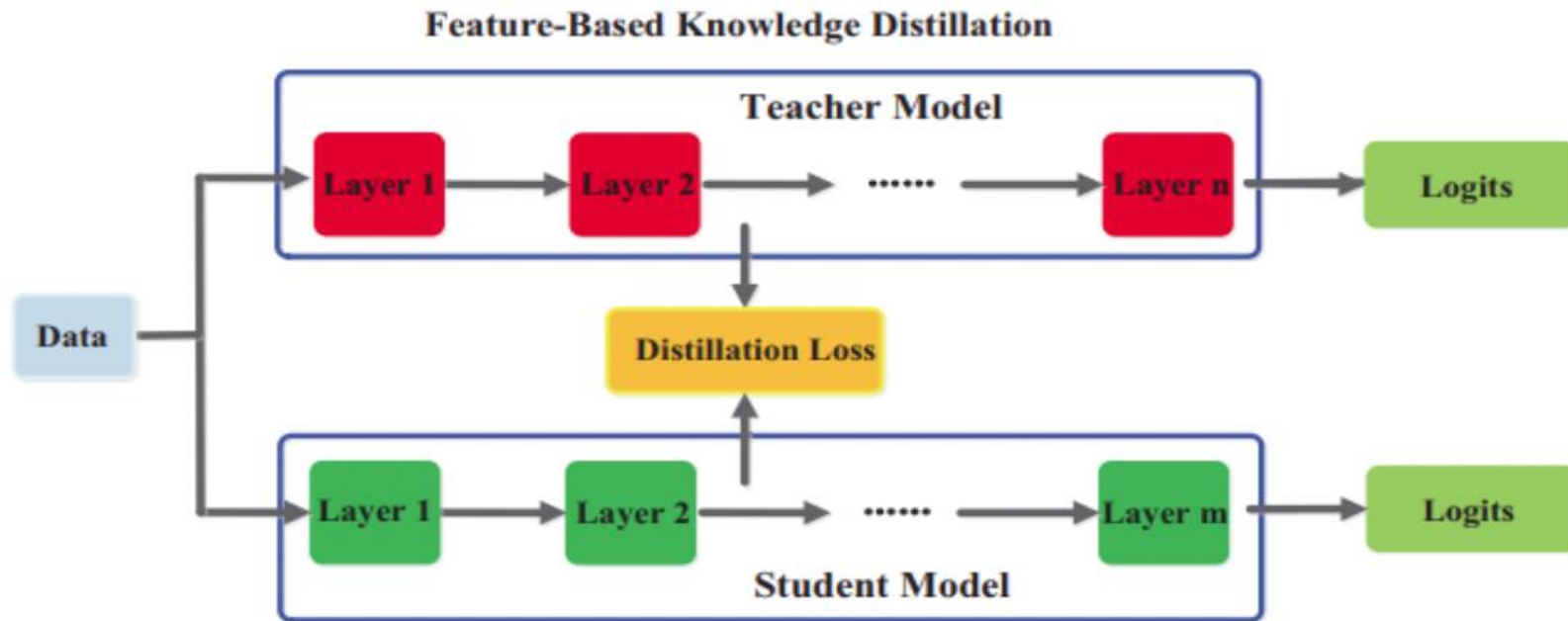
During training, the loss function will not only consider the difference between the output of the distilled model and the ground truth labels but also the difference between the output of the distilled model and the output of the CNN for the same input.



Lightweight Models

Key Design Strategies for Efficiency: knowledge distillation

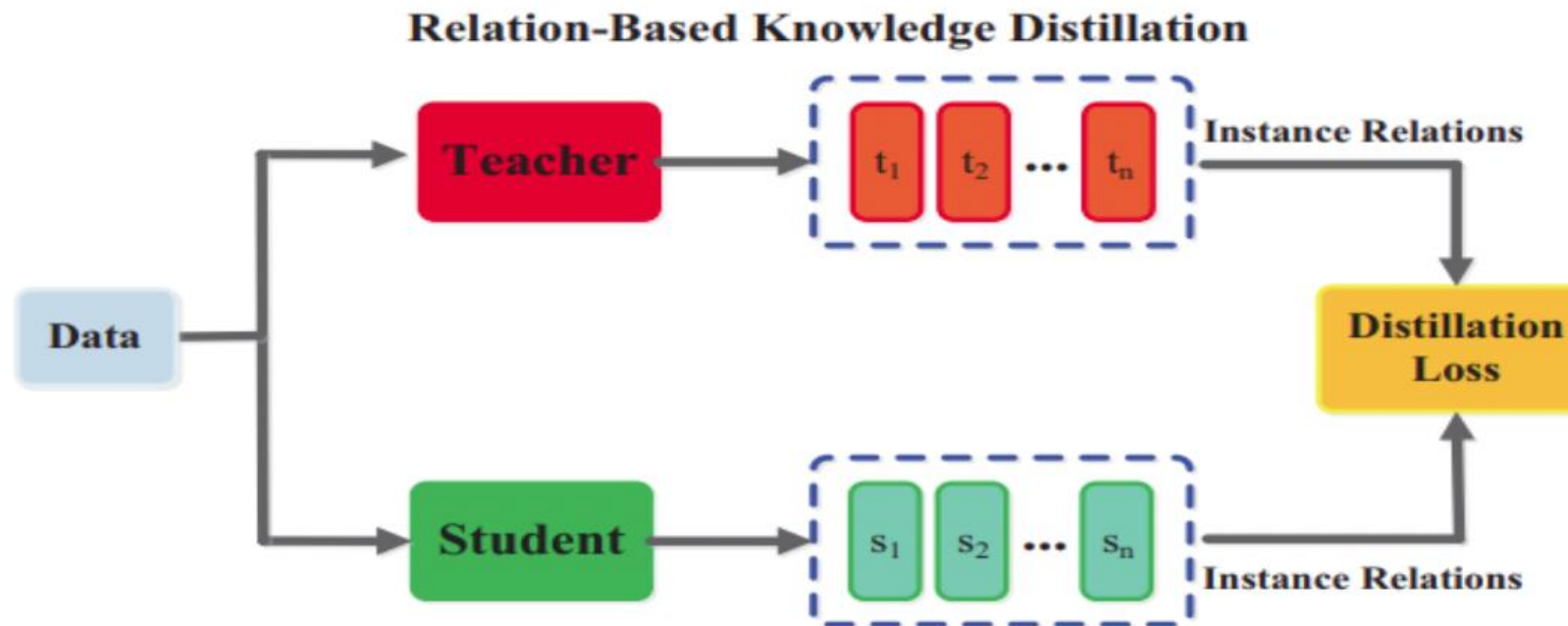
During training, the loss function will not only consider the difference between the output of the distilled model and the ground truth labels but also the difference between the output of the distilled model and the output of the CNN for the same input.



Lightweight Models

Key Design Strategies for Efficiency: knowledge distillation

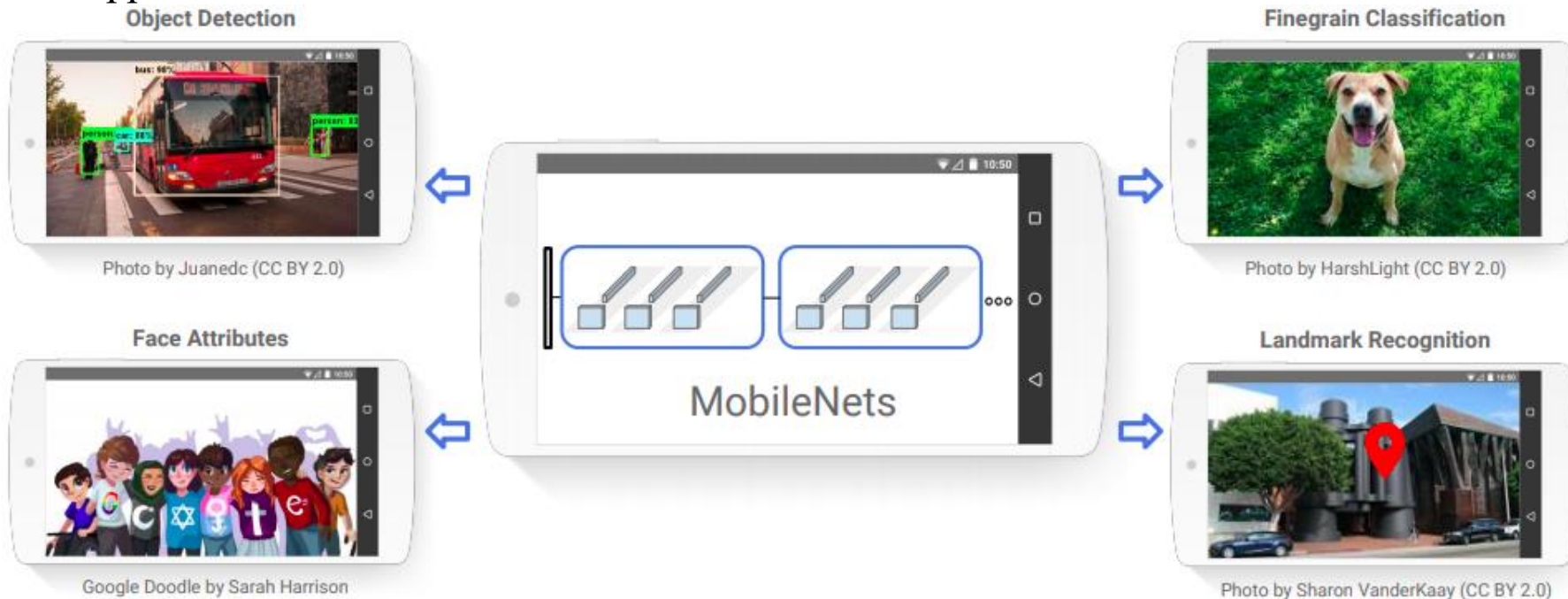
During training, the loss function will not only consider the difference between the output of the distilled model and the ground truth labels but also the difference between the output of the distilled model and the output of the CNN for the same input.



Lightweight Models – Key Architectures

MobileNetV1/V2/V3 (Depthwise convs, inverted residuals)

MobileNet is a simple but efficient and not very computationally intensive convolutional neural networks for mobile vision applications



Reading: Andrew G. Howard et al., MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications

Lightweight Models – Key Architectures

MobileNetV1/V2/V3 (Depthwise convs, inverted residuals)

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size	
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$	
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$	
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$	
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$	
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$	
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$	
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$	
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$	
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$	
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$	
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$	
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$	
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$	
5×	Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
	Conv / s1	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
	Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
	Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
	Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
	Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$	
FC / s1	1024×1000	$1 \times 1 \times 1024$	
Softmax / s1	Classifier	$1 \times 1 \times 1000$	

Table 8. MobileNet Comparison to Popular Models

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
GoogleNet	69.8%	1550	6.8
VGG 16	71.5%	15300	138

Table 4. Depthwise Separable vs Full Convolution MobileNet

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
Conv MobileNet	71.7%	4866	29.3
MobileNet	70.6%	569	4.2

Lightweight Models – Key Architectures

MobileNetV2

Key Idea: *Inverted Residuals with Linear Bottlenecks*

Inverted Residual Blocks:

- Expands channels (1×1 conv) $\rightarrow$ depthwise conv $\rightarrow$ projects back (1×1 linear).
- **No ReLU in bottleneck** (avoids information loss in low-dimensional space).

Advantages:

- More efficient than V1 (~ 3.4 M params for V2-1.0).
- Better accuracy due to residual connections.

Limitations:

- Still struggles with very small models (< 1 M params).

MobileNetV3 (2019)

Key Idea: *Neural Architecture Search (NAS) + Hardware-Aware Design*

Two Versions:

- **MobileNetV3-Large** (higher accuracy).
- **MobileNetV3-Small** (extremely efficient).

Key Improvements:

- **Squeeze-and-Excitation (SE)** blocks for channel attention.
- **Hardware-friendly activation:** h-swish (ReLU6 + sigmoid approximation).
- **NAS optimization** for FLOPs/accuracy trade-off.

Advantages:

- **20% faster** than V2 with similar accuracy.

Lightweight Models – Key Architectures

ShuffleNet (Channel shuffling for cheap computation)

The ShuffleNet utilizes **pointwise group convolution** and **channel shuffle** to reduce computation cost while maintaining accuracy

Depthwise separable convolution save a lot of computation cost from standard convolution, but Depthwise separable convolution has the majority of its computations concentrated in the pointwise convolution. For example, in MobileNetV1, 75% of the parameters and 95% of the computation time are spent on the pointwise convolution.

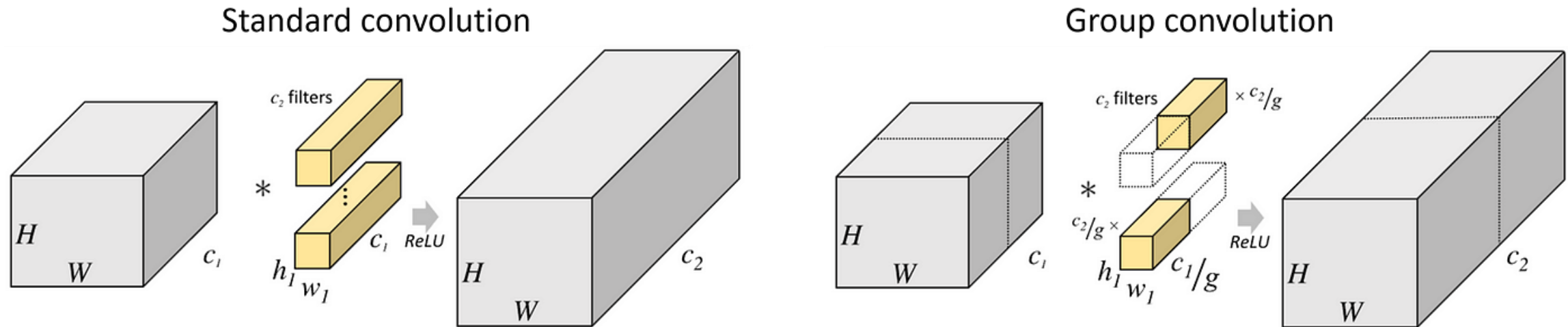
Table 2. Resource Per Layer Type

Type	Mult-Adds	Parameters
Conv 1×1	94.86%	74.59%
Conv DW 3×3	3.06%	1.06%
Conv 3×3	1.19%	0.02%
Fully Connected	0.18%	24.33%

Lightweight Models – Key Architectures

ShuffleNet (Channel shuffling for cheap computation)

The ShuffleNet utilizes **pointwise group convolution** and **channel shuffle** to reduce computation cost while maintaining accuracy



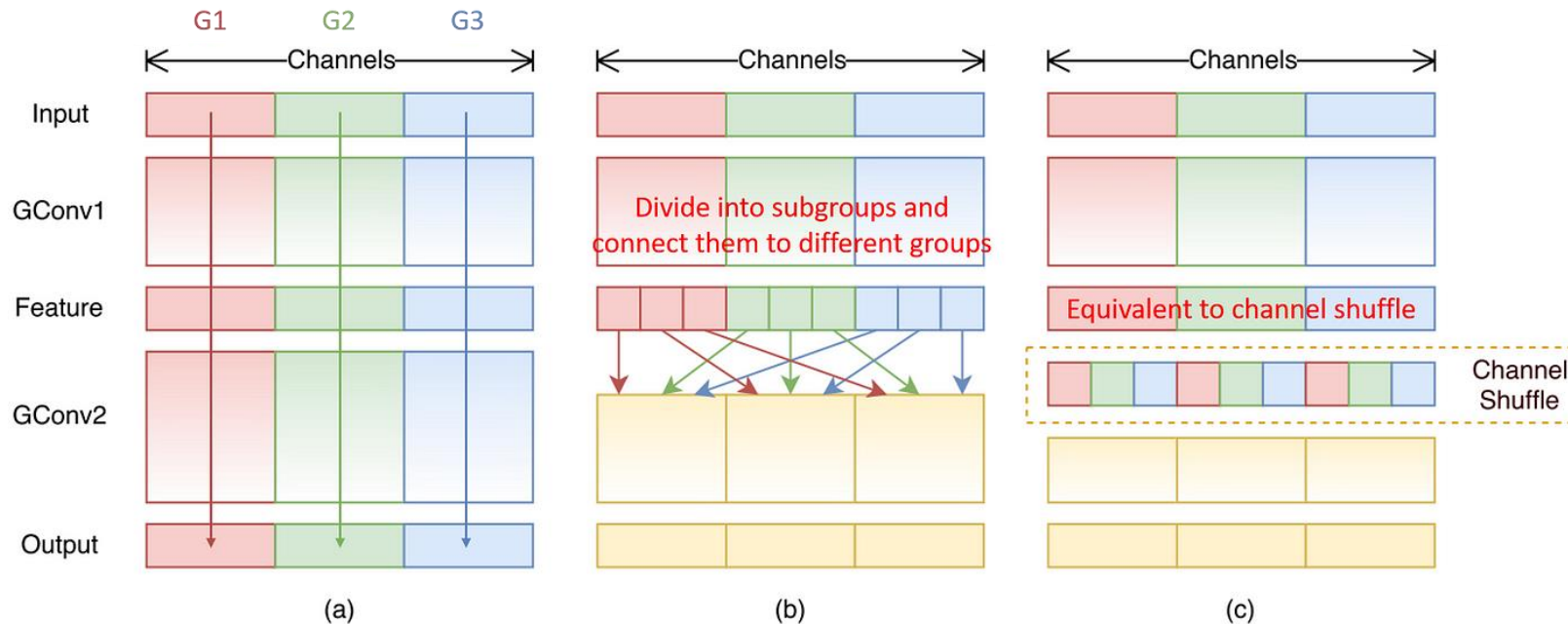
$$\left(K \times K \times \frac{M}{g} \times F \times F \times \frac{N}{g} \right) \times g = \frac{1}{g} (K \times K \times M \times F \times F \times N)$$

Group convolution cost is only 1/g of the standard convolution

Lightweight Models – Key Architectures

The ShuffleNet utilizes **pointwise group convolution** and **channel shuffle** to reduce computation cost while maintaining accuracy

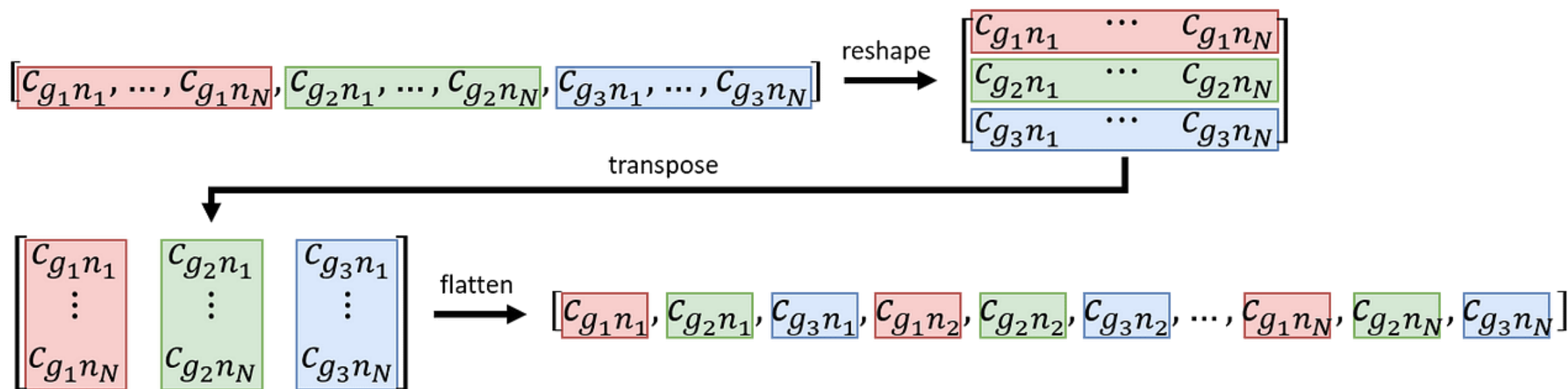
In group convolution, the information between different groups is completely isolated. The convolution kernels only operate within their respective groups, and the channels remain non-interacting when concatenated, which limits the model's representational capacity.



Channel shuffle enables information exchange between groups.

Lightweight Models – Key Architectures

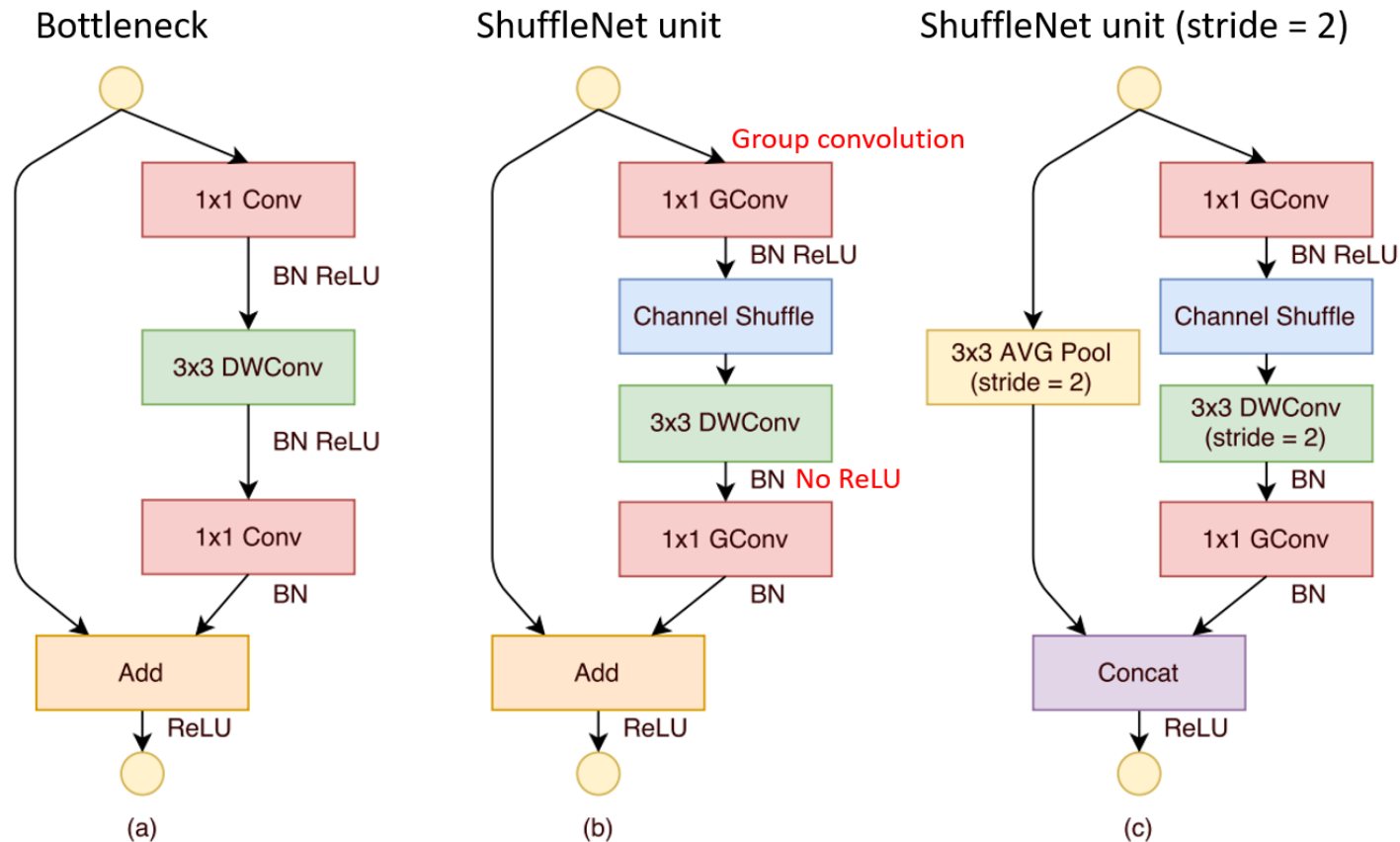
The ShuffleNet utilizes **pointwise group convolution** and **channel shuffle** to reduce computation cost while maintaining accuracy



The implementation of channel shuffle

Lightweight Models – Key Architectures

The ShuffleNet utilizes **pointwise group convolution** and **channel shuffle** to reduce computation cost while maintaining accuracy



(a) Bottleneck; (b) ShuffleNet unit; (c) ShuffleNet unit (stride = 2)

Lightweight Models – Key Architectures

The ShuffleNet utilizes **pointwise group convolution** and **channel shuffle** to reduce computation cost while maintaining accuracy

Error rate decreases as # groups increases

Model	Complexity (MFLOPs)	Classification error (%)				
		$g = 1$	$g = 2$	$g = 3$	$g = 4$	$g = 8$
ShuffleNet 1x	140	33.6	32.7	32.6	32.8	32.4
ShuffleNet 0.5x	38	45.1	44.4	43.2	41.6	42.3
ShuffleNet 0.25x	13	57.1	56.8	55.0	54.2	52.7

Lower error rate with channel shuffle

Model	Cls err. (% , no shuffle)	Cls err. (% , shuffle)	Δ err. (%)
ShuffleNet 1x ($g = 3$)	34.5	32.6	1.9
ShuffleNet 1x ($g = 8$)	37.6	32.4	5.2
ShuffleNet 0.5x ($g = 3$)	45.7	43.2	2.5
ShuffleNet 0.5x ($g = 8$)	48.1	42.3	5.8
ShuffleNet 0.25x ($g = 3$)	56.3	55.0	1.3
ShuffleNet 0.25x ($g = 8$)	56.5	52.7	3.8

Lightweight Models – Key Architectures

The ShuffleNet utilizes **pointwise group convolution** and **channel shuffle** to reduce computation cost while maintaining accuracy

Table 4: Classification error vs. various structures (% , smaller number represents better performance)

Complexity (MFLOPs)	VGG-like ⁴	ResNet	Xception-like	ResNeXt	ShuffleNet (ours)
140	56.0	38.7	35.1	34.3	34.1 ($1\times, g=3$)
38	-	48.9	46.1	46.3	43.7 ($0.5\times, g=4$)
13	-	61.6	56.7	59.2	53.7 ($0.25\times, g=8$)
40 (arch2)	-	48.5	45.7	47.2	42.7 ($0.5\times, g=8$)
13 (arch2)	-	61.3	56.5	61.0	53.3 ($0.25\times, g=8$)

Table 6: Complexity comparison

Model	Cls err. (%)	Complexity (MFLOPs)
VGG-16 [27]	28.5	15300
ShuffleNet $2\times (g=3)$	29.1	524
PVANET [18] (<i>our impl.</i>)	35.3	557
ShuffleNet $1\times (g=3)$	34.1	140
AlexNet [19]	42.8	720
SqueezeNet [13]	42.5	833
ShuffleNet $0.5\times$ (arch2, $g=8$)	42.7	40

Table 5: ShuffleNet vs. MobileNet [12] on ImageNet Classification

Model	Complexity (MFLOPs)	Cls err. (%)	Δ err. (%)
1.0 MobileNet-224	569	29.4	-
ShuffleNet $2\times (g=3)$	524	29.1	0.3
0.75 MobileNet-224	325	31.6	-
ShuffleNet $1.5\times (g=3)$	292	31.0	0.6
0.5 MobileNet-224	149	36.3	-
ShuffleNet $1\times (g=3)$	140	34.1	2.2
0.25 MobileNet-224	41	49.4	-
ShuffleNet $0.5\times$ (arch2, $g=8$)	40	42.7	6.7
ShuffleNet $0.5\times$ (shallow, $g=3$)	40	45.2	4.2

Lightweight Models – Key Architectures

SqueezeNet (Fire modules, <1MB models)

Strategy 1: Make the network smaller by replacing 3x3 filters with 1x1 filters (1x1 filter has 9X fewer parameters than a 3x3 filter)

Strategy 2: Reduce the number of inputs for the remaining 3x3 filters (fewer inputs to conv layers result in fewer parameters, achieved by using only 1x1 filters prior to the 3x3 conv layer)

Strategy 3: Downsample late in the network so that convolution layers have large activation maps.

General Strategy: Strategies 1 and 2 are about carefully decreasing the quantity of parameters in a CNN while attempting to preserve accuracy. Strategy 3 is about maximizing accuracy on a limited budget of parameters.

Lightweight Models – Key Architectures

SqueezeNet (Fire modules, <1MB models)

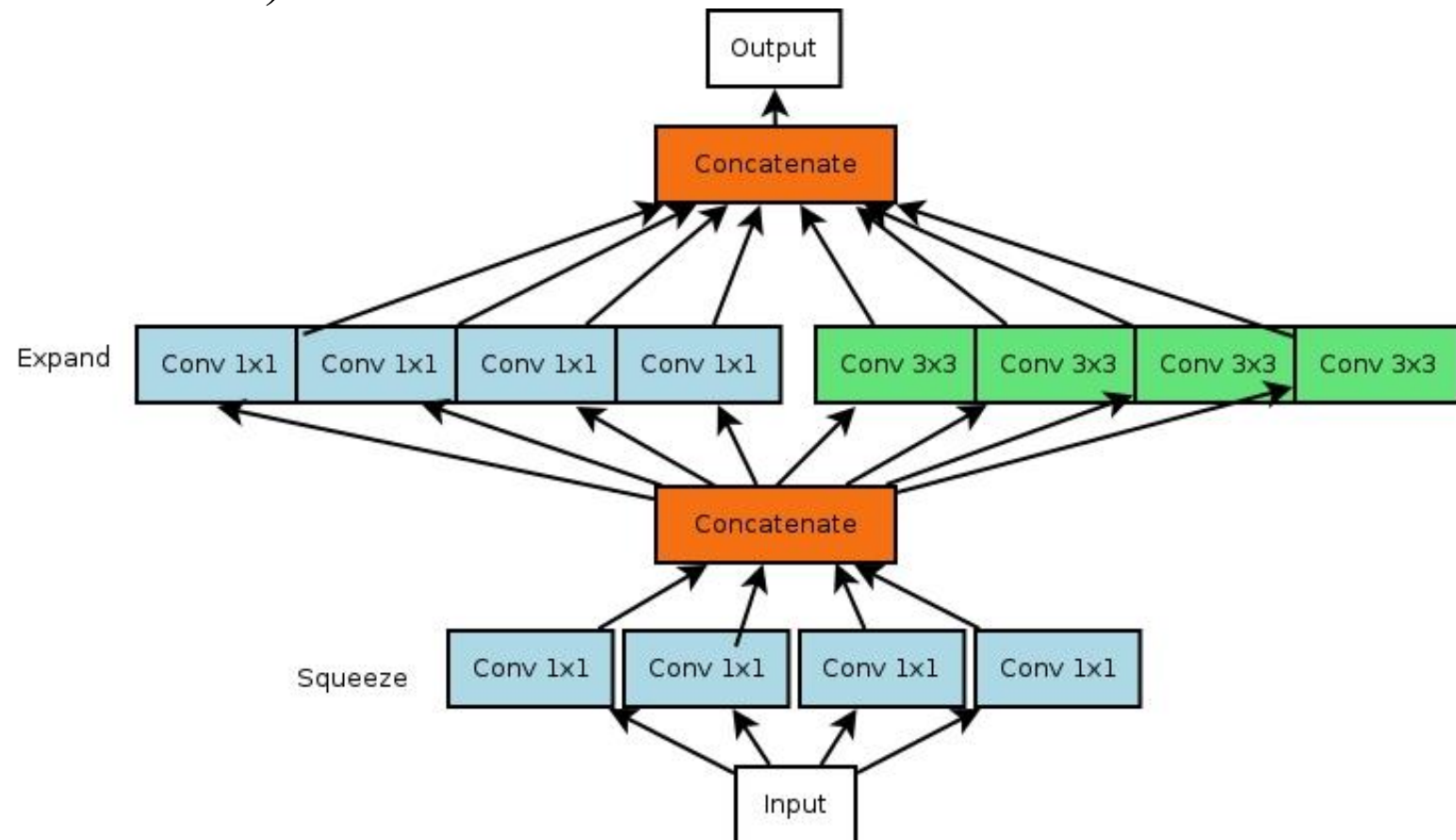
Fire Module

building block used in the SqueezeNet
employs Strategies 1, 2, and 3

comprised of squeeze layers which have
only 1x1 filters (strategy 1)

comprised of expand layers which have a
mix of 1x1 and 3x3 convolution filters

- number of filters in squeeze layer must be less than the expand layer (strategy 2)



Lightweight Models – Key Architectures

SqueezeNet (Fire modules, <1MB models)

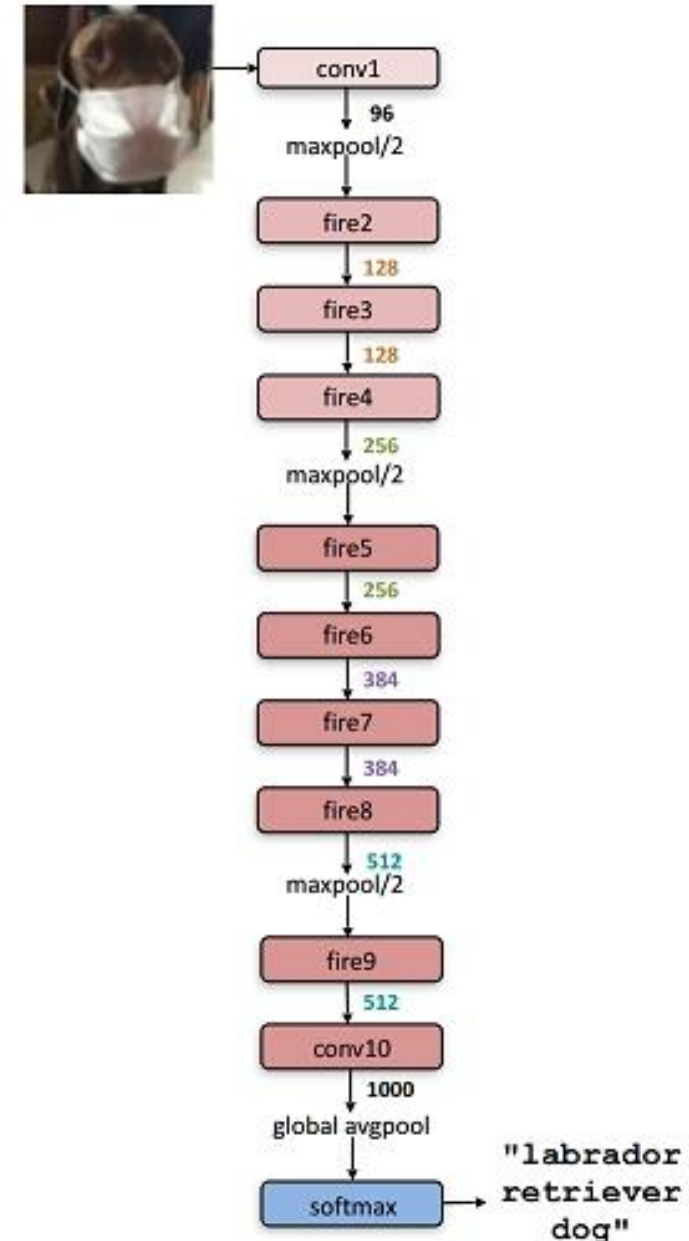
SqueezeNet Architecture

layer 1: regular convolution layer

layer 2-9: fire module (squeeze +
expand layer)

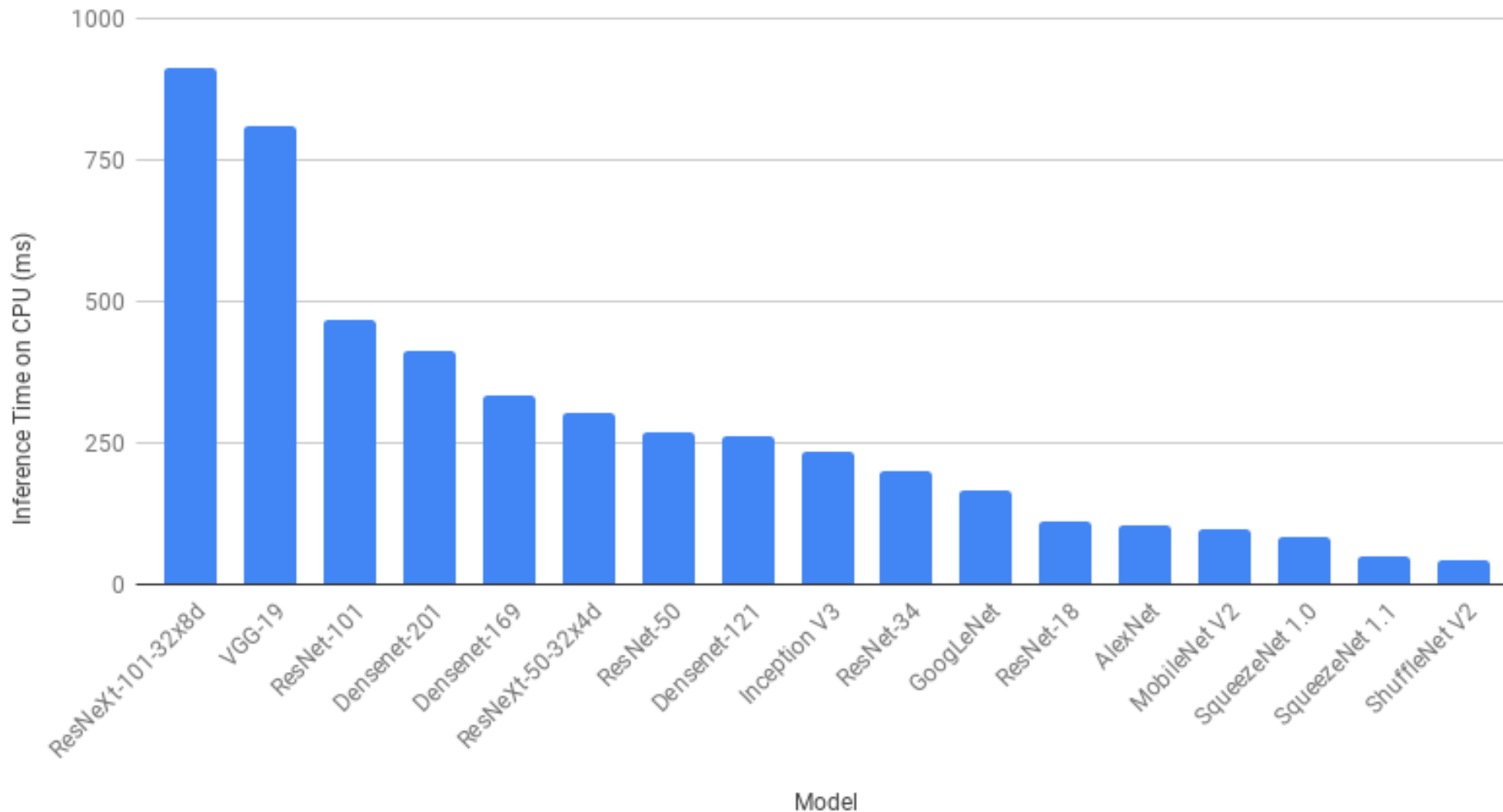
layer 10: regular convolution layer

layer 11: softmax layer



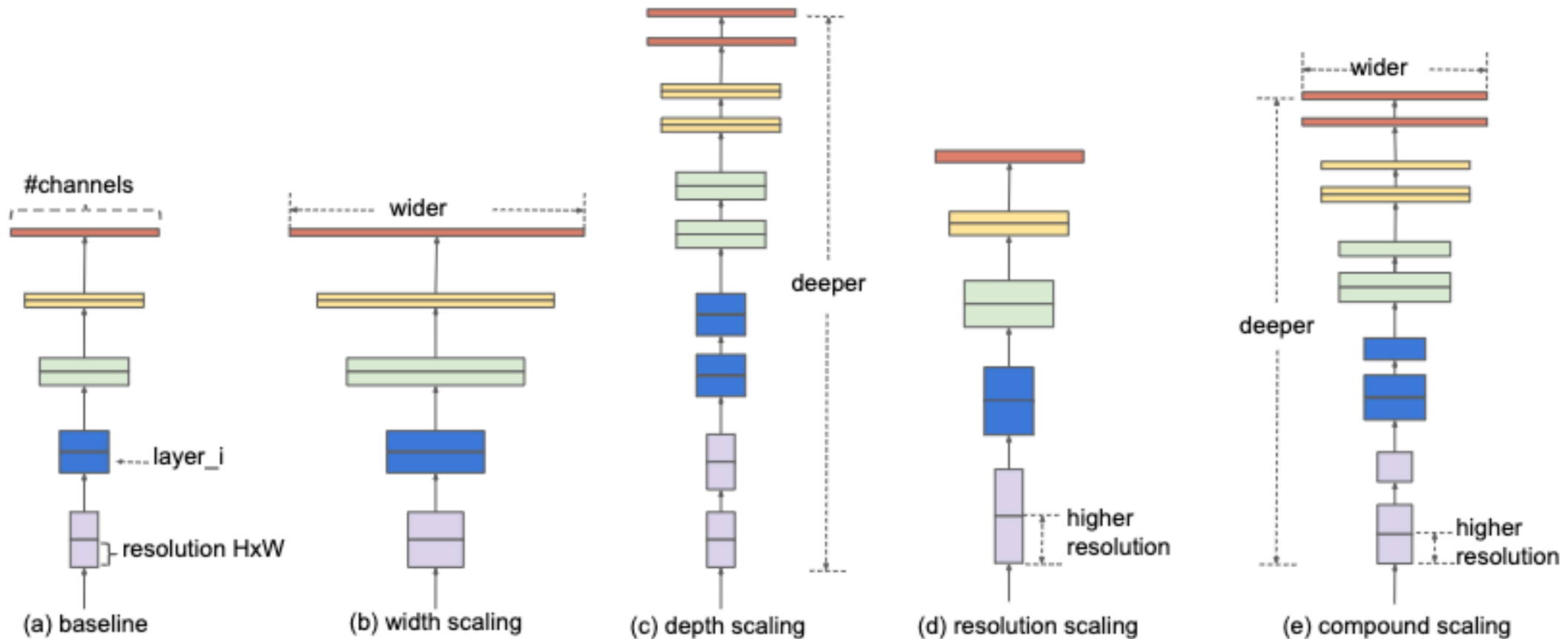
Lightweight Models – Key Architectures

Model Inference Time Comparison on CPU (ms) (Lower is better)



Lightweight Models – Key Architectures

EfficientNet-Lite (Optimized for edge)



Lightweight Models – Key Architectures

EfficientNet-Lite (Optimized for edge)

EfficientNet models were slimmed down to perform better on edge devices. The following changes were made:

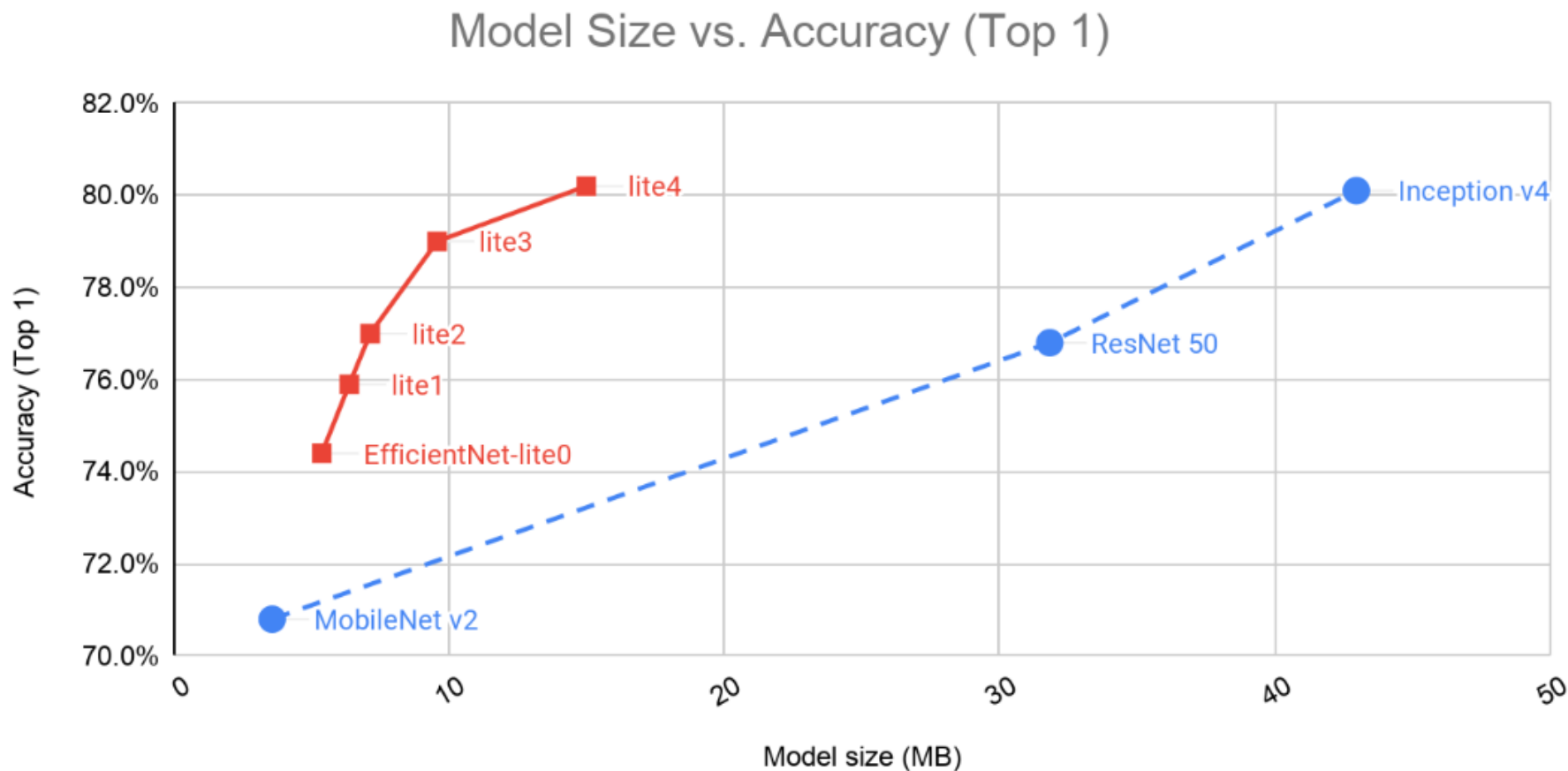
All the Squeeze and excite blocks were removed as they are not well supported by edge devices.

All swish activations were replaced with ReLU6.

The stem and the head layers were fixed (kept the same) while scaling models up. This was to ensure that the models could be kept small and fast.

Lightweight Models – Key Architectures

EfficientNet-Lite (Optimized for edge)



Lightweight Models - Summary

There's a trade-off between accuracy and computational cost (measured in FLOPs and memory usage). More accurate models often require more computational resources (higher FLOPs and memory), while simpler, more efficient models might sacrifice some accuracy. **However, advancements in network design and optimization techniques aim to minimize this trade-off, enabling higher accuracy with reasonable computational costs.**

Factors affecting accuracy: Network architecture (e.g., depth, width, use of specialized layers like depthwise separable convolutions), training data, and optimization strategies all play a crucial role.

FLOPs (Floating Point Operations): Indicates the computational complexity of a model during inference. Higher FLOPs mean more computations are required, leading to slower inference times, especially on resource-constrained devices.

Memory Usage: Refers to the amount of memory (RAM) required to store the model's parameters and activations during computation. Larger models with more parameters consume more memory.

QA Time